C. ABDUL HAKEEM COLLEGE OF ENGINEERING AND TECHNOLOGY

Hakeem Nagar, Melvisharam - 632 509, Ranipet District,Tamil Nadu, India. (Approved by AICTE, New Delhi and Affiliated to Anna University, Chennai)

(Regd. Under Sec 2(F) & 12(B) of the UGC Act 1956)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATASCIENCE

CCS335 – CLOUD COMPUTING RECORD

(REGULATION - 2021)

Name of the Student:

Register Number:

Degree / Branch:

Year / Semester:

Academic Year:

C. ABDUL HAKEEM COLLEGE OF ENGINEERING AND TECHNOLOGY

Hakeem Nagar, Melvisharam - 632 509, Ranipet District,Tamil Nadu, India

(Approved by AICTE, New Delhi and Affiliated to Anna University, Chennai)

Regd. Under Sec 2(f) & 12(B) of the UGC Act 1956)

Name of the Candidate:

Year: III **Semester: V** **Degree/Branch: B.TECH/AI&DS**

Subject Code: CCS335

Subject Name: CLOUD COMPUTING LABORATORY

University Register Number:

CERTIFICATE

Certified that this is the bonafide record of work done by the above student in

CCS335-CLOUD COMPUTING LABORATORY

during 2026 - 2027.

Signature of Lab In-charge **Signature of Head of Department**

Submitted for the University Practical Examination held on _____

EXAMINERS

Date: _____ **Centre code: 5106**

Internal: _____ **External: _____**

INDEX

EXP NO.	DATE	EXPERIMENTS	PAGE NO	MARK	SIGNATURE
1.		Install virtualbox with Linux OS on the top of the windows host operating system			
2.		Install a C compiler in the Virtual Machine			
3.		Install Google app Engine Create Hello World App and other simple web Application using python/java			
4.		Use the GAE Launcher to Launch the web application			
5.		Simulate a cloud scenario using cloudsim and run a scheduling algorithm that is not present in cloudsim			
6.		Find a procedure to transfer the files from one virtual machine to another virtual machine			
7.		Install hadoop single node cluster and run date simple application like word count			
8.		Creating and executing your first container using Docker			
9.		Run a container from Docker hub			

EX.NO-1: **INSTALL VIRTUAL BOX WITH LINUX OS ON TOP OF WINDOWS**

DATE :

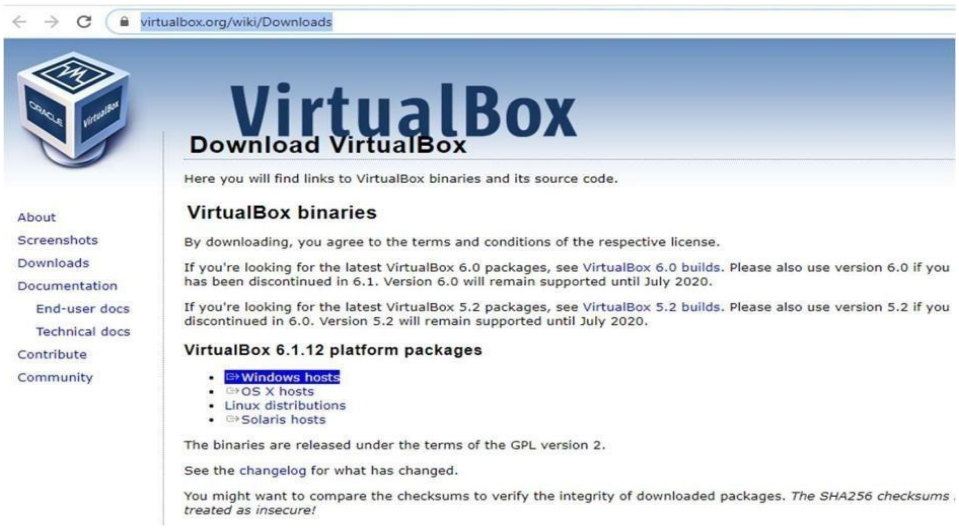
AIM:

To install Virtual box with Ubuntu OS on top of windows host operating system.

PROCEDURE:

Steps to install Virtual Box:

1. Download Virtual Box installer for windows.
2. The installer can be downloaded from the link https://www.virtualbox.org/wiki/Downloads



3. Click “Windows host” to download the binary version for windows host.
4. The installer file downloaded will have the file name format like Virtual Box-VersionNumber-BuildNumber-Win.exe.

   Example: VirtualBox-6.1.12-139181-Win.exe.

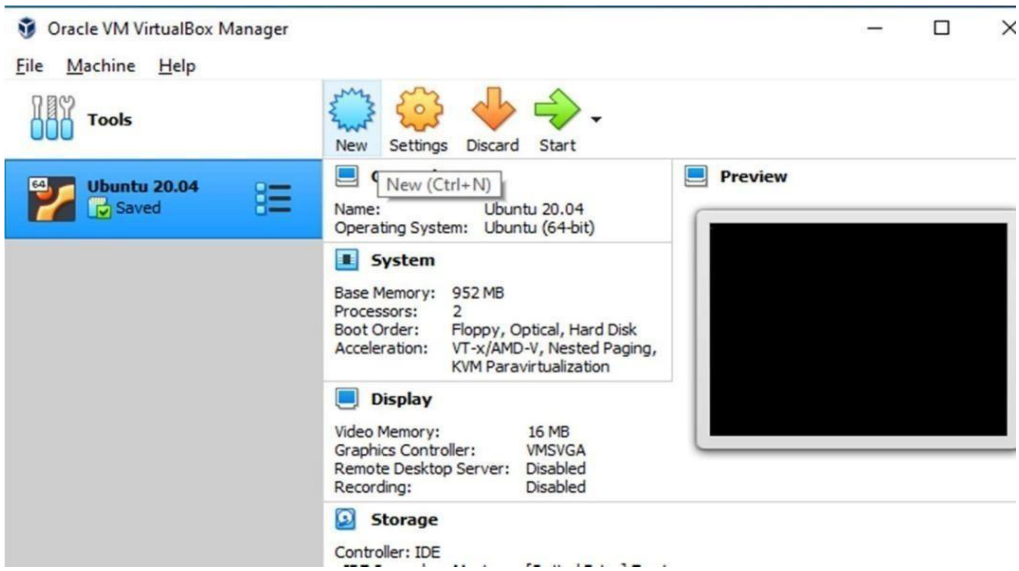
5. Double click on the installer to launch the setup Wizard. Click on Next to continue.



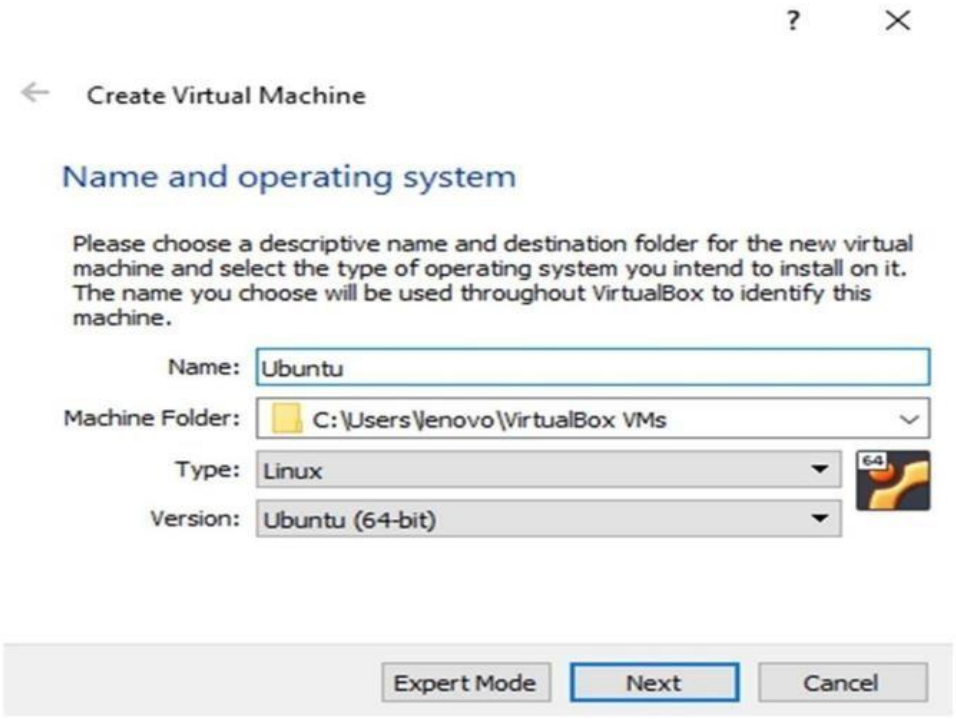
6. Custom setup dialog box will be opened. Accept the default settings and click next.
7. Select the way you want the features to be installed. You can accept the default and click next.
8. A dialog box opens with Network Interfaces warning. Click Yes to proceed.
9. Click install to begin the installation process.
10. When prompted with a message to install (Trust) Oracle Universal Serial Bus, click Install to continue.
11. After the installation completes, click finish to exit the setup wizard.
12. Launch the Oracle VM Virtual Box.

STEPS TO CREATE A VIRTUAL MACHINE [UBUNTU] IN VIRTUALBOX:

1. Open the Oracle VM VirtualBox.
2. Click New icon or Ctrl + N‟ to create a new virtual machine.



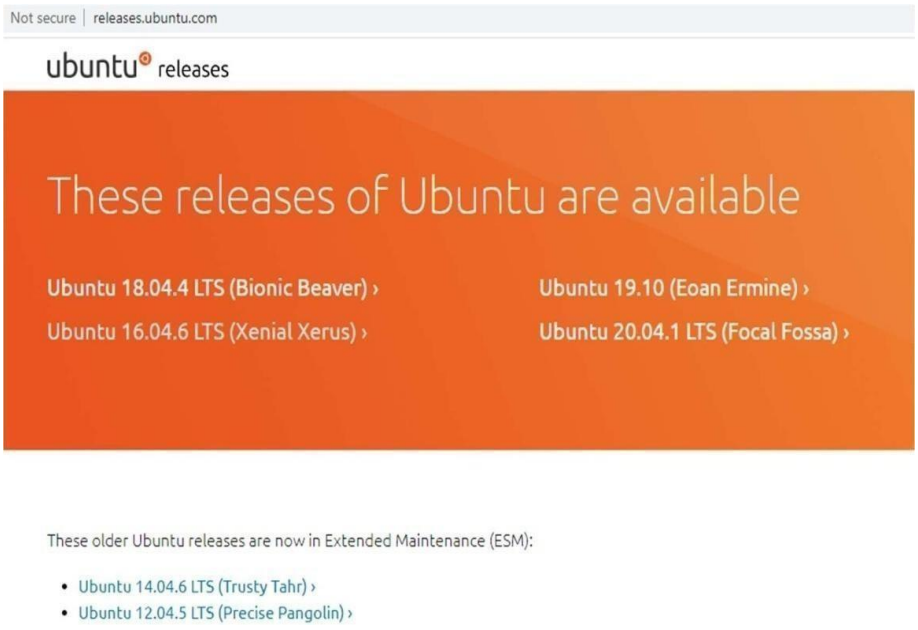
3. Enter a name for the new virtual machine. Choose the Type and Version. Note that Virtual-Box automatically changes 'Type' to Linux and 'Version' to 'Ubuntu (64 bit)' if the name is given as „Ubuntu". Click Next.



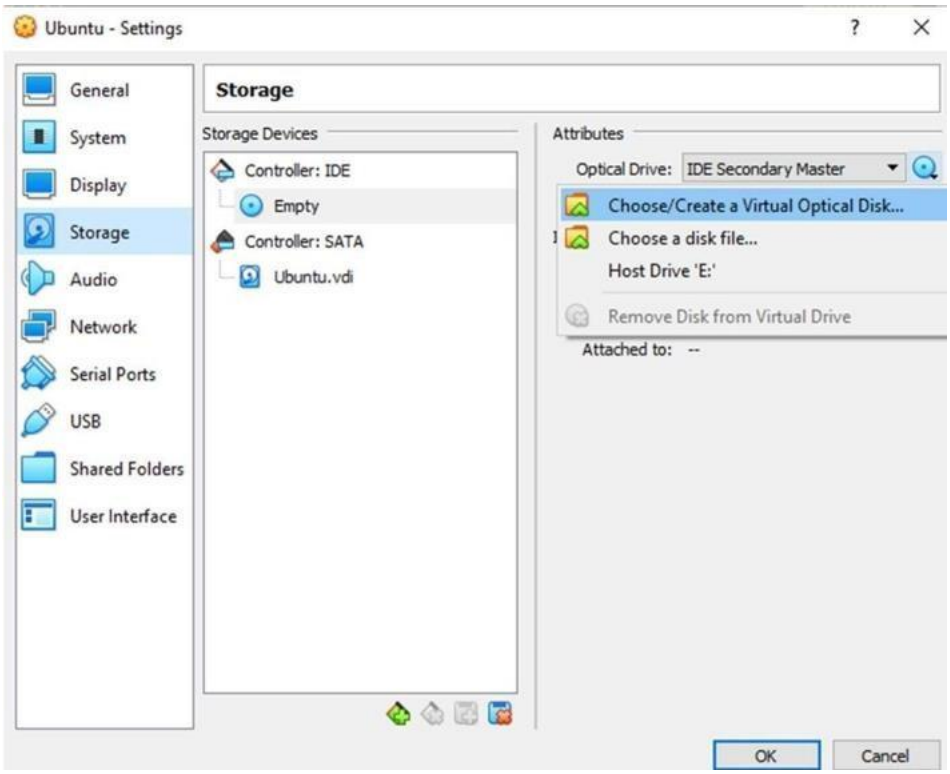
4. Select the amount of RAM to use. The ideal amount of RAM will automatically be selected. Do not increase the RAM into the red section of the slider; keep the slider in the green section.
5. Accept the default 'Create a virtual hard drive now' and click 'Create' button.
6. Choose the hard disk file type as VDI (VirtualBox Disk Image). Click Next.
7. Click Next to accept the default option „Dynamically allocated“ for storage on physical hard drive.
8. Select the size of the virtual hard disk and click create.
9. The newly created virtual machine will be displayed in the dashboard.
10. Download the ISO file [Ubuntu disk image file]. Latest version of Ubuntu iso file can be downloaded from the link h ttps://ubuntu.com/download/desktop. Click Download button.



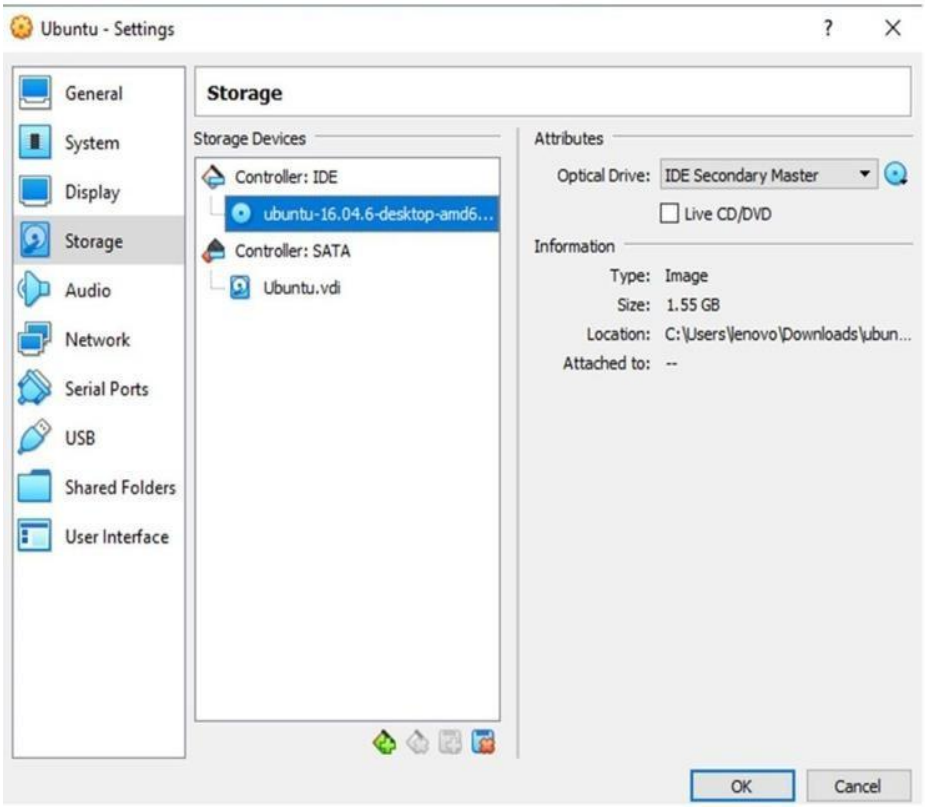
11. For previous versions, goto http://releases.ubuntu.com. Choose the preferred version of Ubuntu and download the iso file.



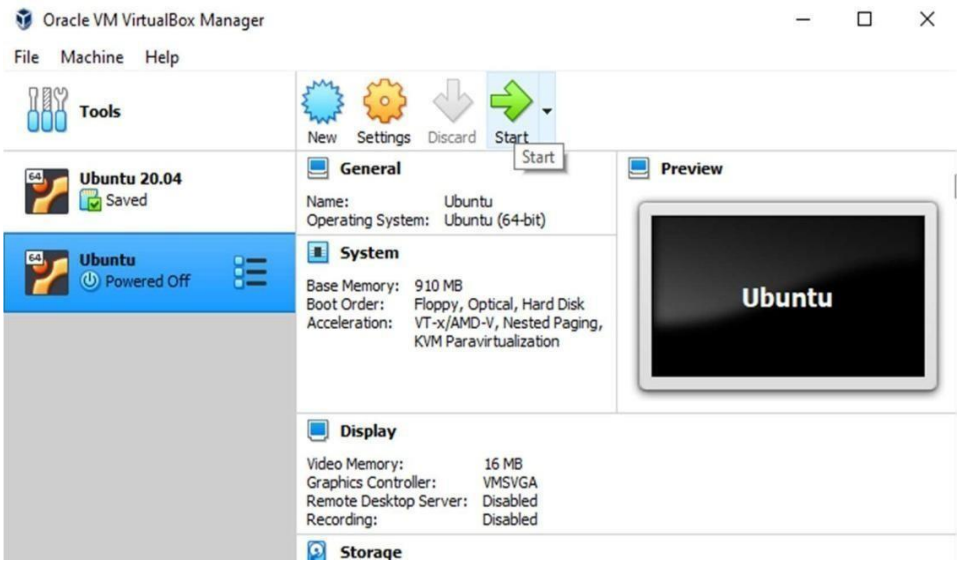
12. To setup the Ubuntu disk image file (iso file) goto settings.
13. Click Storage. Under „Storage Devices“ section click „Empty“.
14. In Attributes section, click the disk image and then "Choose Virtual Optical Disk File".



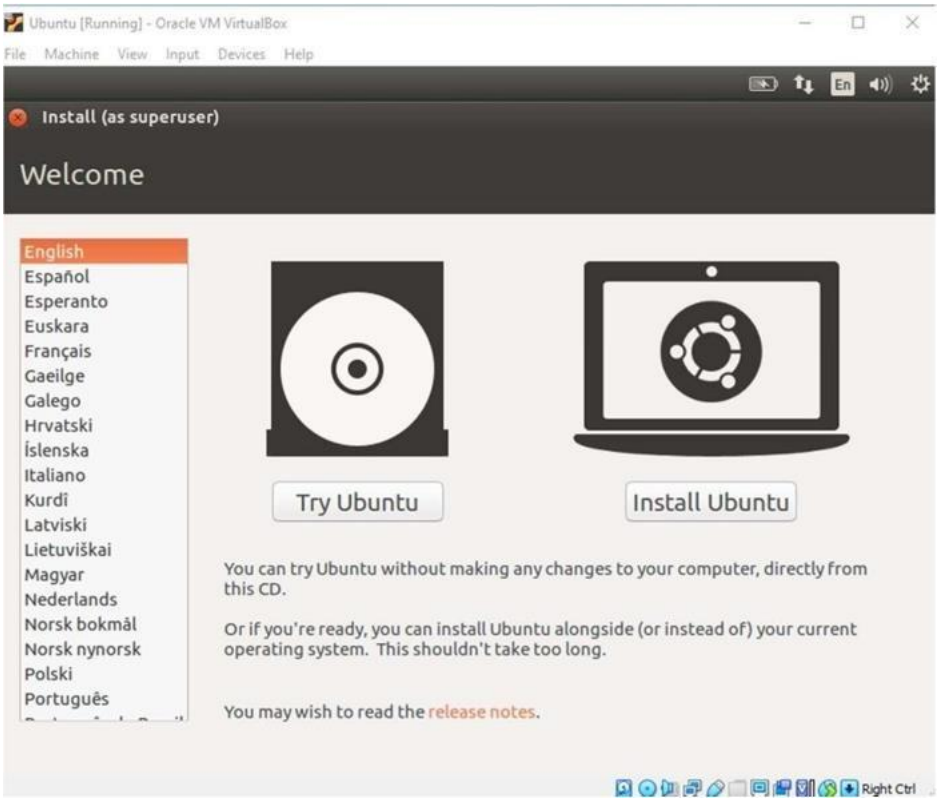
15. Browse and select the downloaded iso file. Click ok.



16. Select the newly created virtual machine in the dashboard and click start button.



17. In the welcome screen, click „Install Ubuntu“ button.



18. Click 'Continue' button.
19. Make sure 'Erase disk and install Ubuntu' option is selected and click 'Install Now' button.
20. Choose the default and click continue.
21. Setup up your profile by creating username and password.
22. After installation is complete, click 'Restart Now' button and follow the instructions.
23. The Ubuntu OS is ready to use. Login with the username and password.

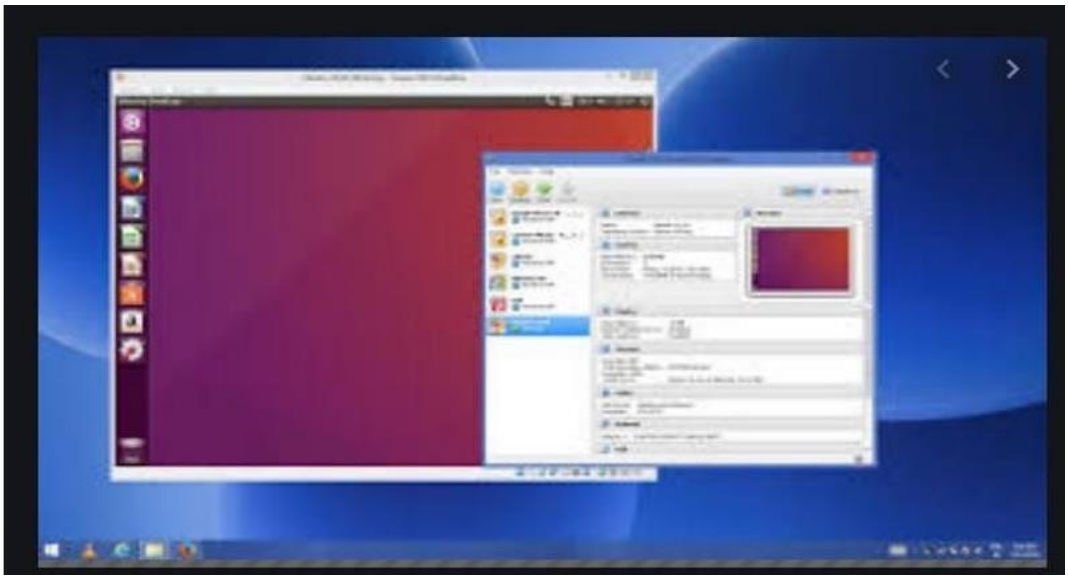
OUTPUT 1:

Virtual box on top of windows.



OUTPUT 2:

Installation of Virtual box with **Linux OS (Guest OS/VM)**on top of windows Host.



RESULT:

Thus the virtual box installation is completed and the Virtual machine is created on top of windows host operating system

EX.NO-2: **INSTALL A C COMPILER IN THE VIRTUAL MACHINE**
DATE :

AIM:

To install a C compiler in the virtual machine and execute a sample program.

PROCEDURE:

1. Launch the virtual box and open the virtual machine (Ubuntu).
2. Run the following command in the virtual machine terminal.

   $ sudo apt update

   $ sudo apt install gcc

   It will install all the necessary packages for gcc complier.
3. Type the C program in the text editor and save the file with with .c extension.

```
//demo.c
#include<stdio.h>
int main()
{
printf("Hello World");
      retur n 0;
 }
```

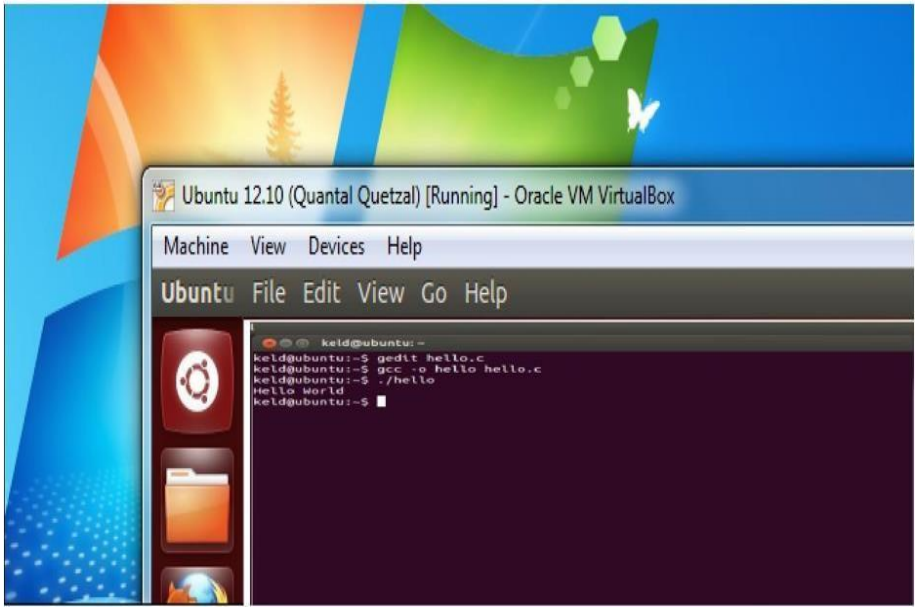
4. Compile and Run the C Program.

```
cc demo.c

./a.out
```

OUTPUT:

Installation of a C compiler in the virtual machine and executing a sample program.



RESULT:

Thus a C compiler is installed on a Ubuntu Virtual Machine on top of Windows Host and Executed a C program on a virtual machine.

EX.NO-3: **INSTALL GOOGLE APP ENGINE CREATE HELLO**
DATE: **WORLDAPP AND OTHER SIMPLE WEB APPLICATION USING PYTHON/JAVA.**

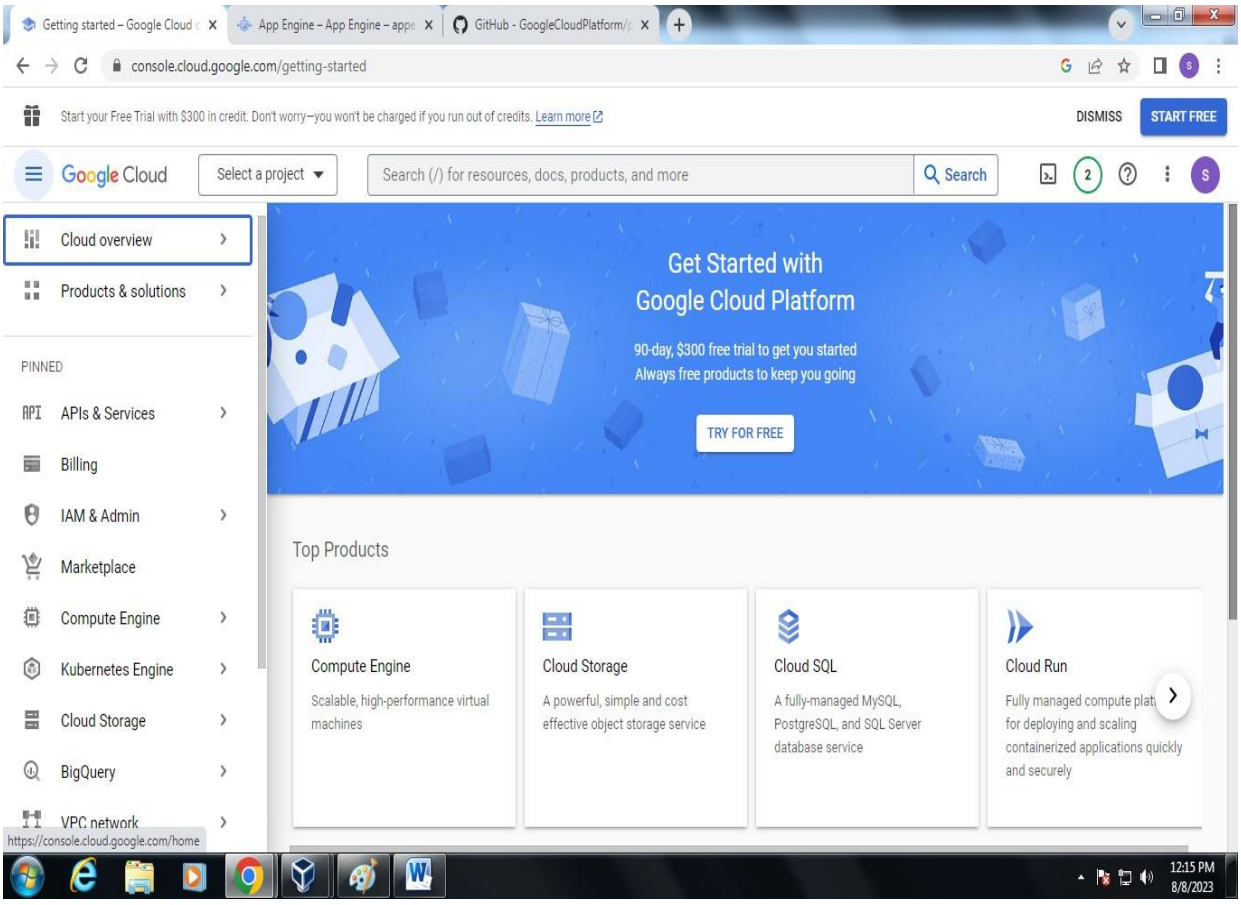
AIM:

To run the phthon sample program to display Helloworld in Google App Engine in google cloud console.

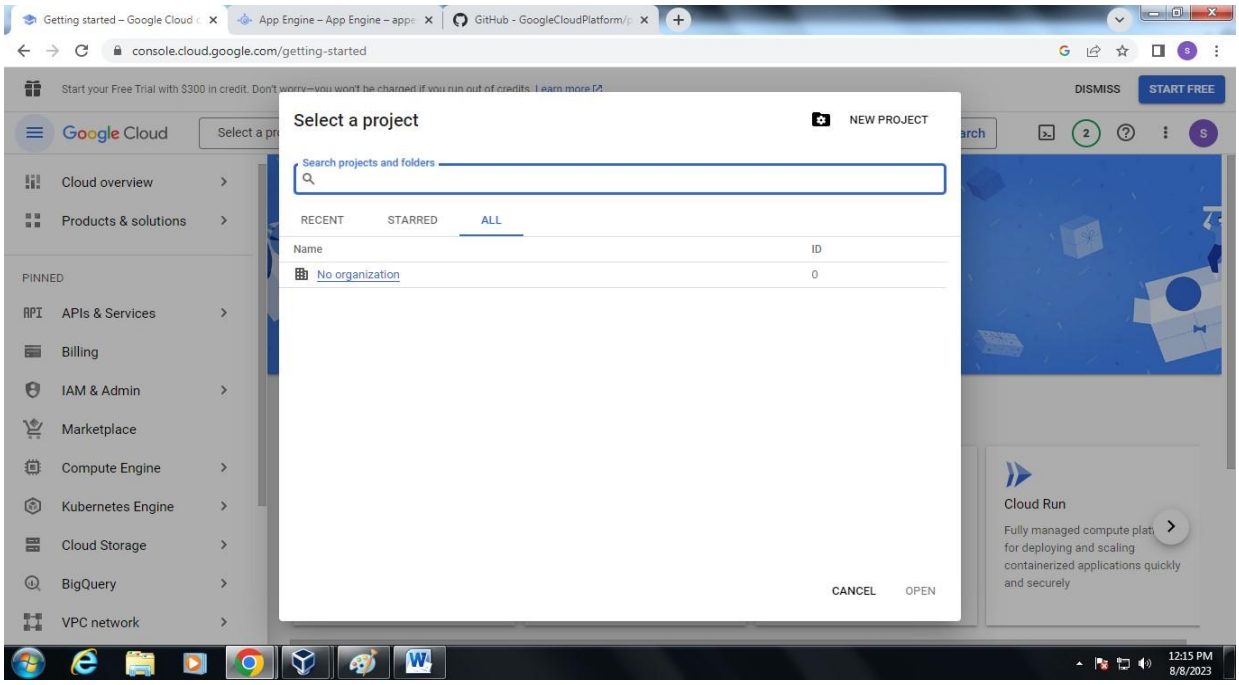
PROCEDURE:

1.Go to the link in chrome “console.cloud.google.com”.

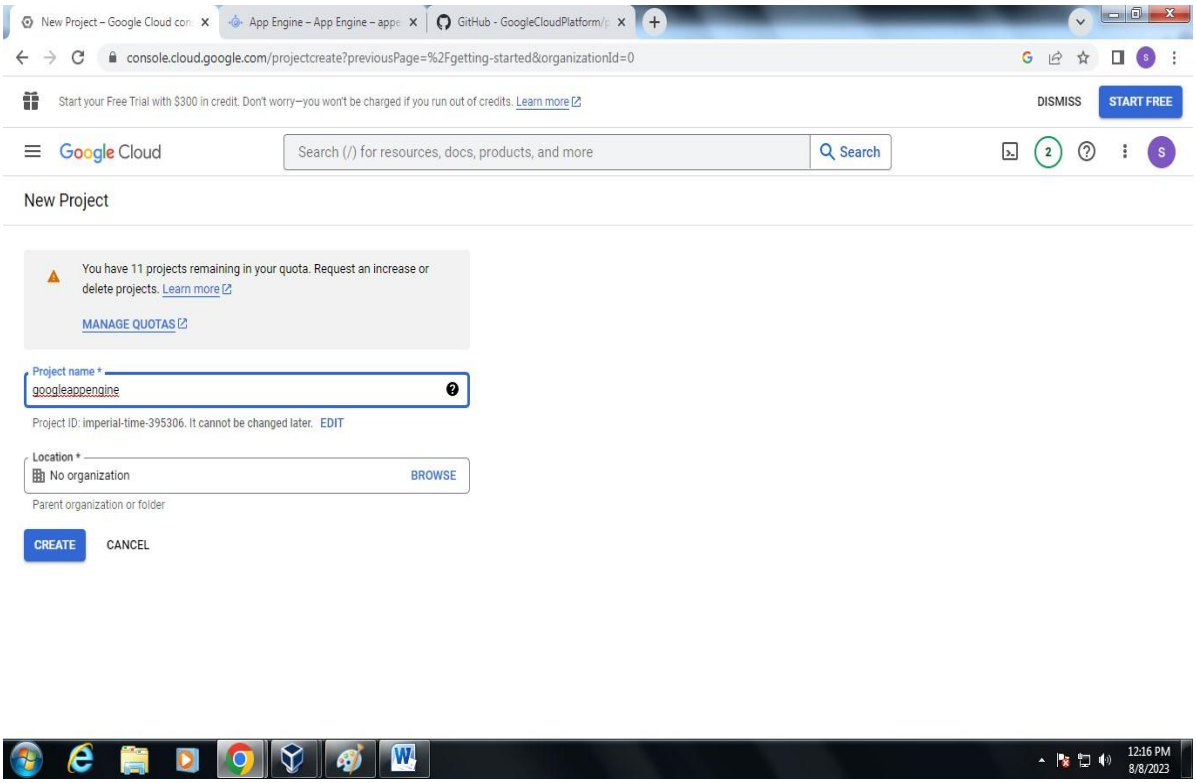
2.Select the project.



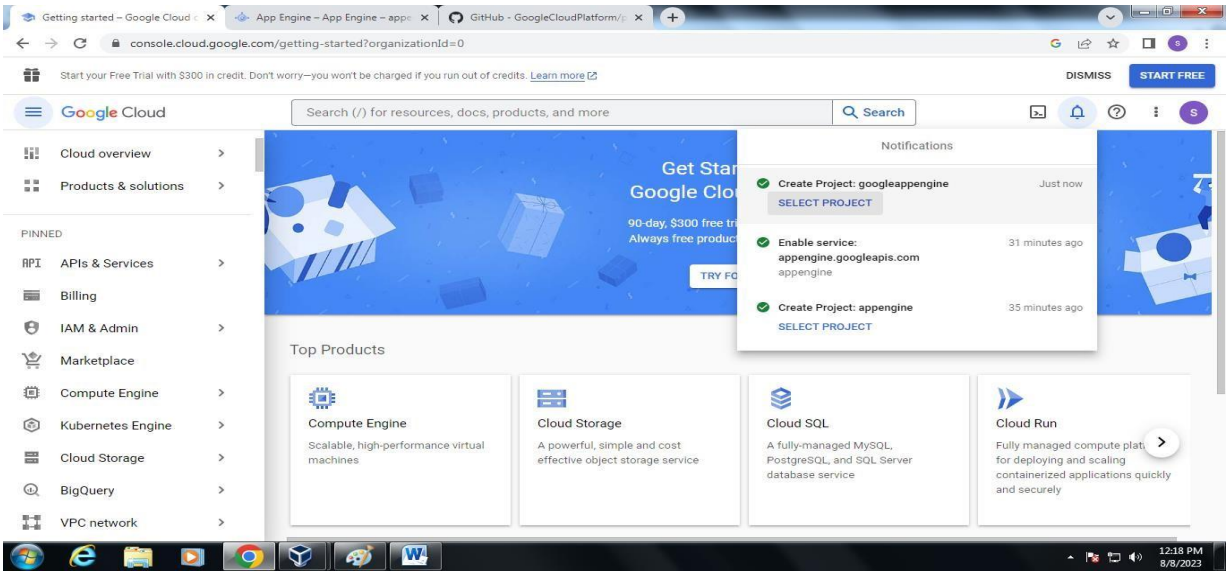
3.Select new project.



4.Give project name with a string of characters that uniquely identifies your project. For example, google appengine.

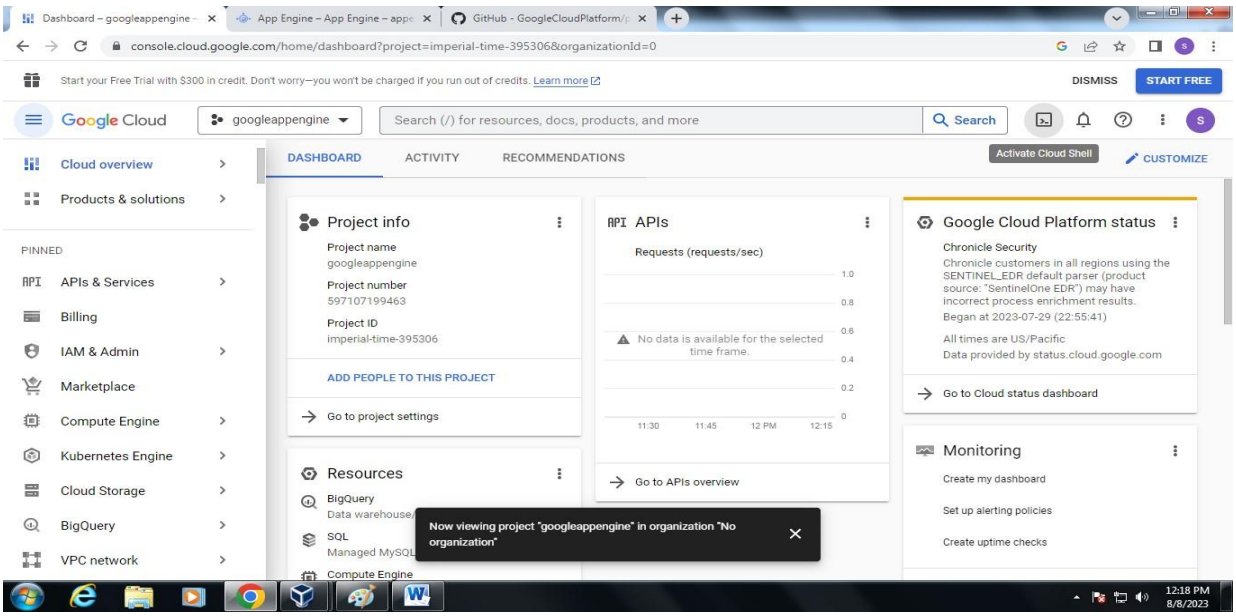


5.Select the created project – googleappengine.



Now the project is created with the project number and ID.

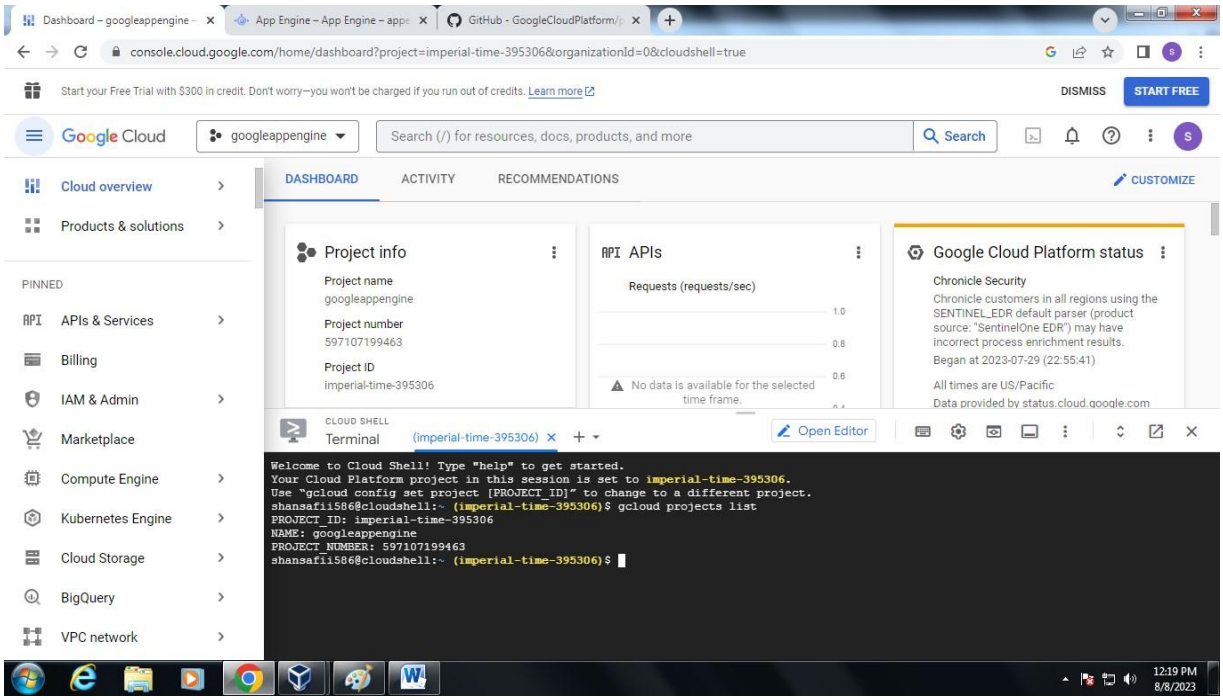
6.Activate Cloud Shell.



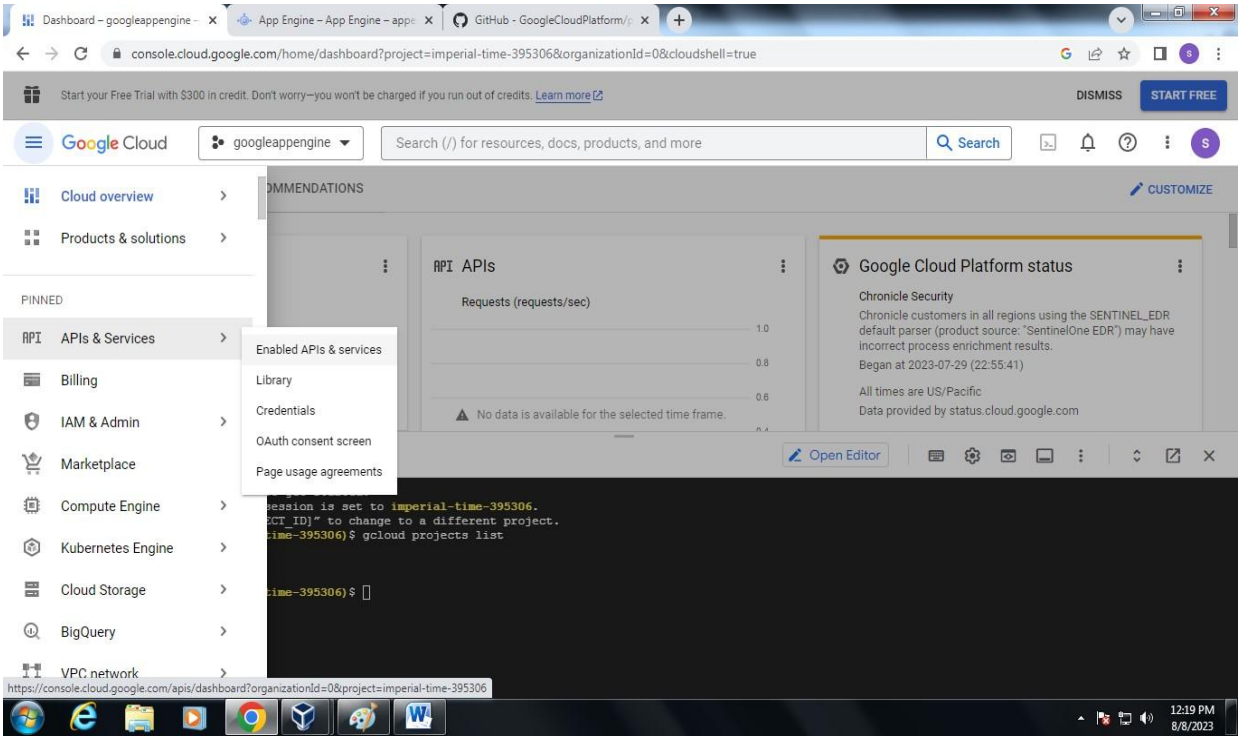
7.In Cloud Shell, type the command.

```
gcloud  projects  list
```

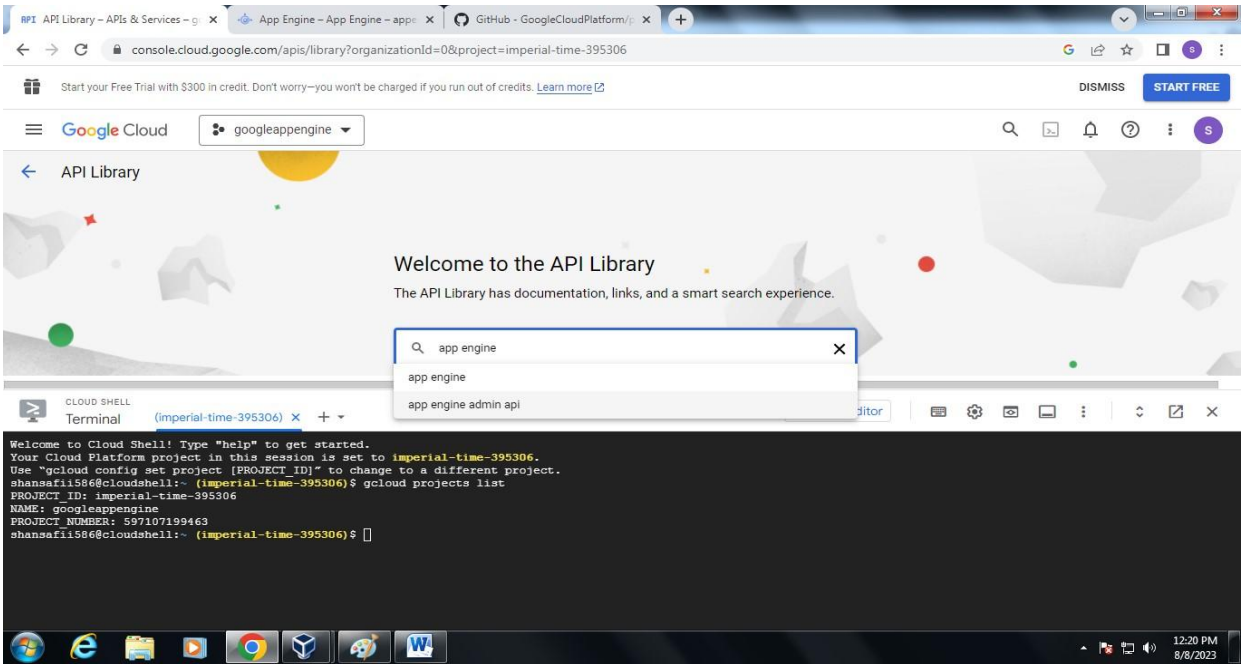
Project details will be listed in cloud shell.



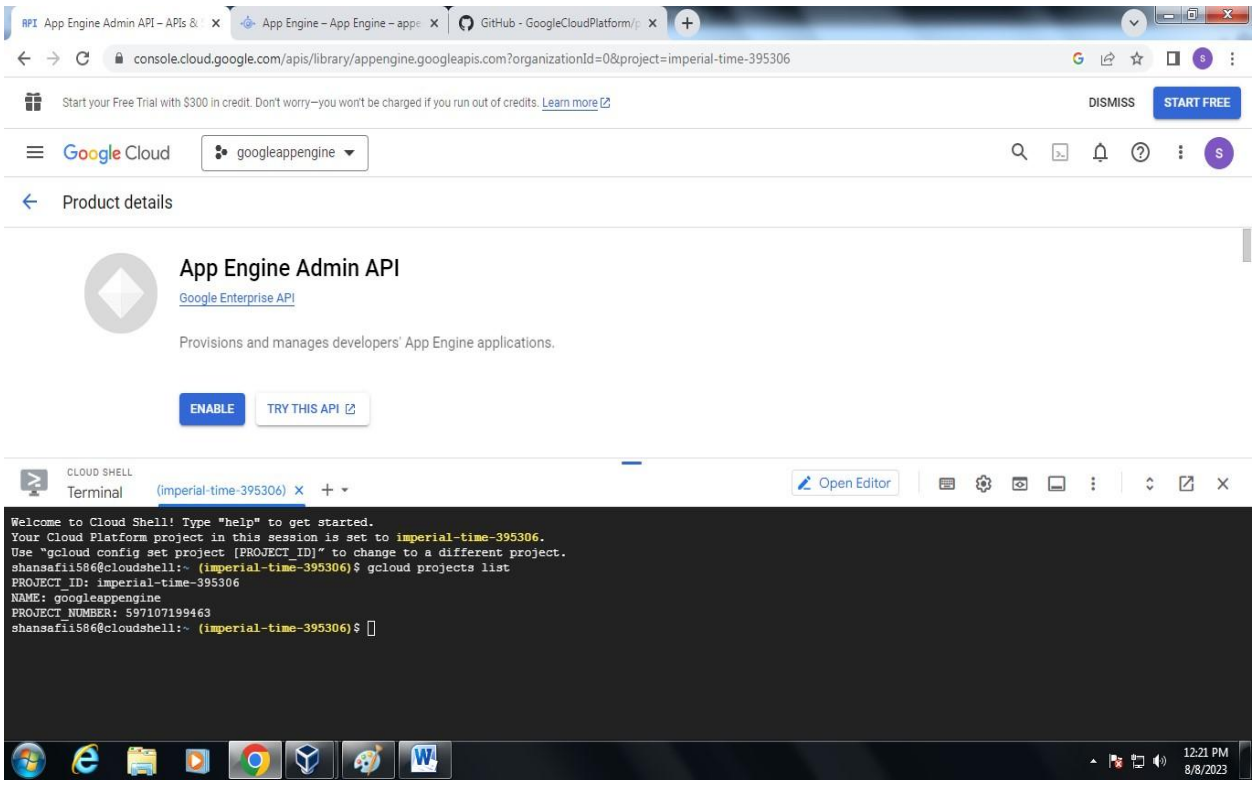
8.Go to menu select “APIs & Services” then select “Enabled APIs & Services”



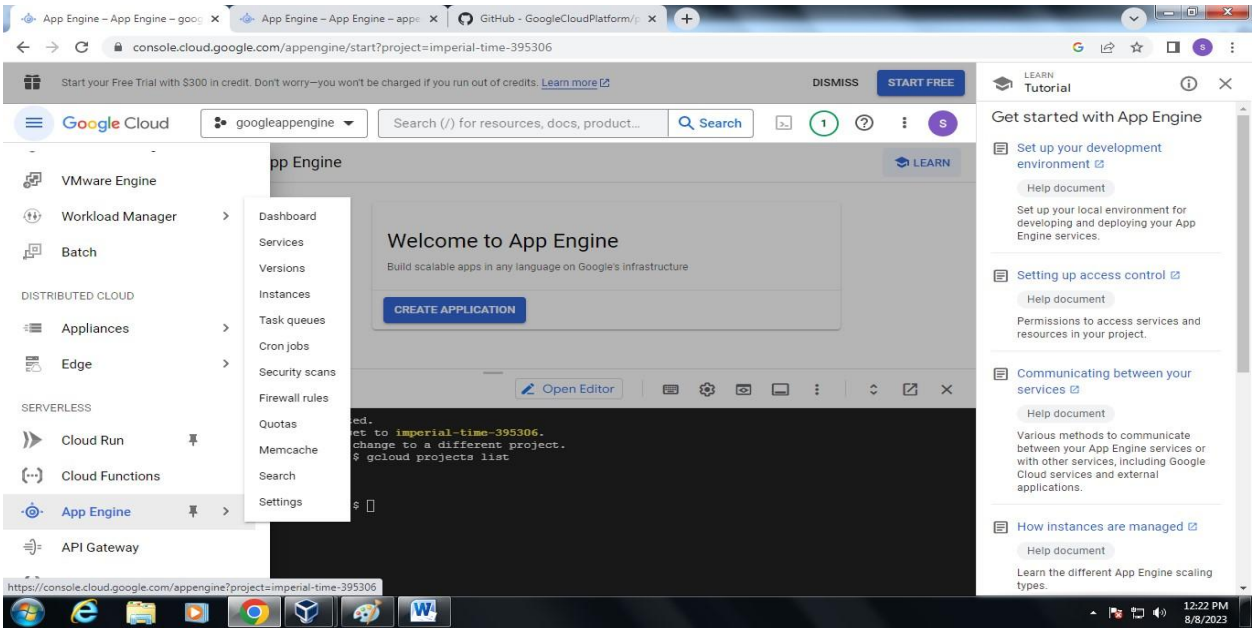
9.Search box will appear. Type “app engine” and select “App Engine Admin API”.



10.Click enable.

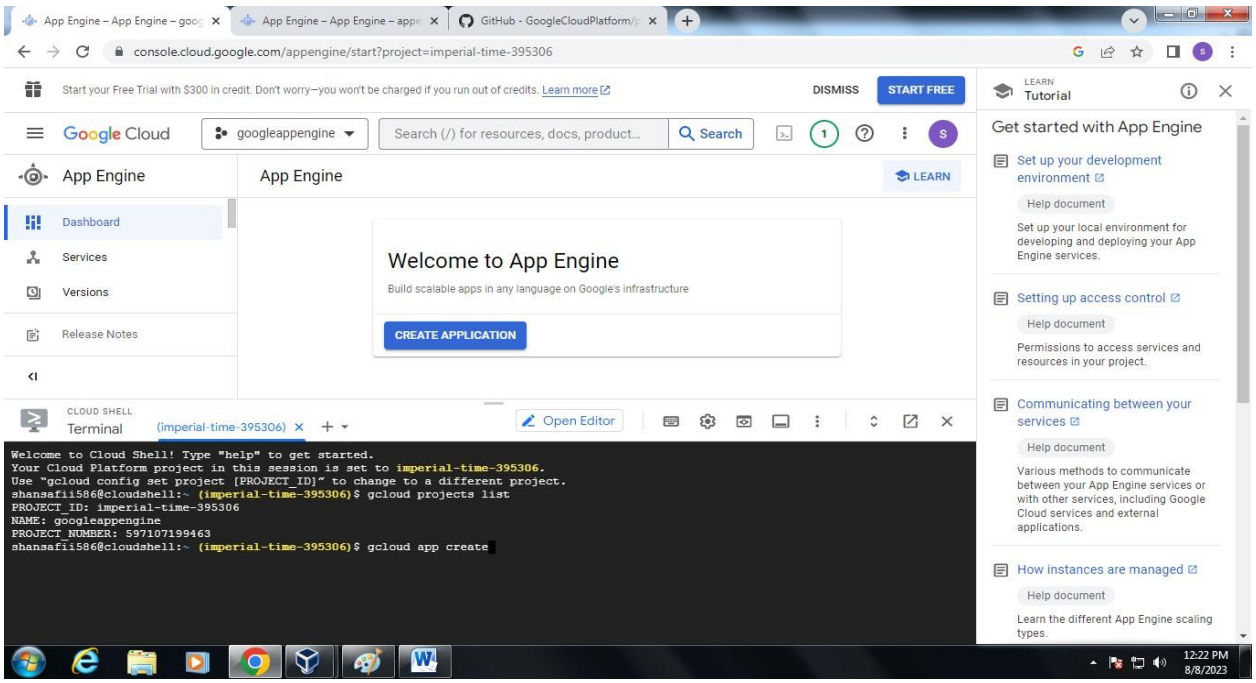


11.Goto menu and select “App Engine”.

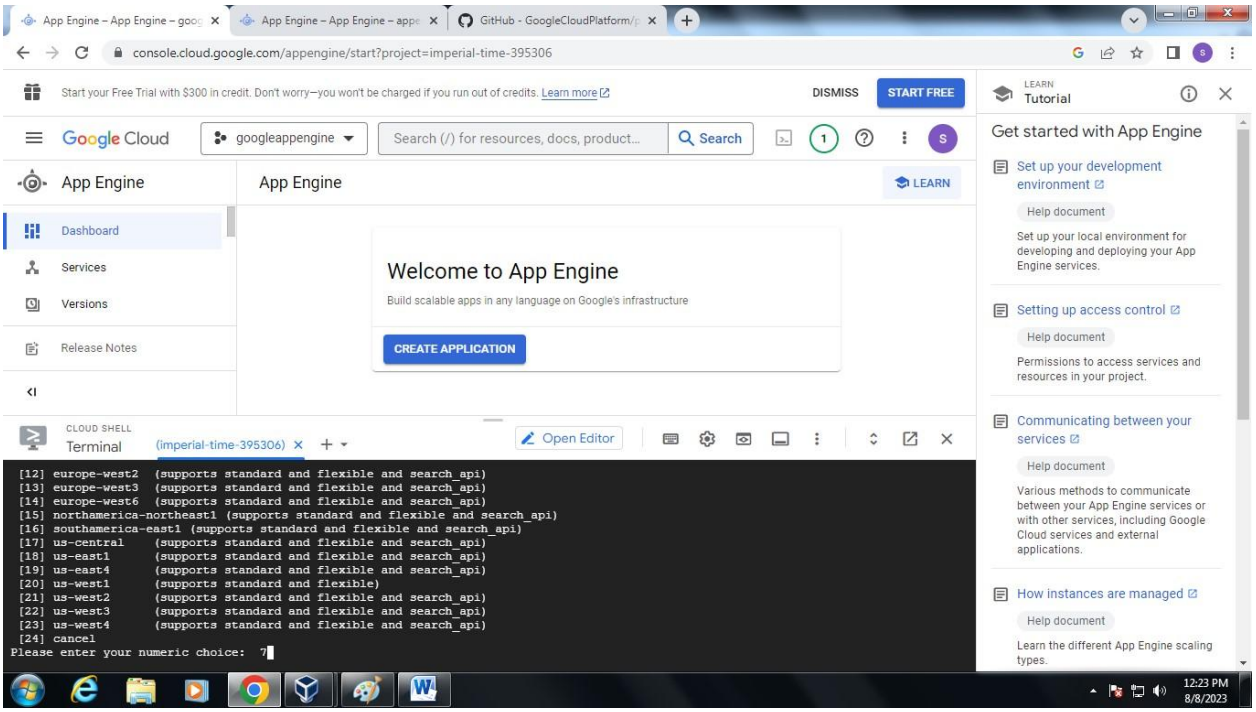


12.In cloud shell type the command to create app

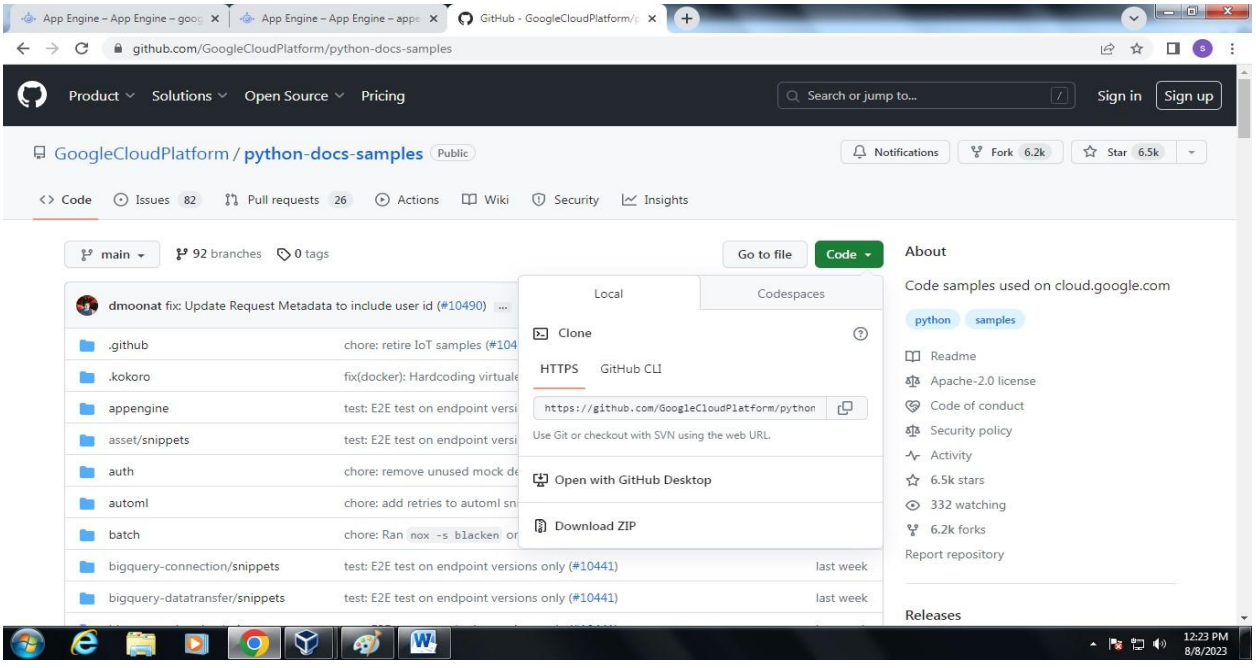
```
gcloud  app create
```

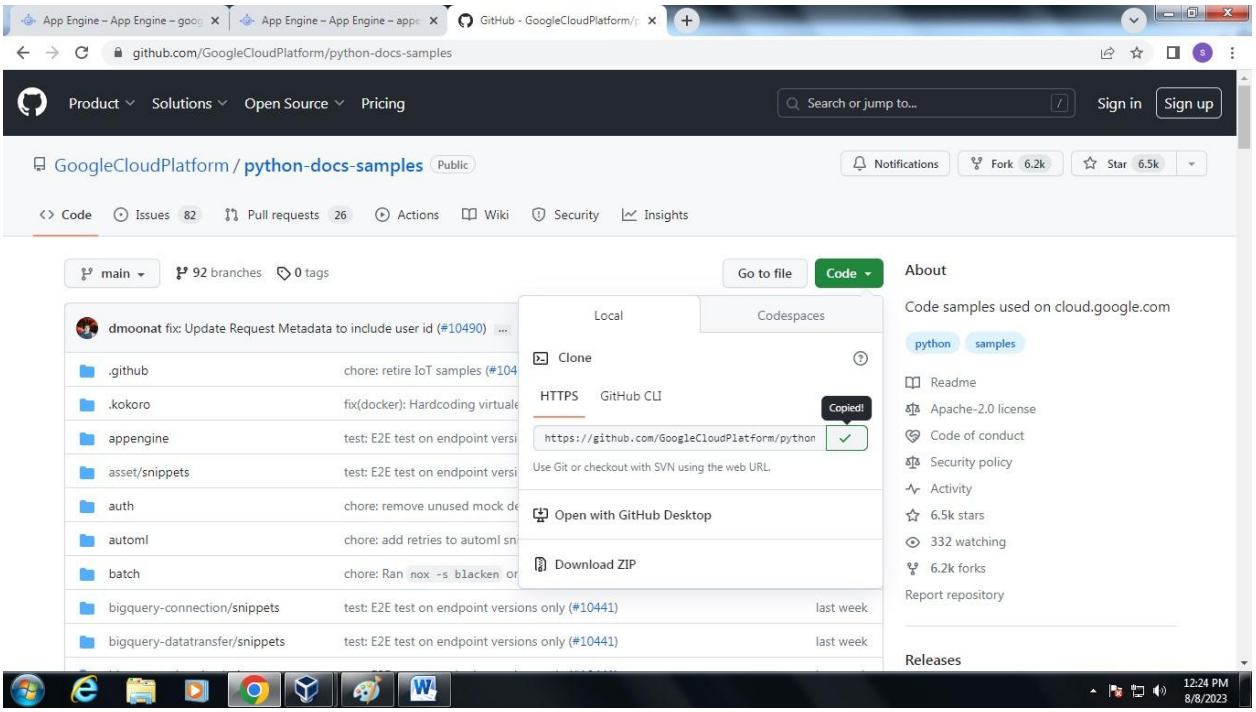


13. Select region 7(asia south east 1)

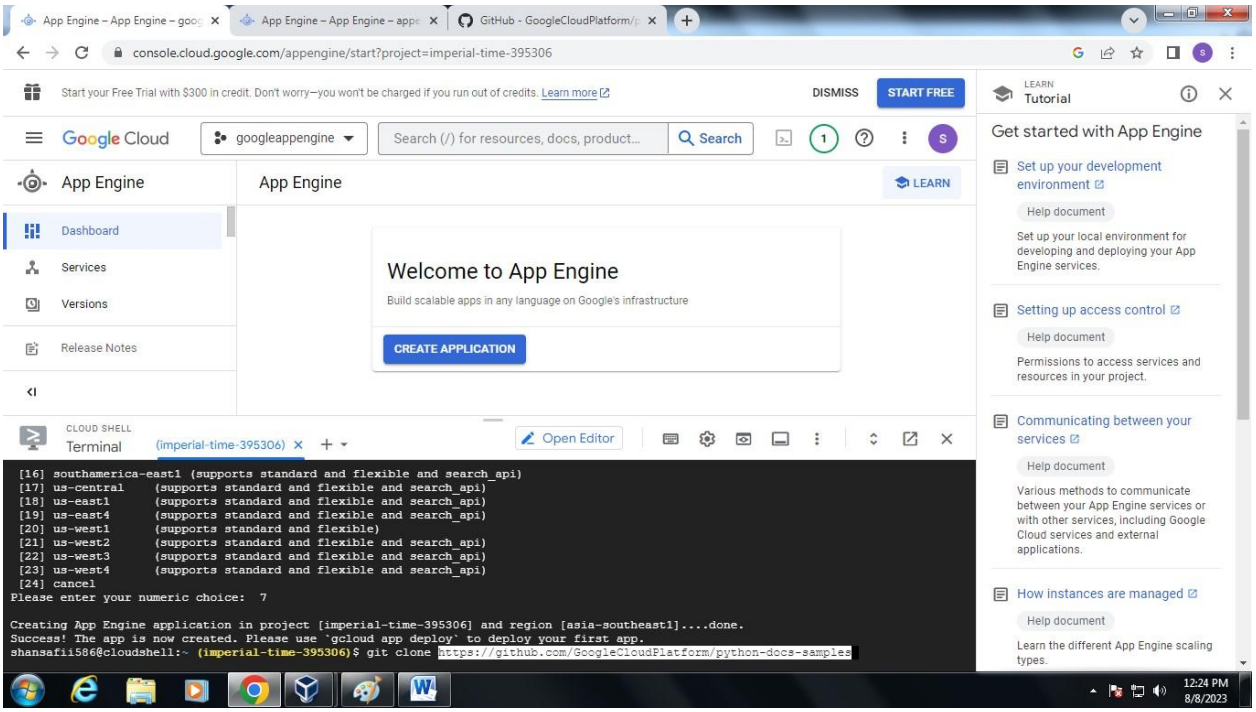


14. goto the drive repository to get the python file github.com/GoogleCloudPlatform/python-docs-samples select code and copy the link.





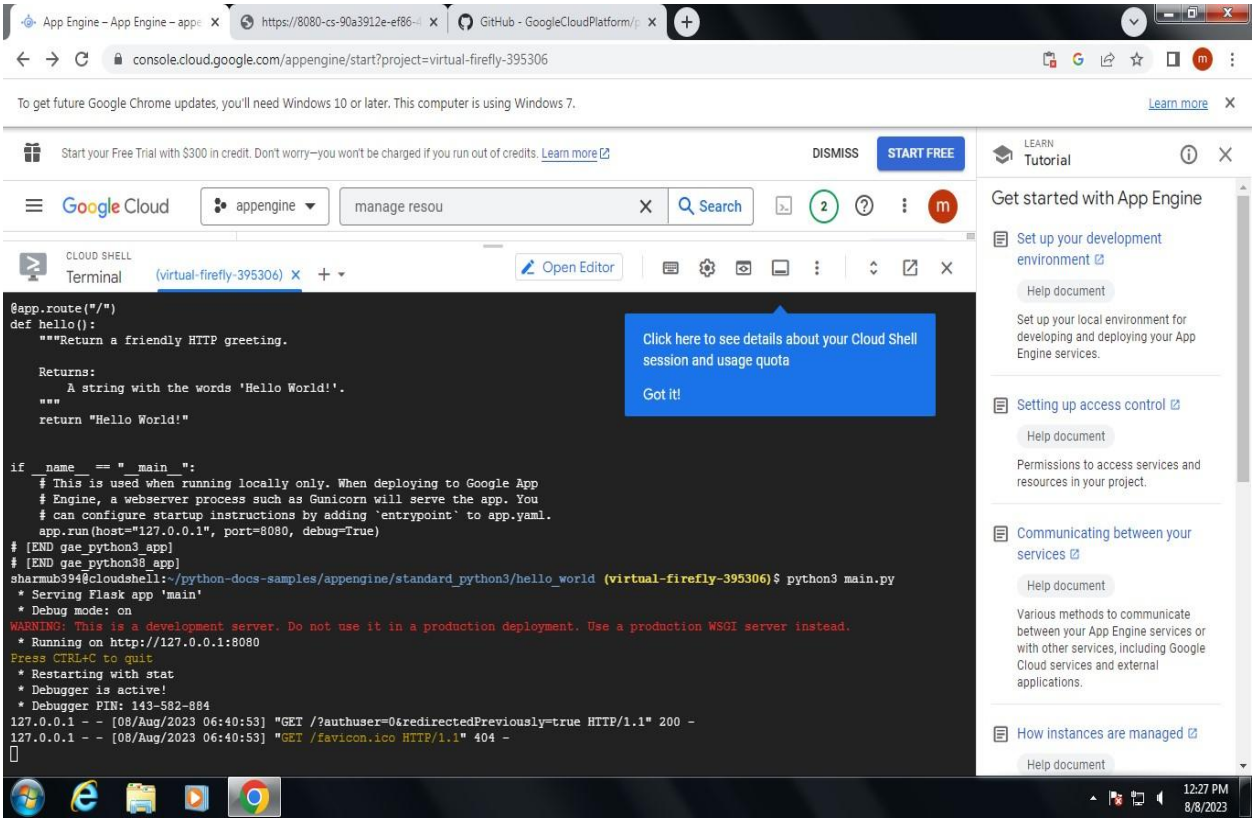
15.Paste the link in cloud shell.

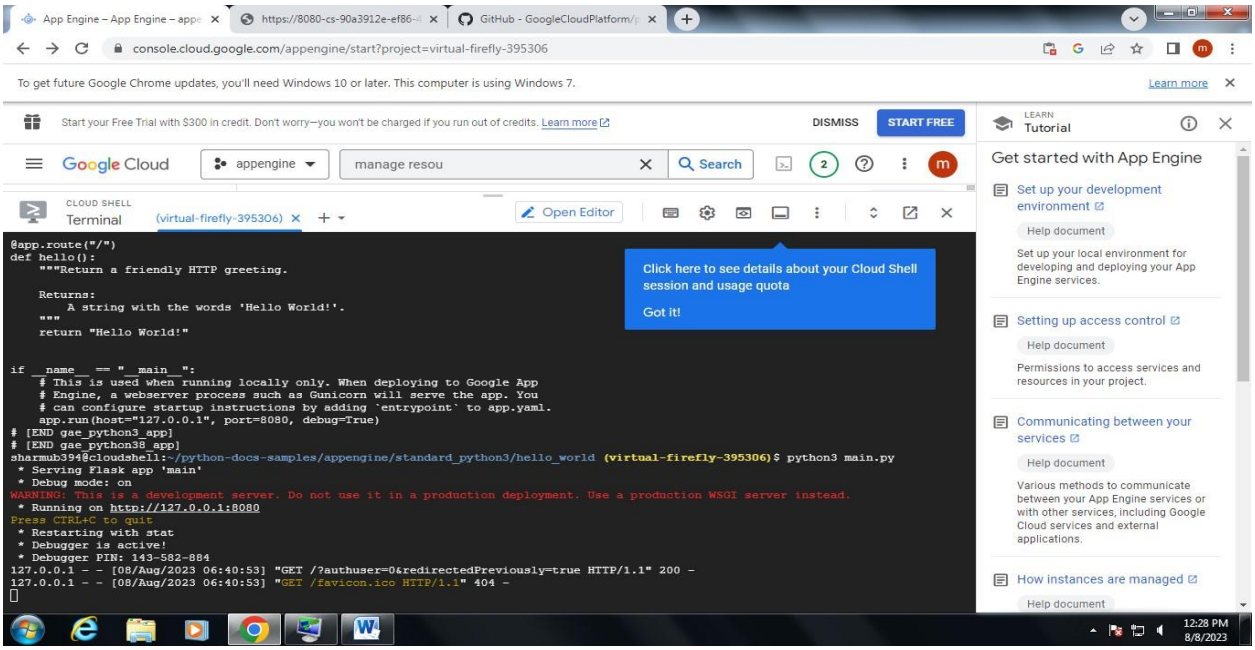


16.In cloud shell, type the command ls.

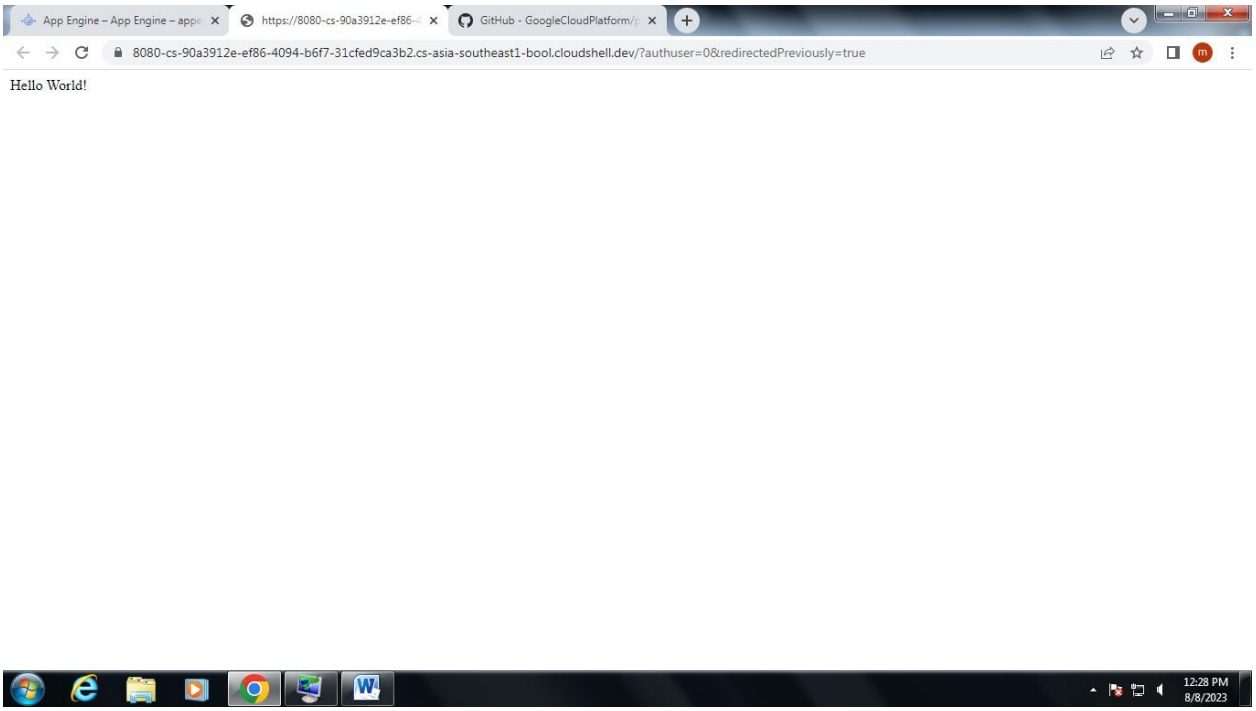
- cd python-docs-samples.
- cd appengine.
- cd standard_python3.

- cd hello_world.
- cat main.py (which contains helloworld program).
- python3 main.py.
- click URL http://127.0.0.1:8080.





17.The Hello world message from the sample app is displayed on the web page.



RESULT:

Thus the project to run the phthon sample program to display Helloworld in Google App Engine in google cloud console is executed succefully.

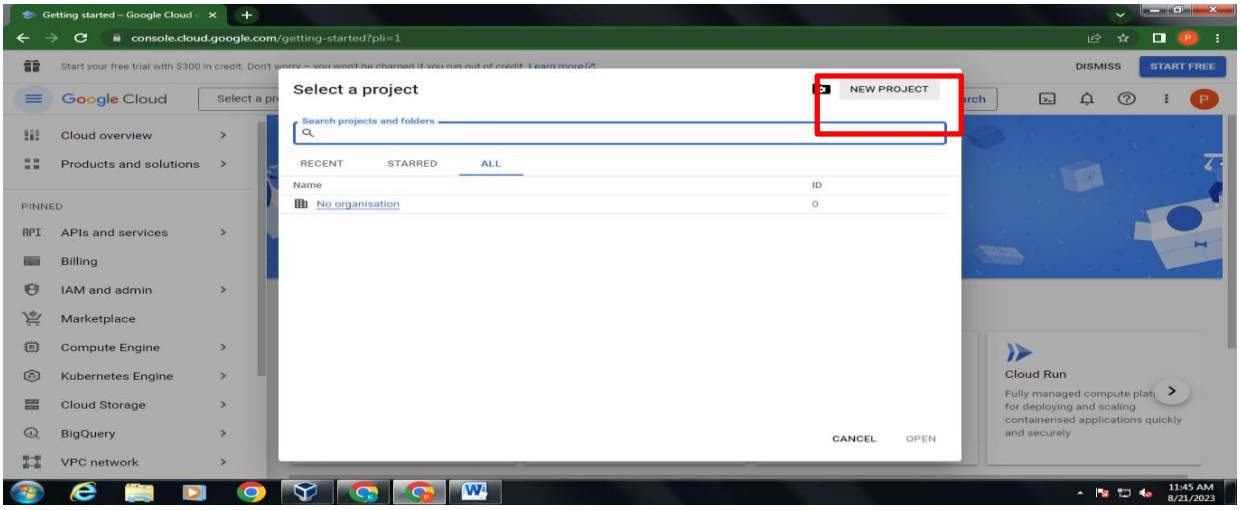
EX.NO- 4: **USE THE GAE LAUNCHER TO LAUNCH THE WEB**
DATE: **APPLICATIONS**

AIM:

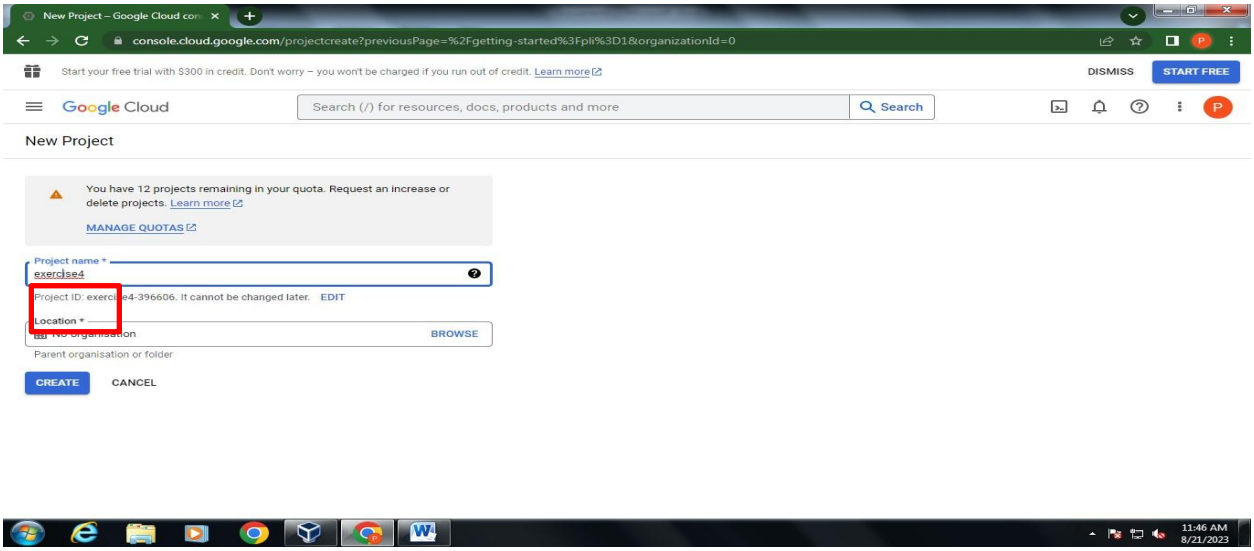
To create a project with the help of GAE launcher to launch web application Helloworld in java.

PROCEDURE:

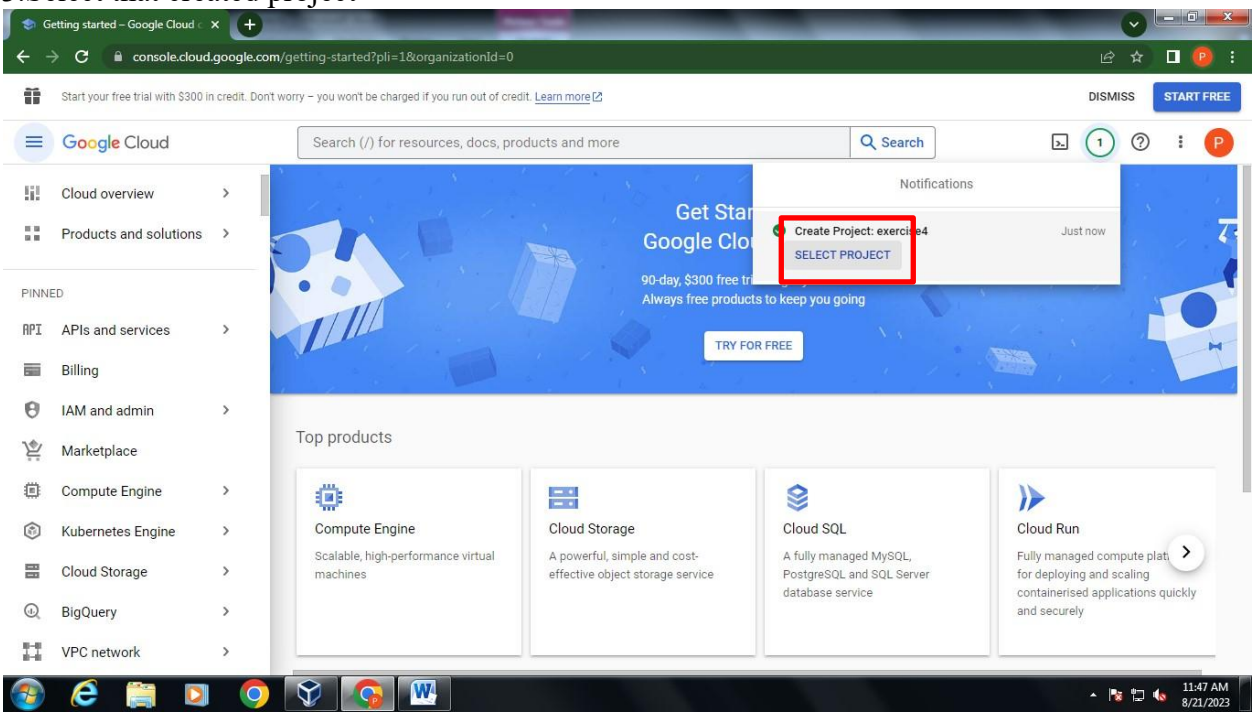
1.Create new project



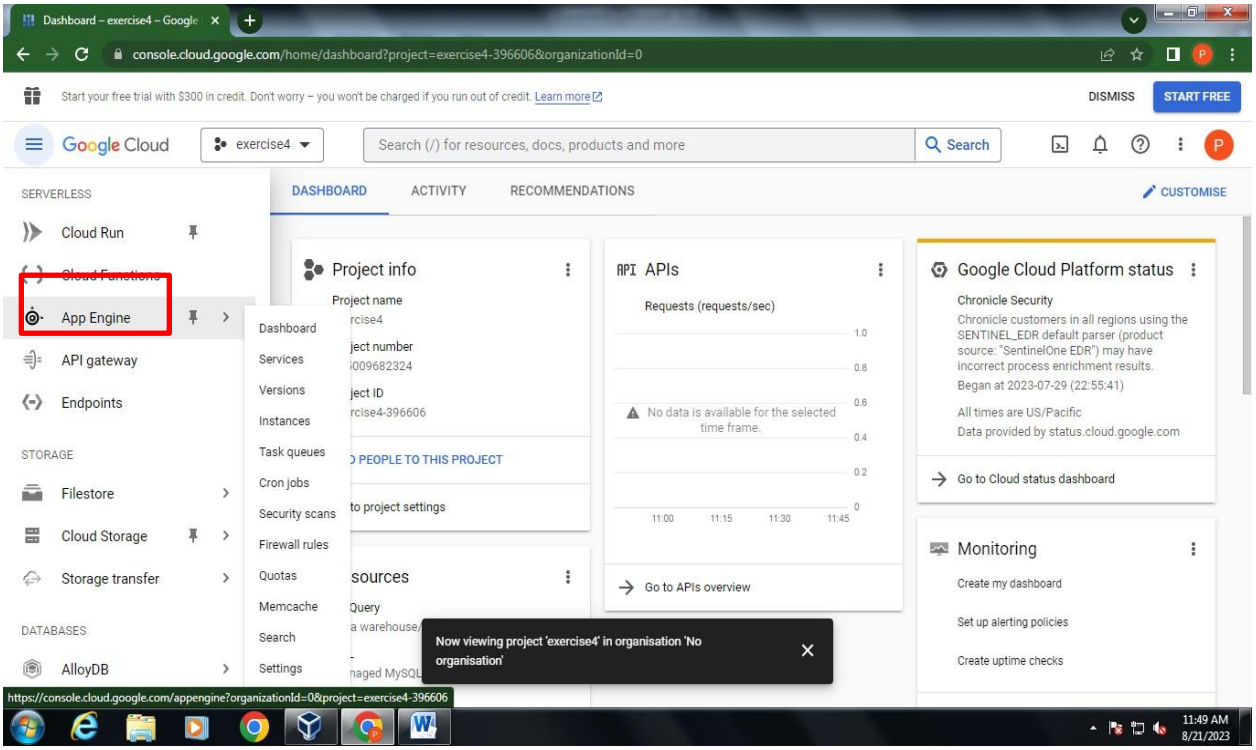
2.Give project name



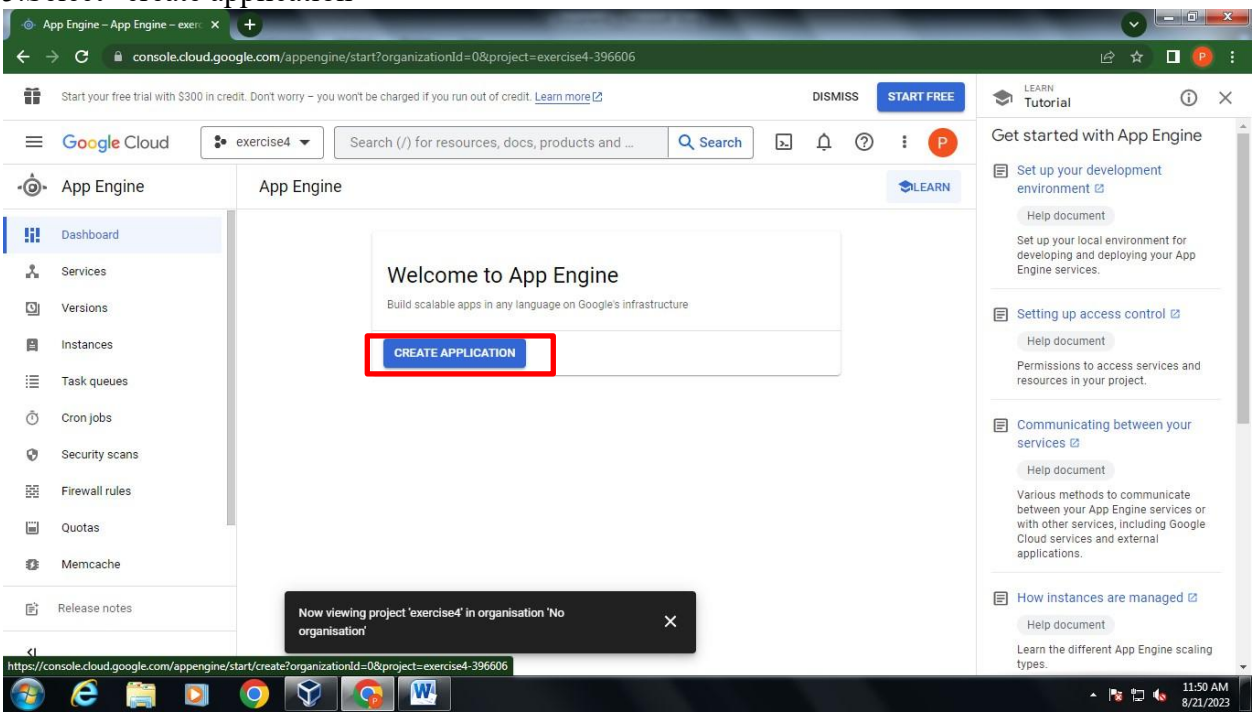
3.Select that created project



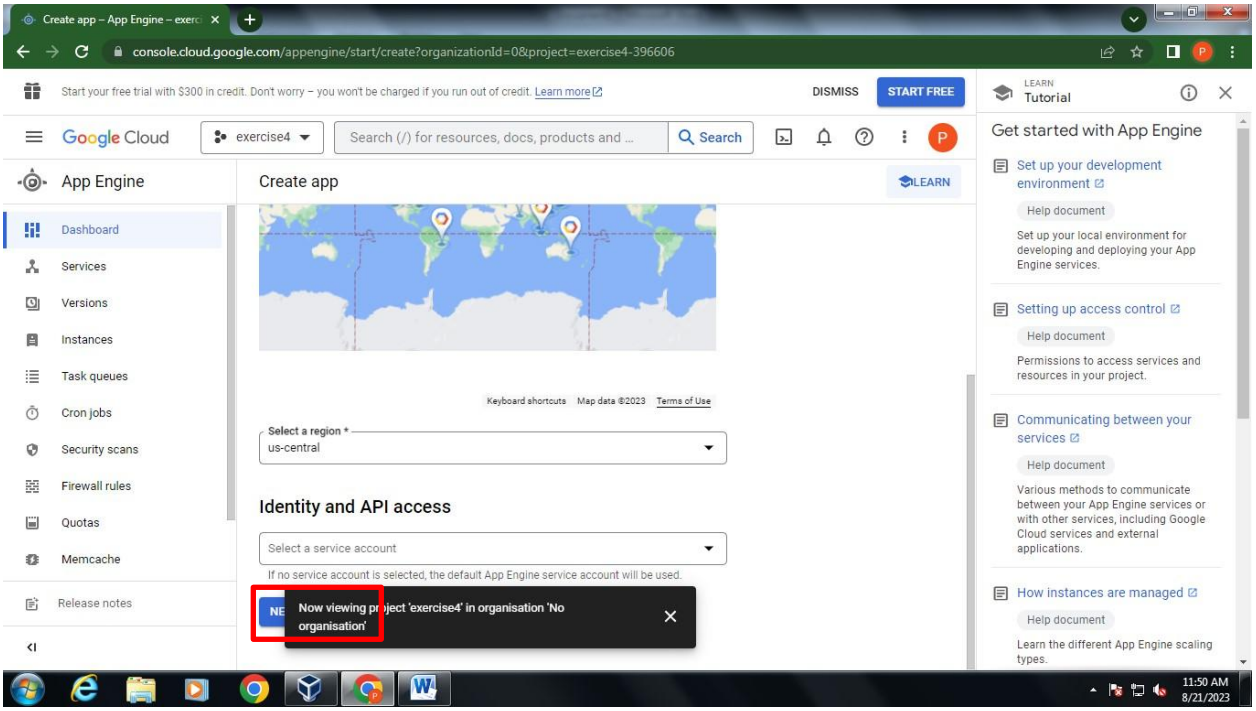
4.In menu goto appengine

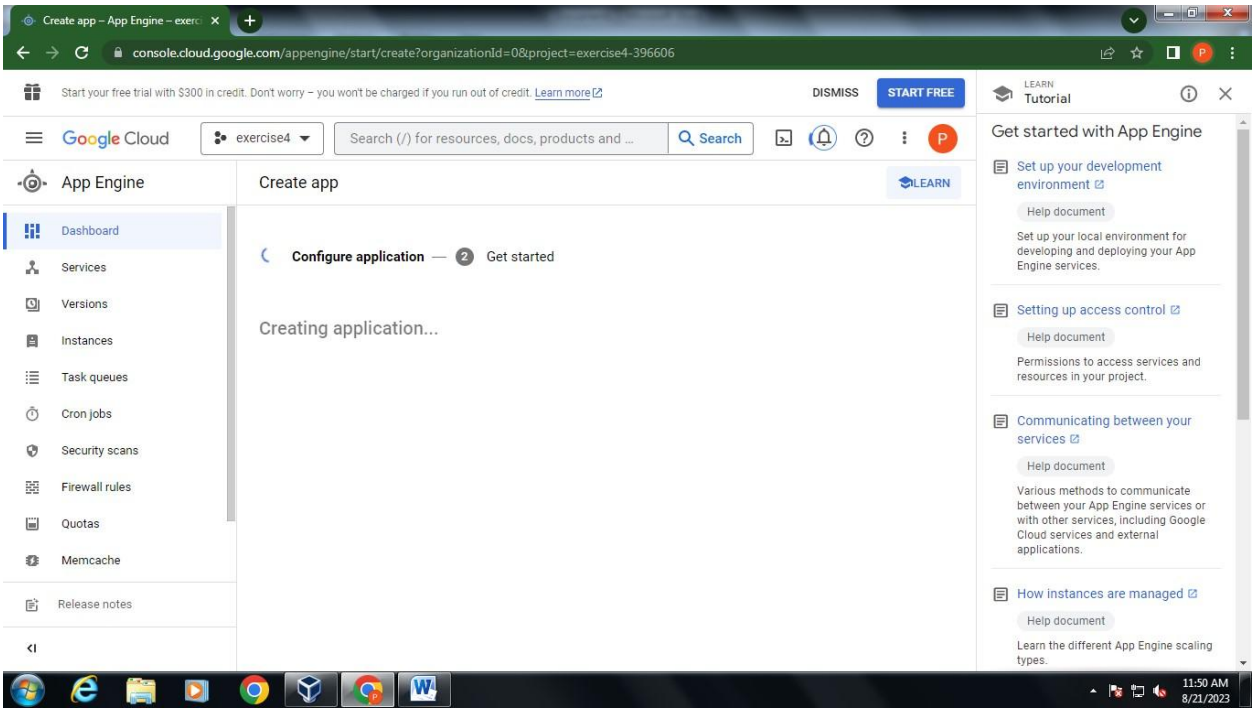


5.Select “create application”

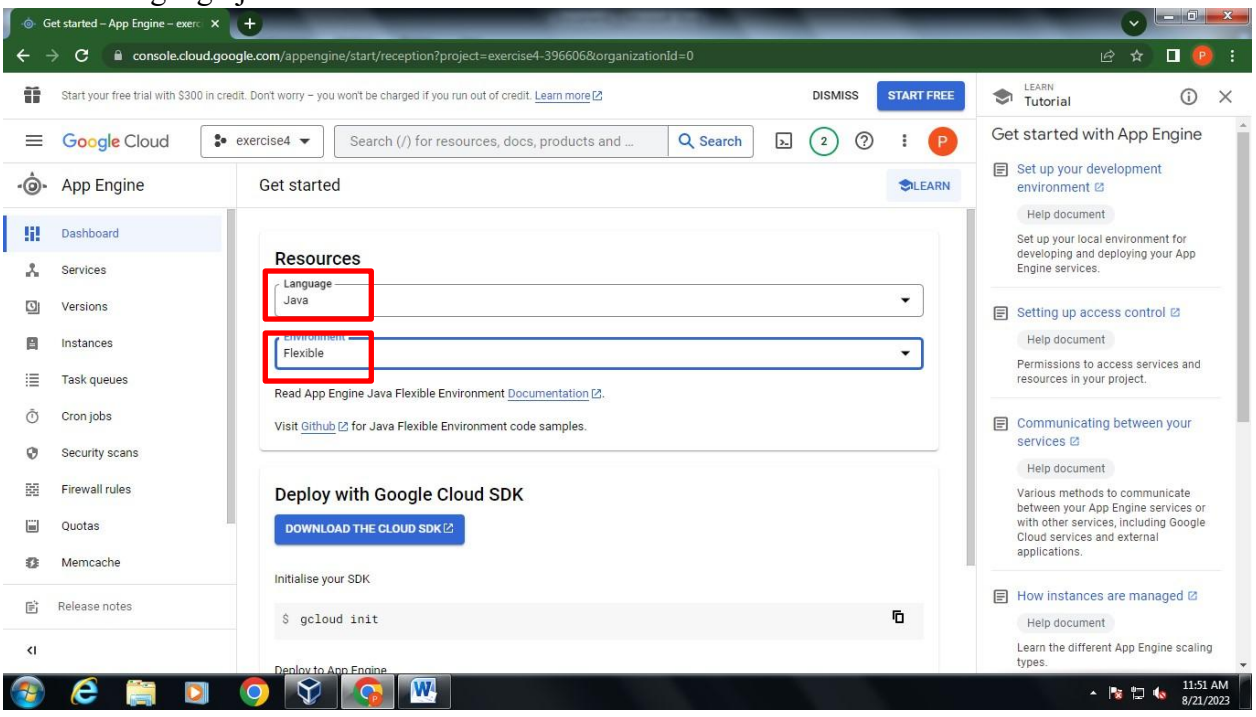


6.Select region us-central and give ok

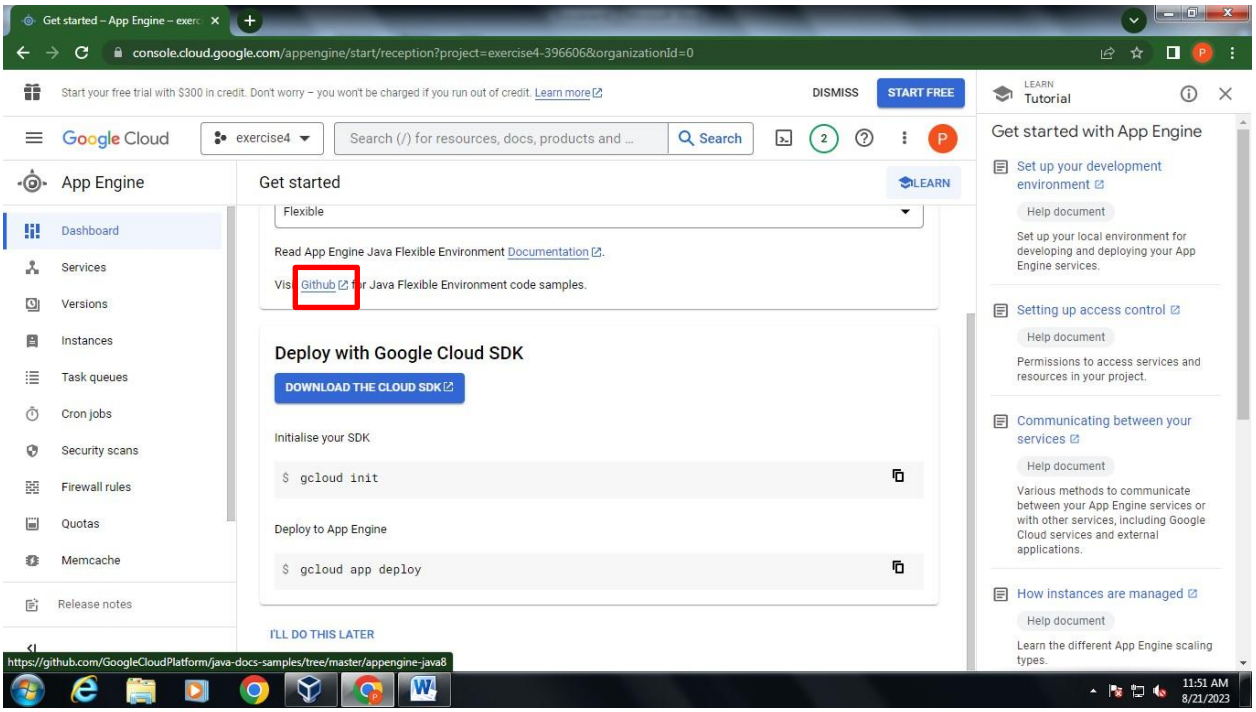


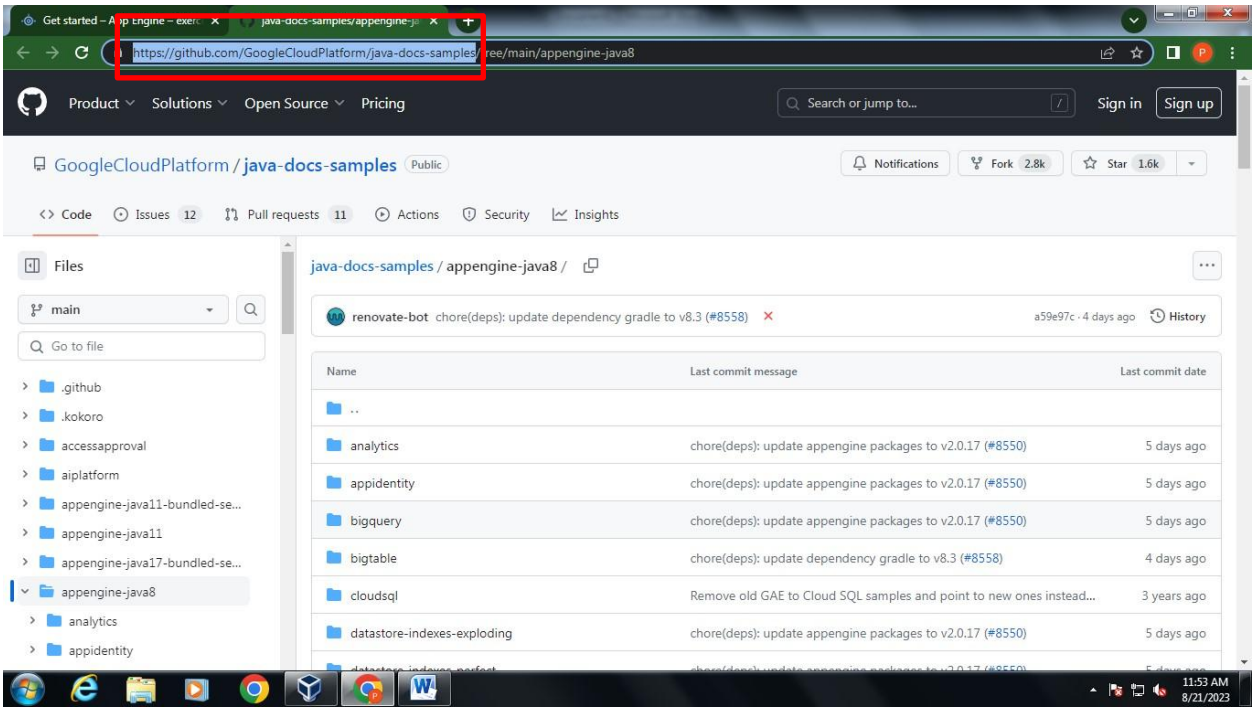


7.Select language-java and Envoronment-flexible

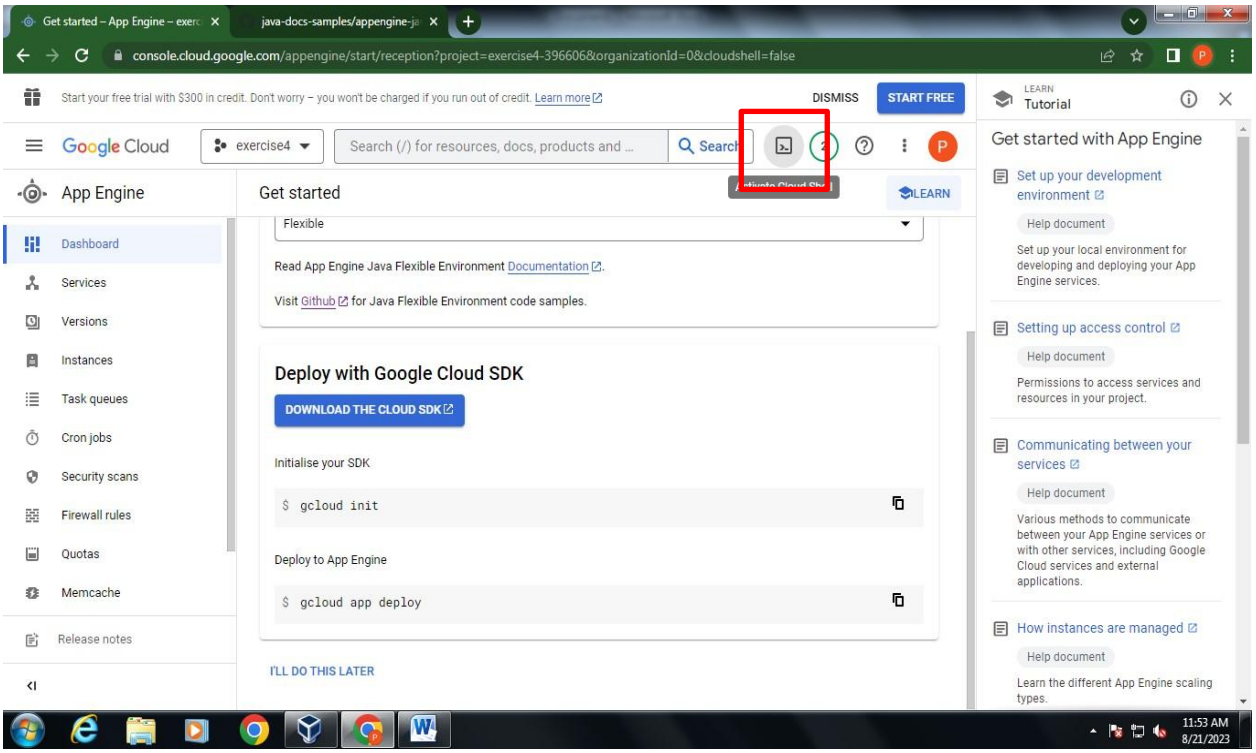


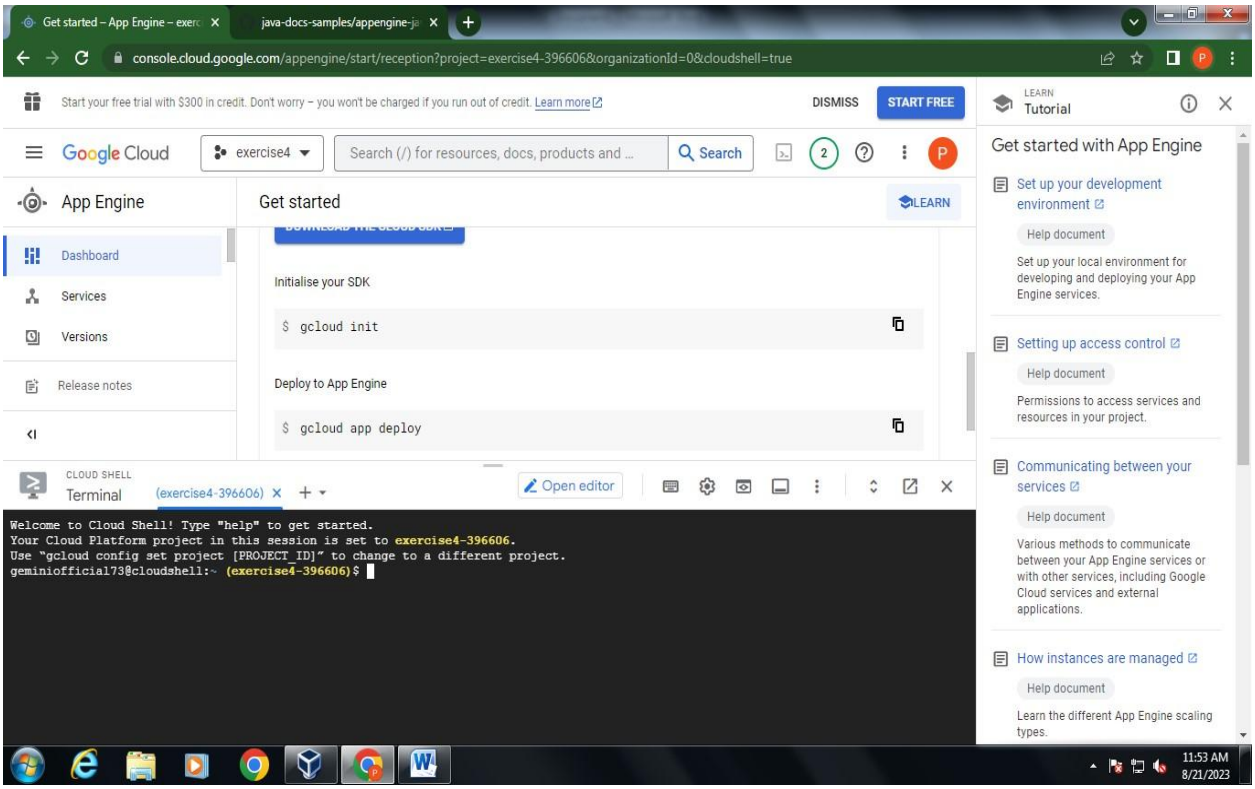
8.Select GITHUB repository for java application





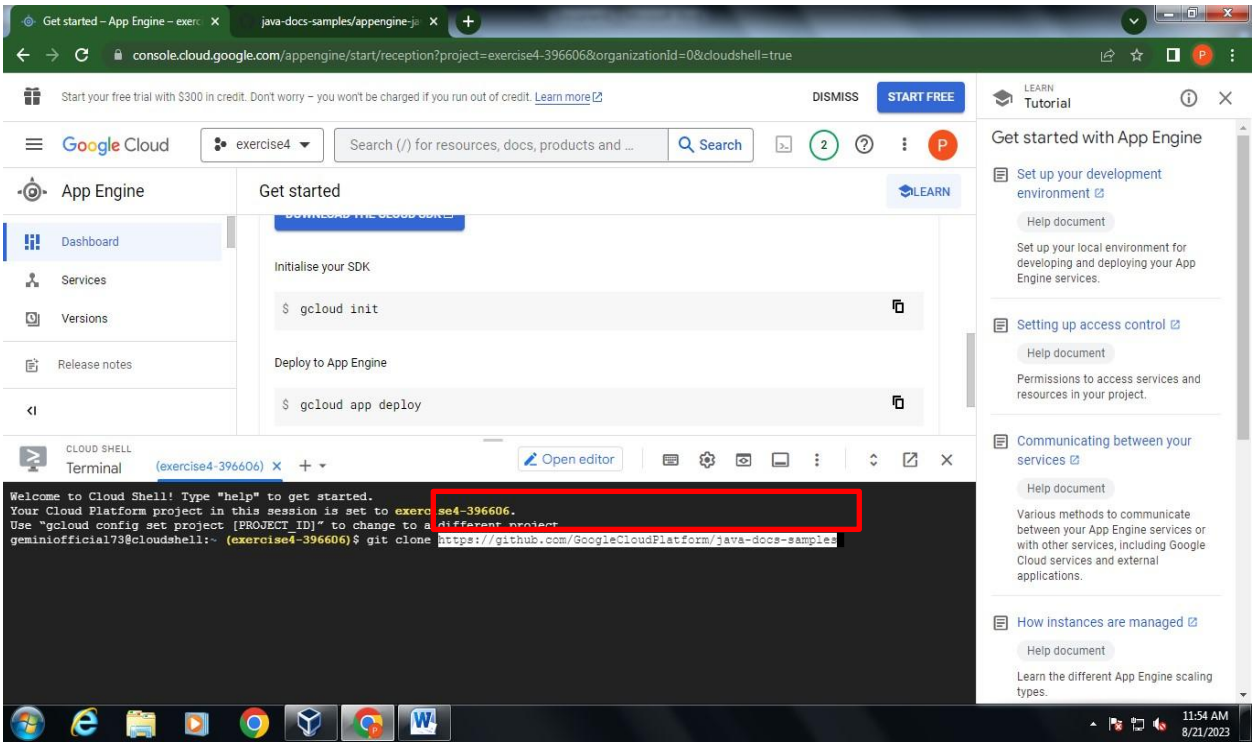
9.Activate Cloud shell



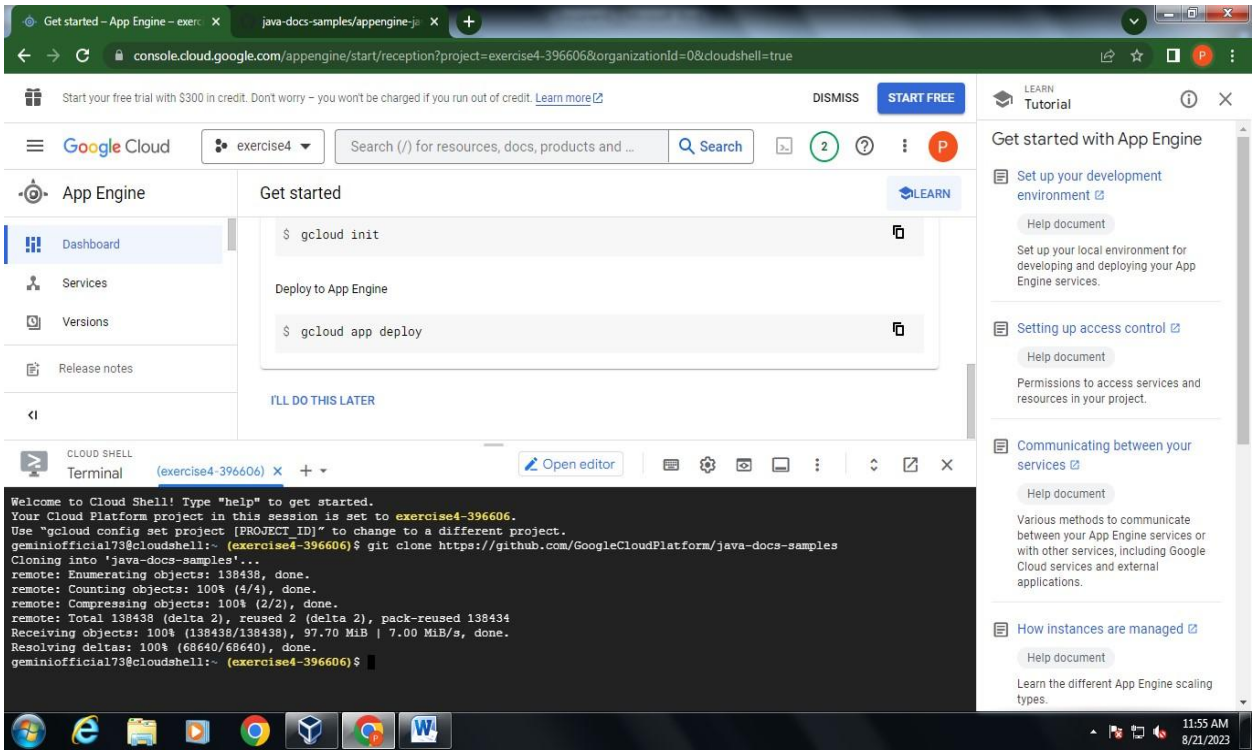


10.Paste the github link in cloud shell

github.com/GoogleCloudPlatform/java-docs-samples

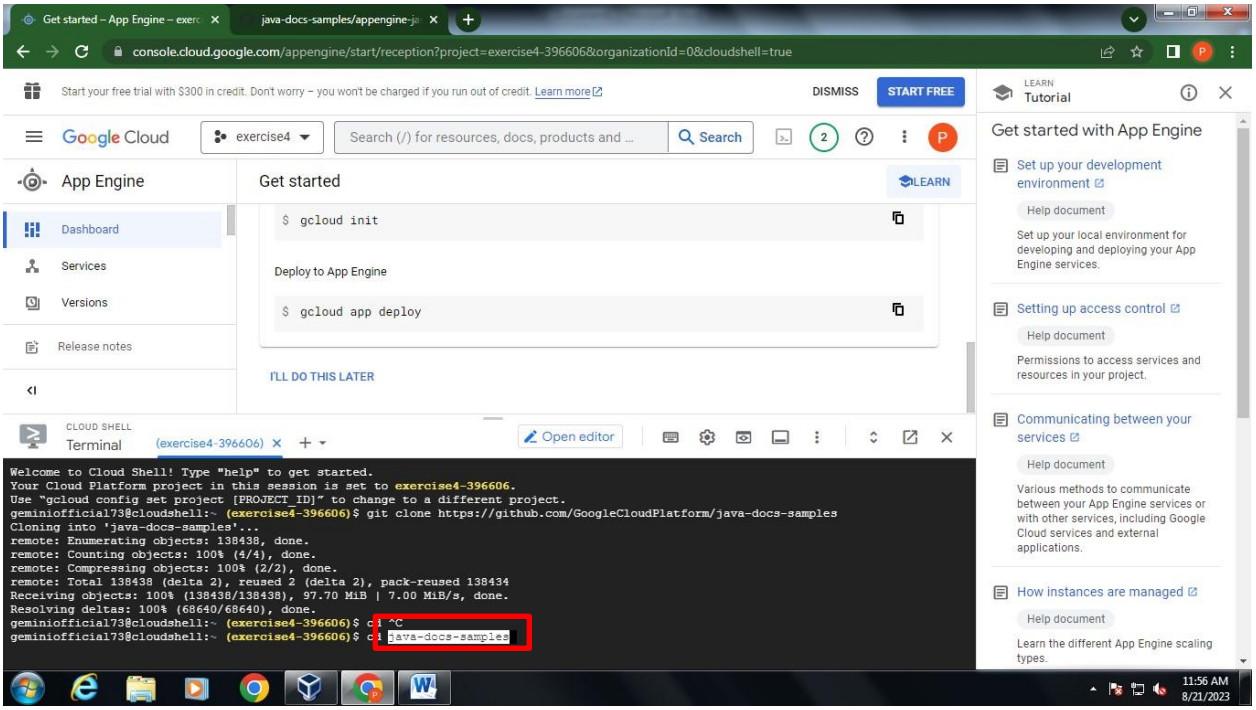


11.Project is authorized in cloud platform

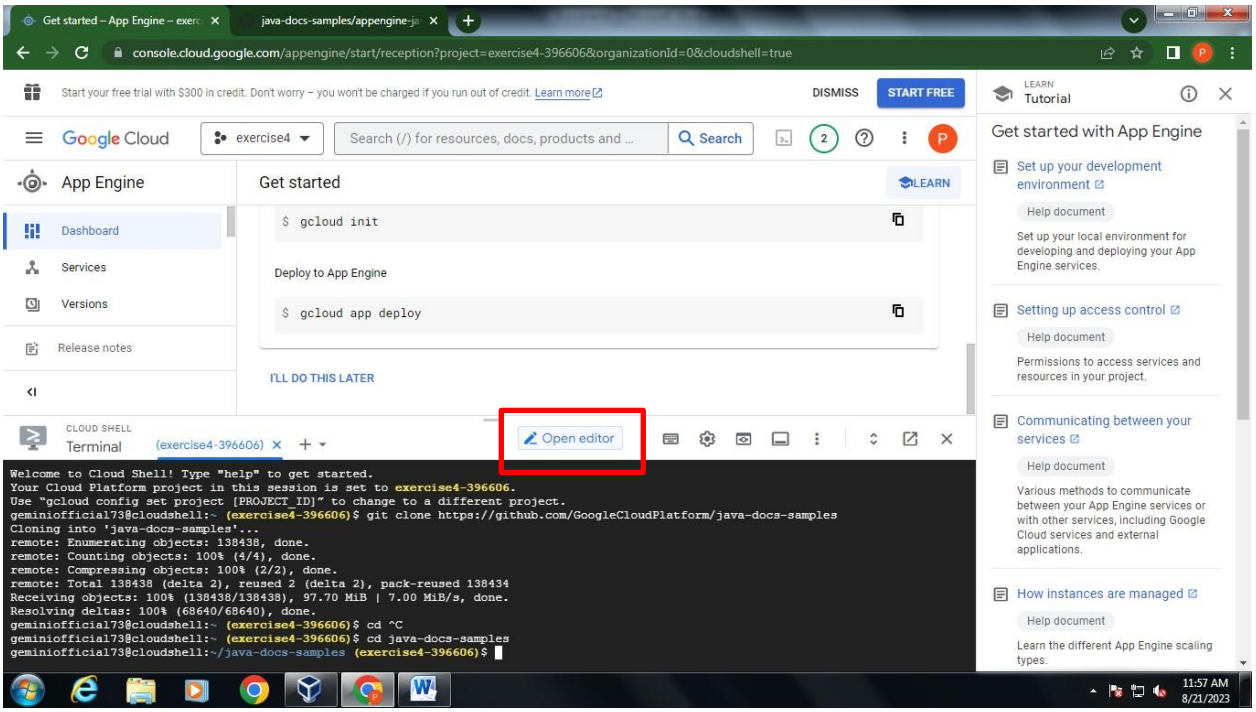


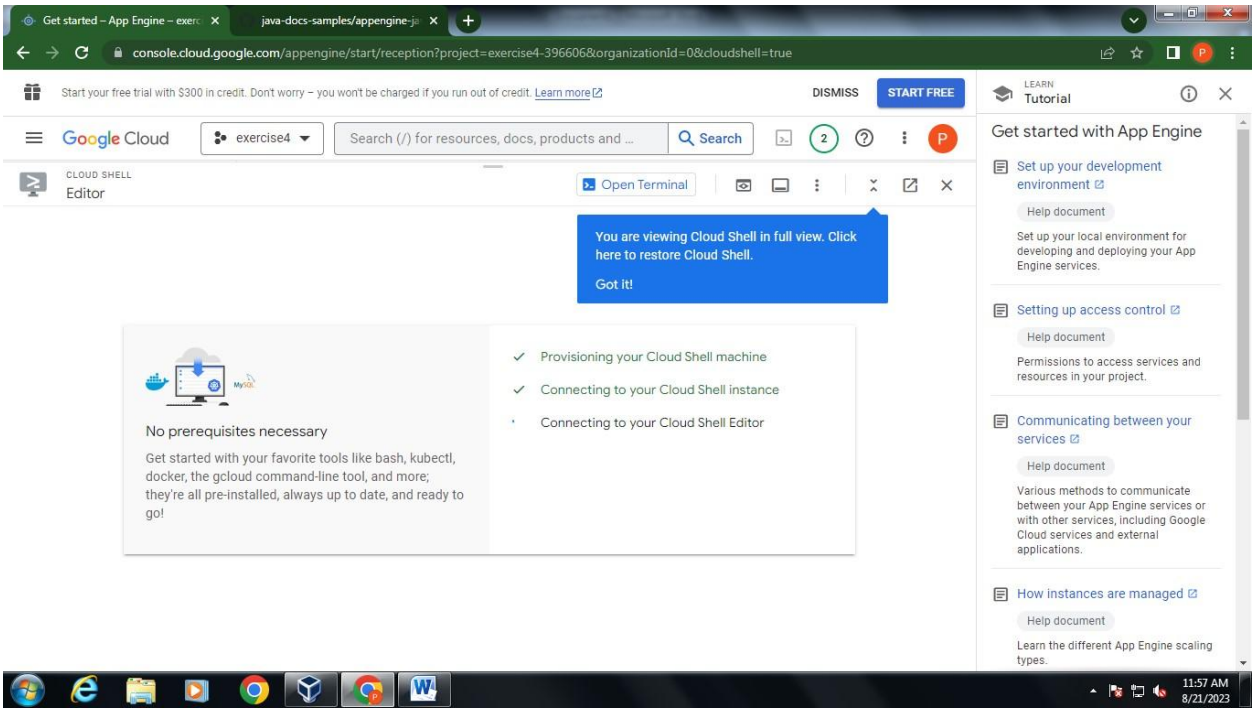
12.Type the commands in cloud shell.

cd java-docs-samples

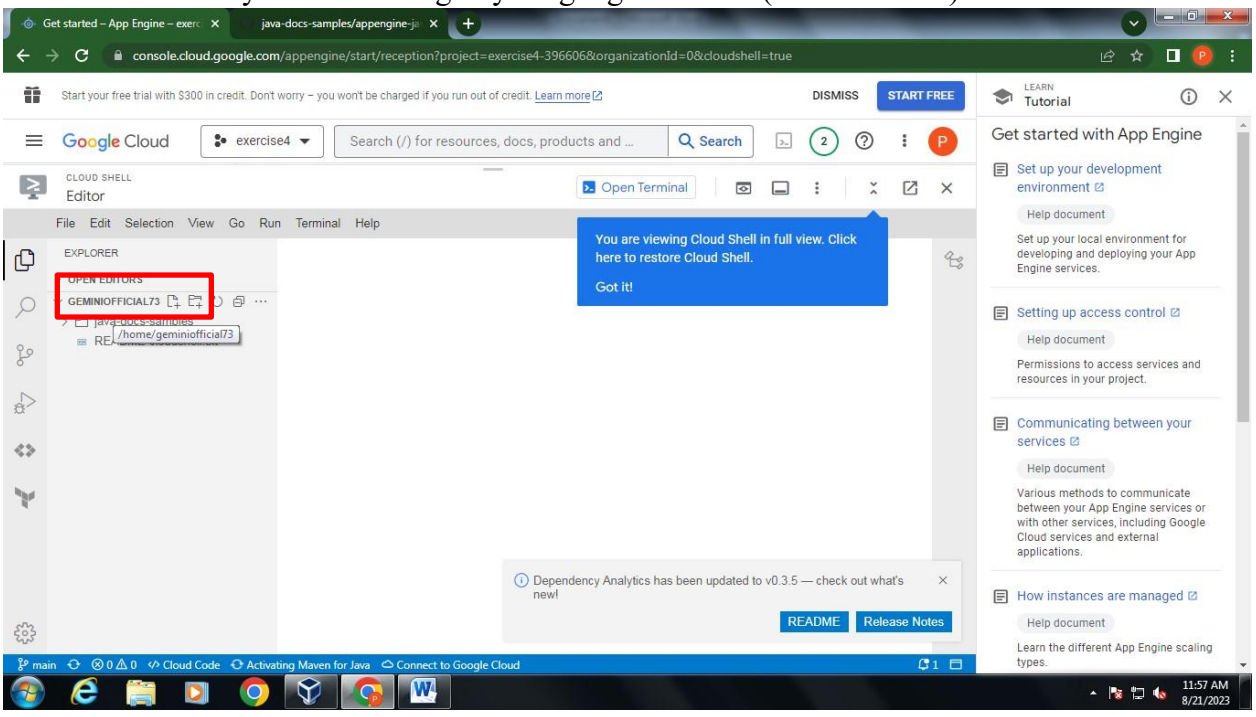


13.open editor.

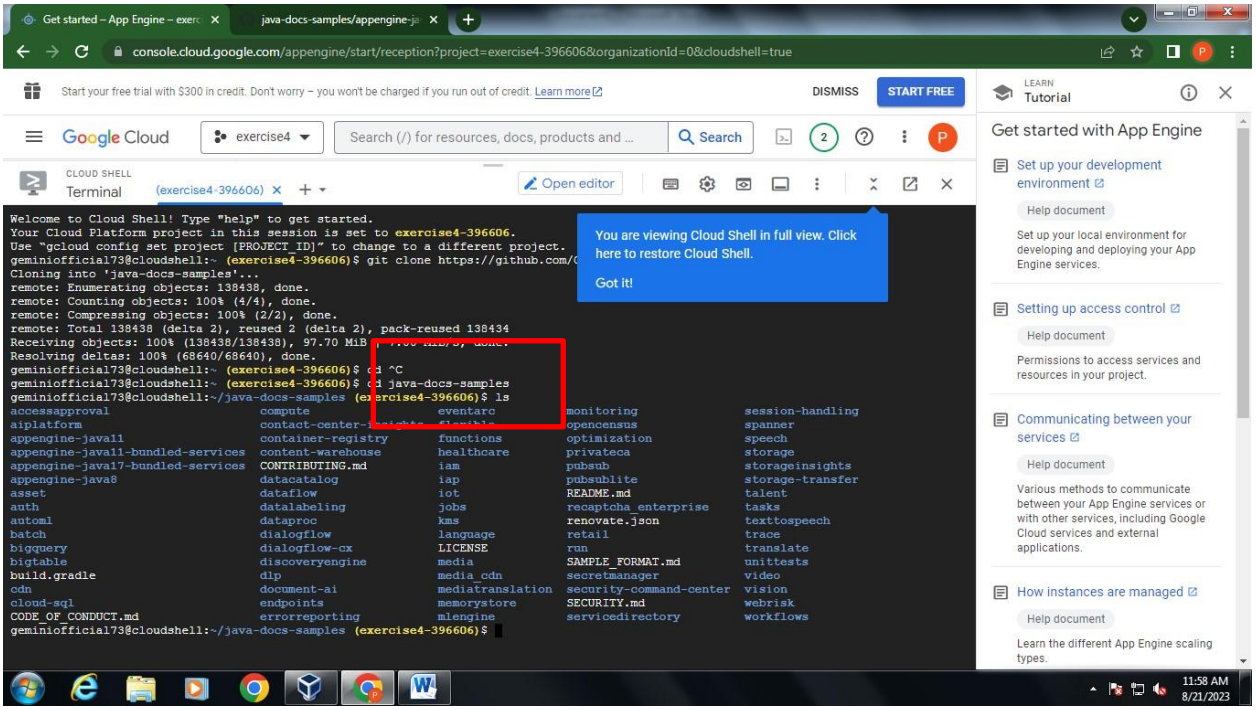




14.Check whether you are working in your google account (account name).



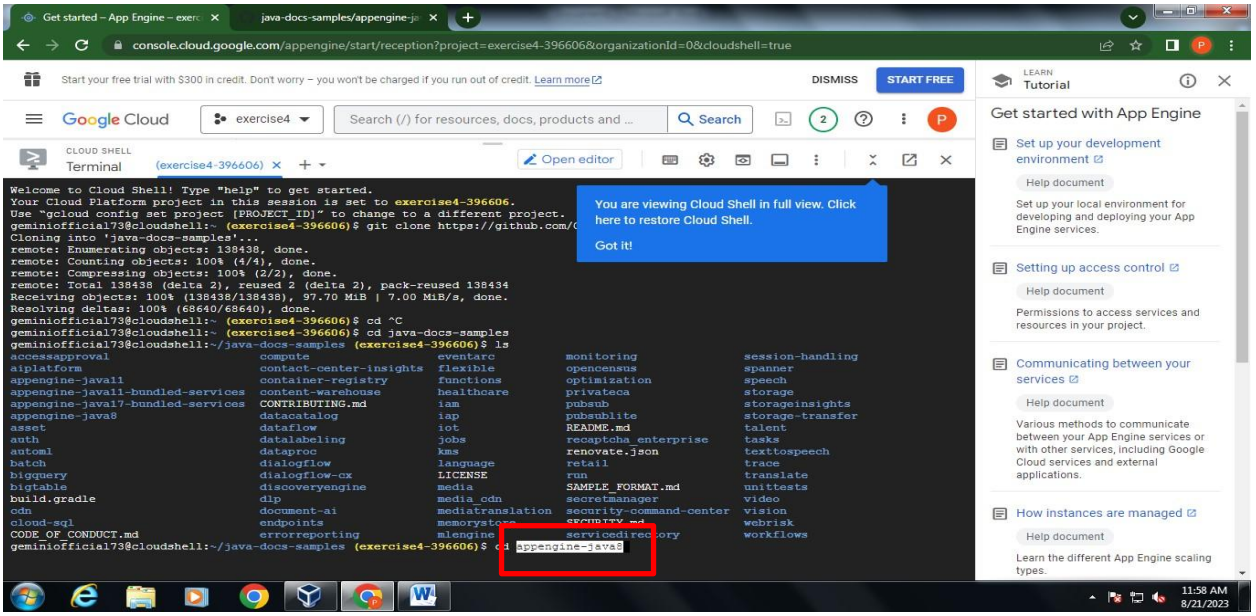
15.Goto cloud shell

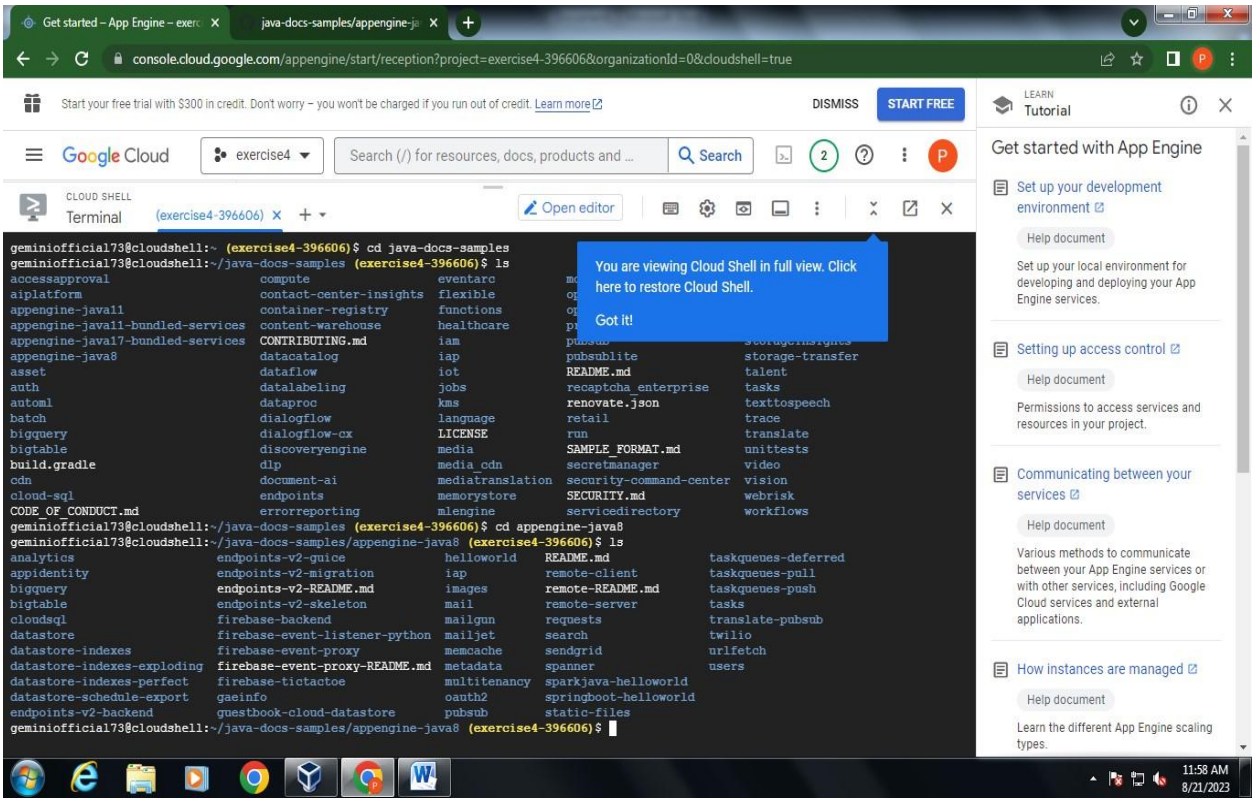


16.type the commands in shell. cd

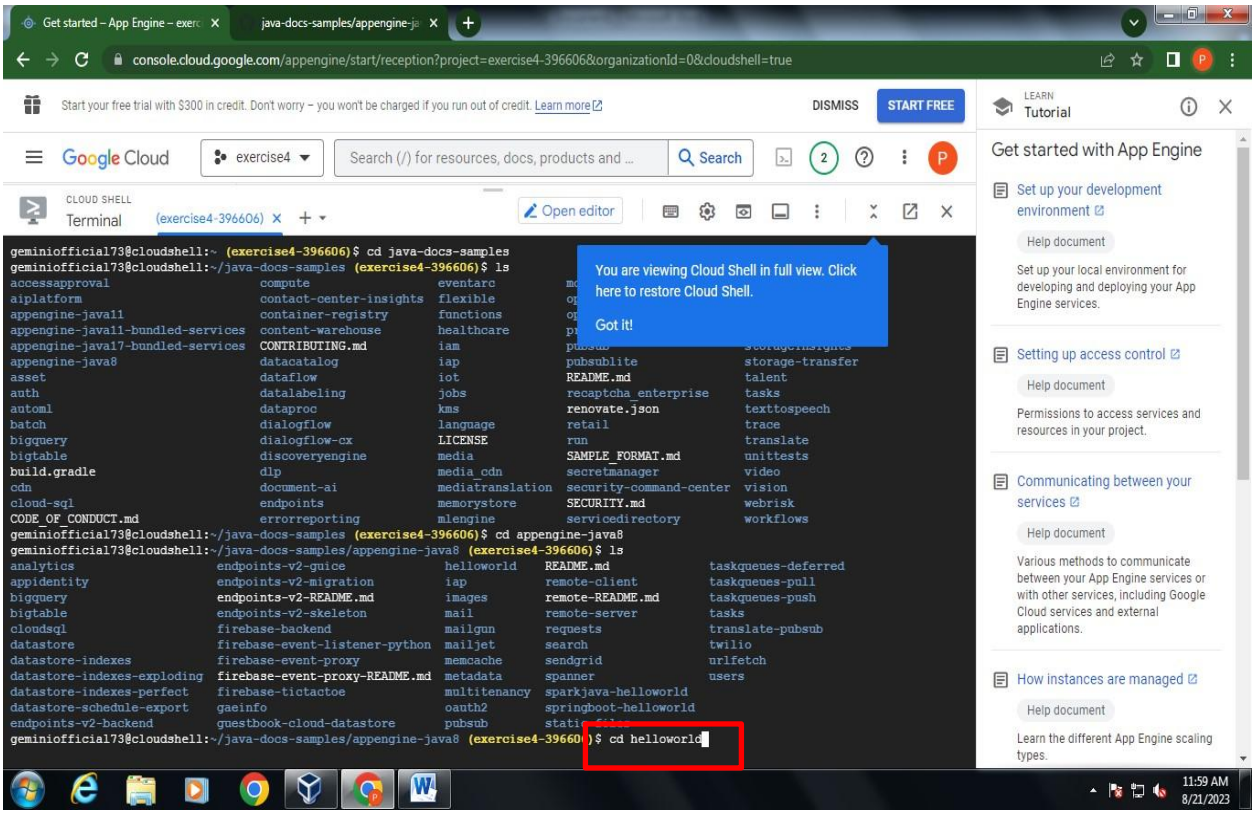
appengine-java8

ls

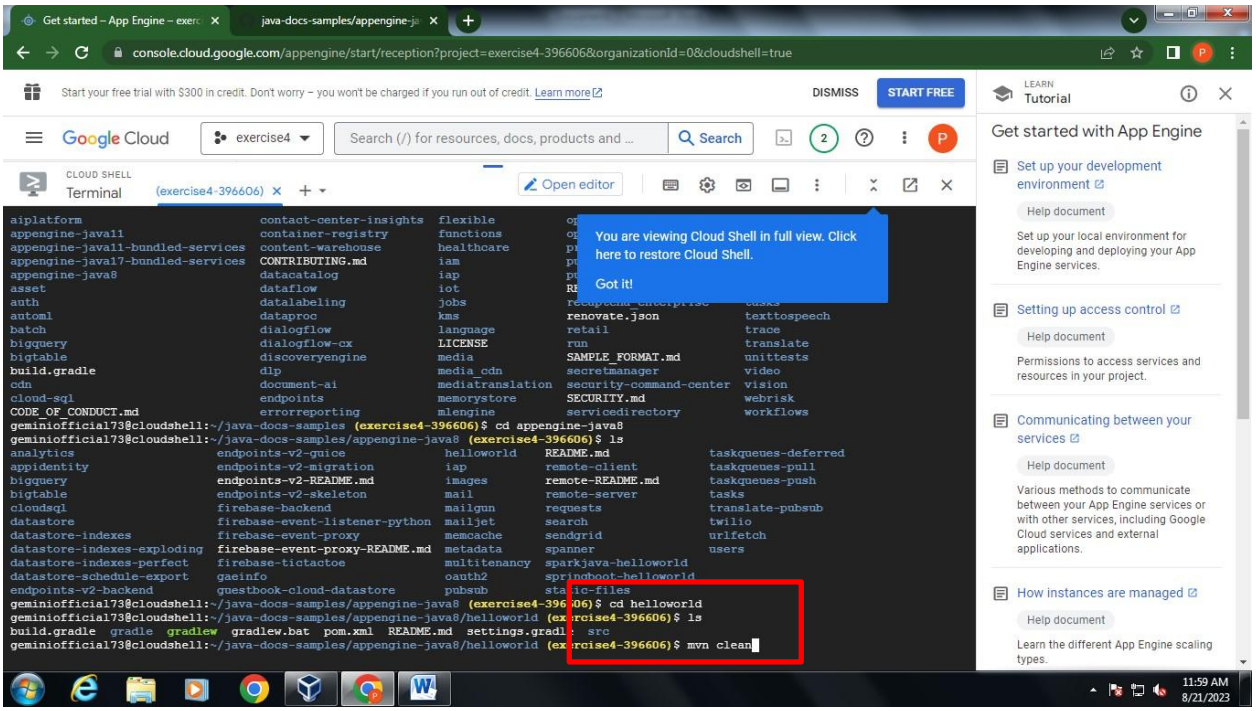


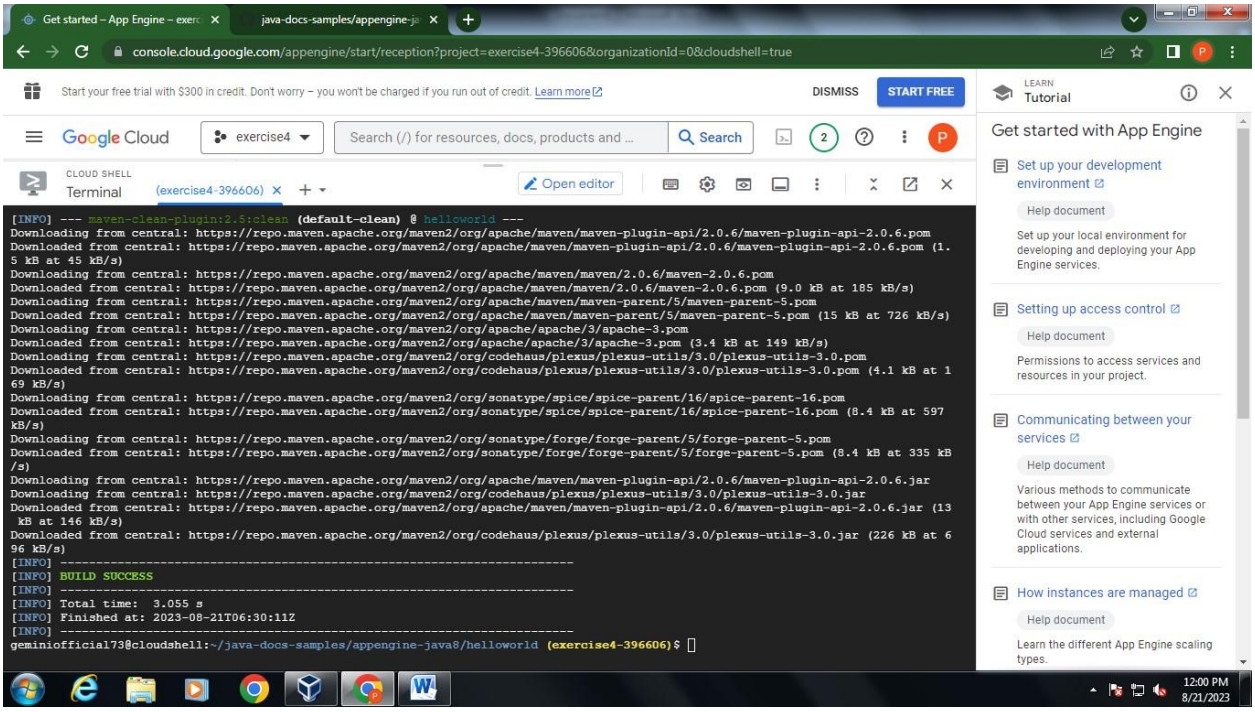


17.cd helloworld



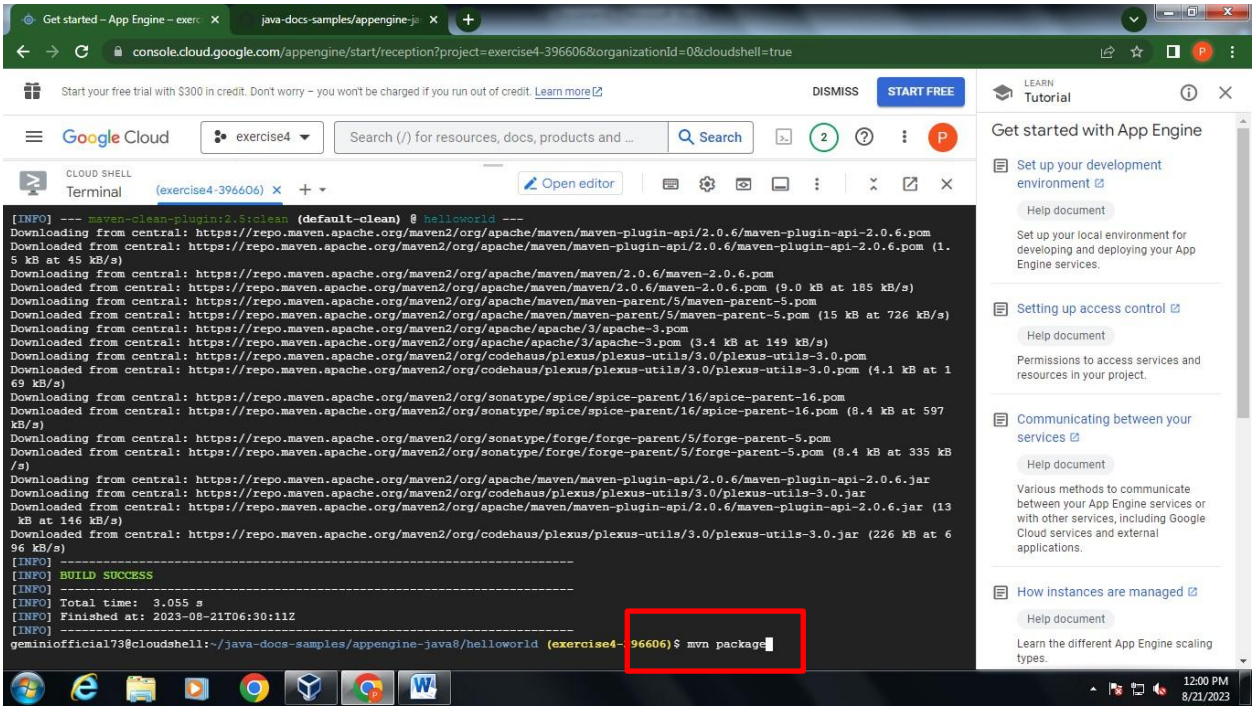
18.type for Maven clean
mvn clean

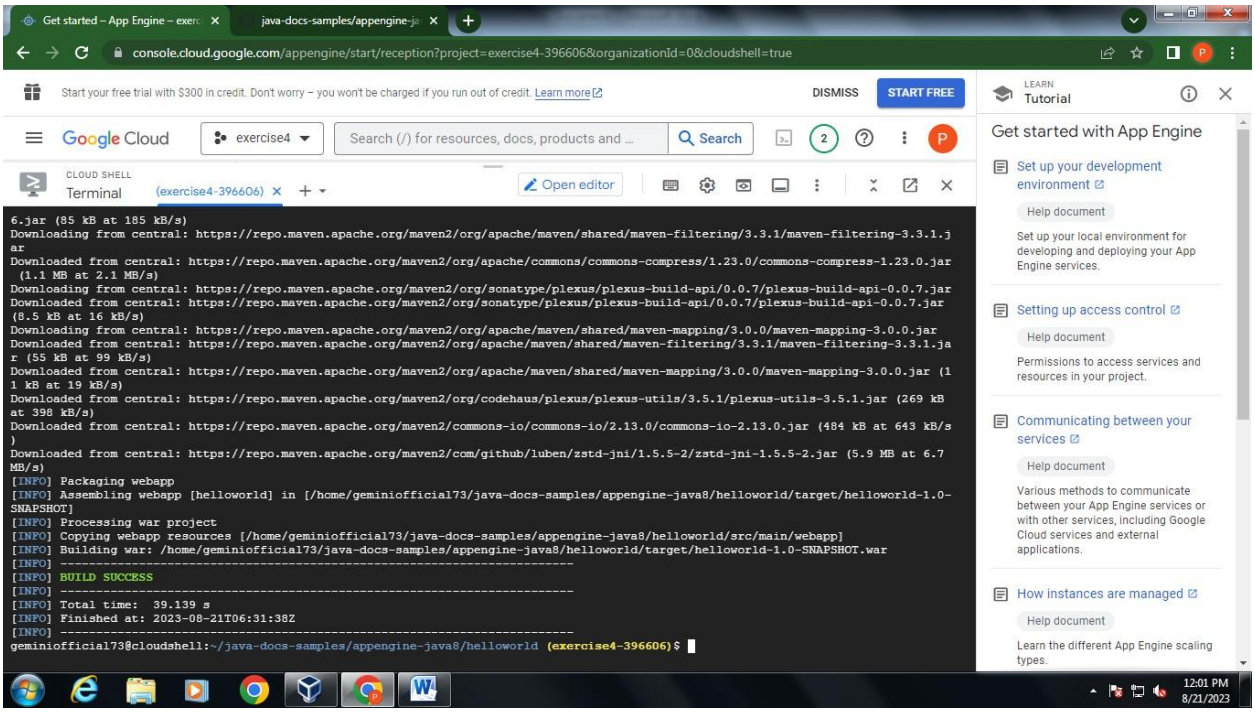


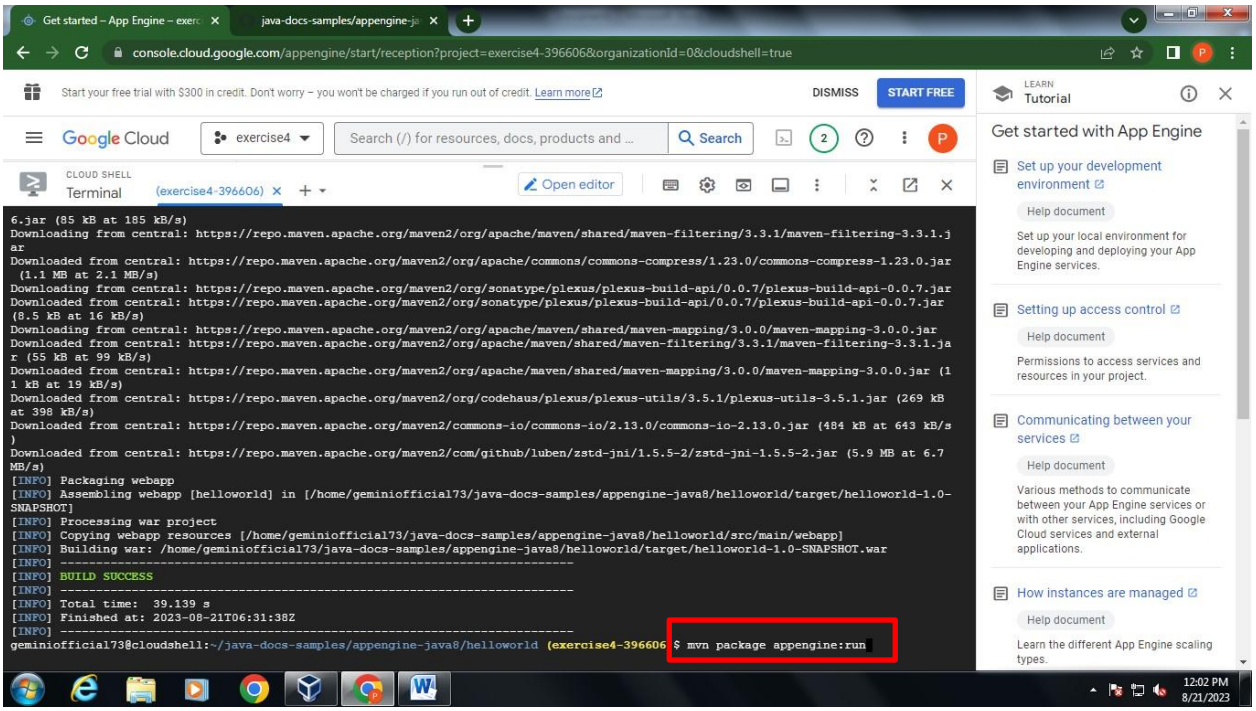


19.In cloud shell.type commands to download maven packages mvn package

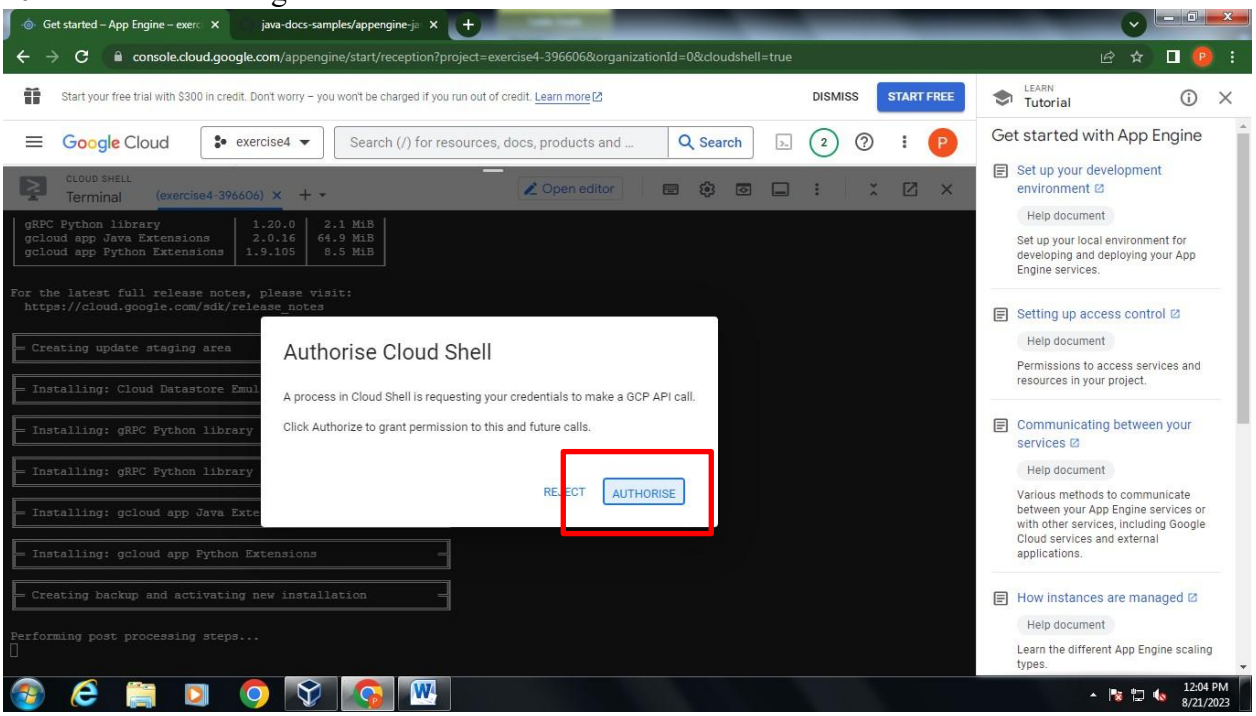
mvn package appengine:run

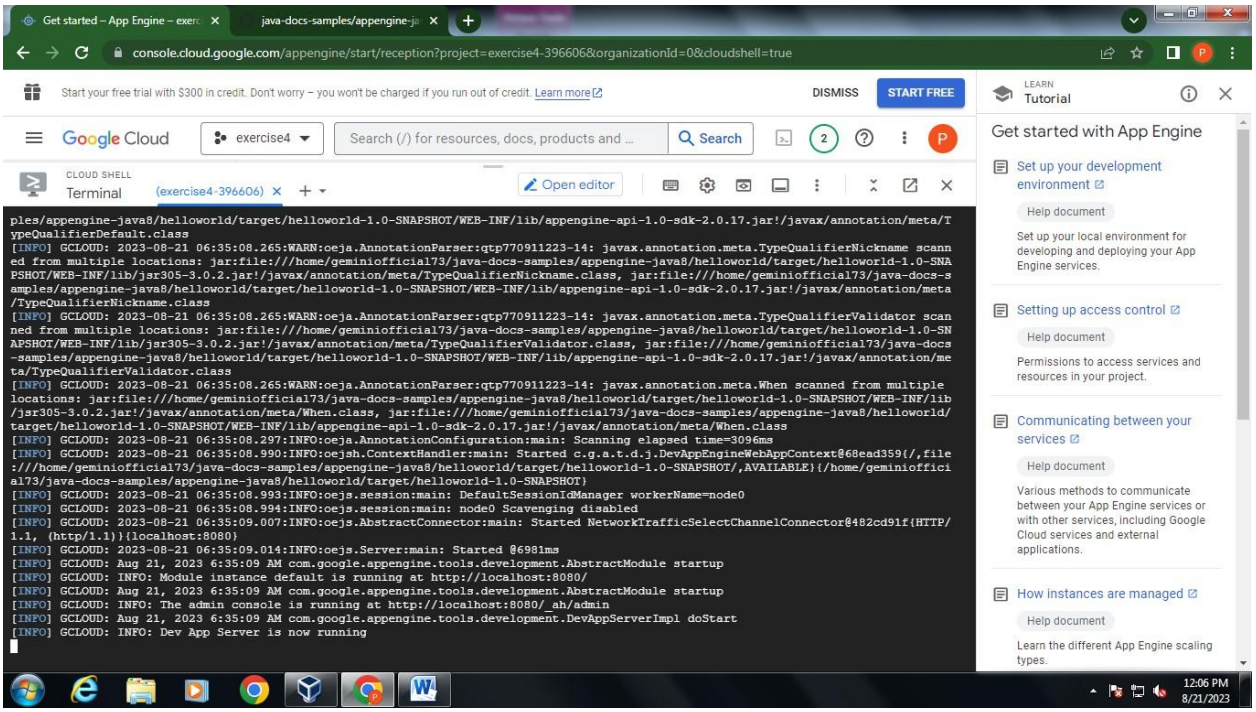




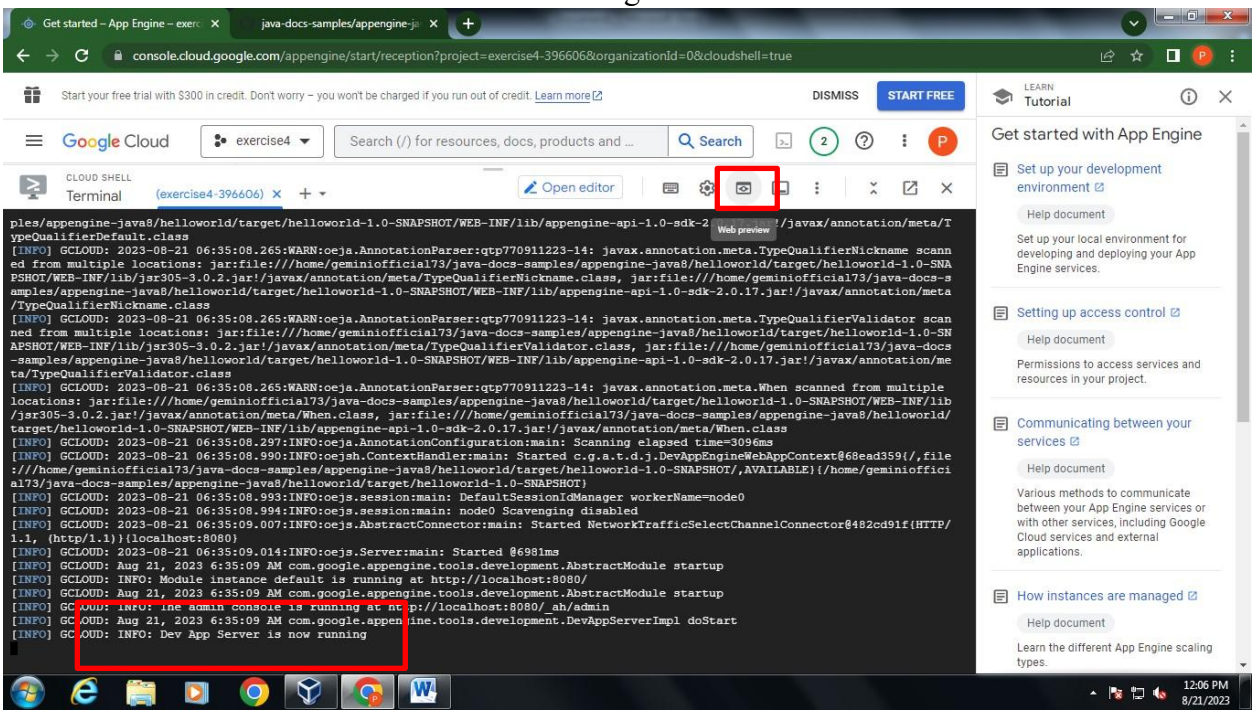


20.click authorize to get an authorization

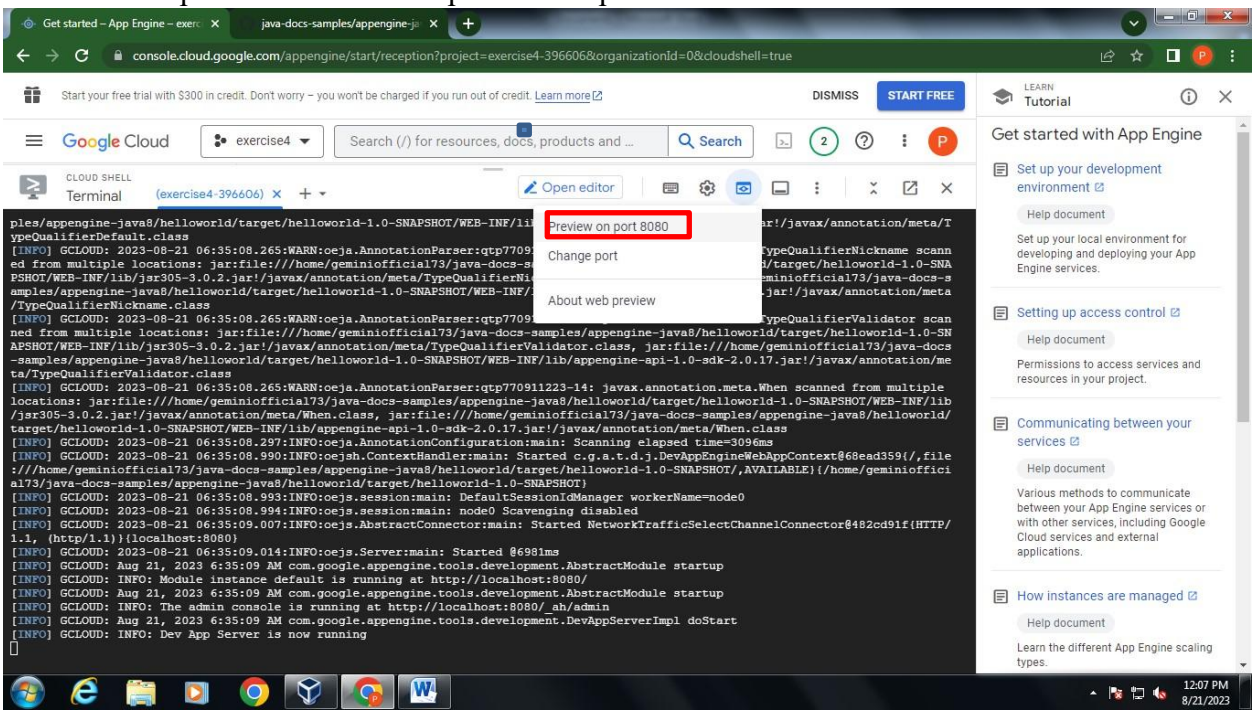




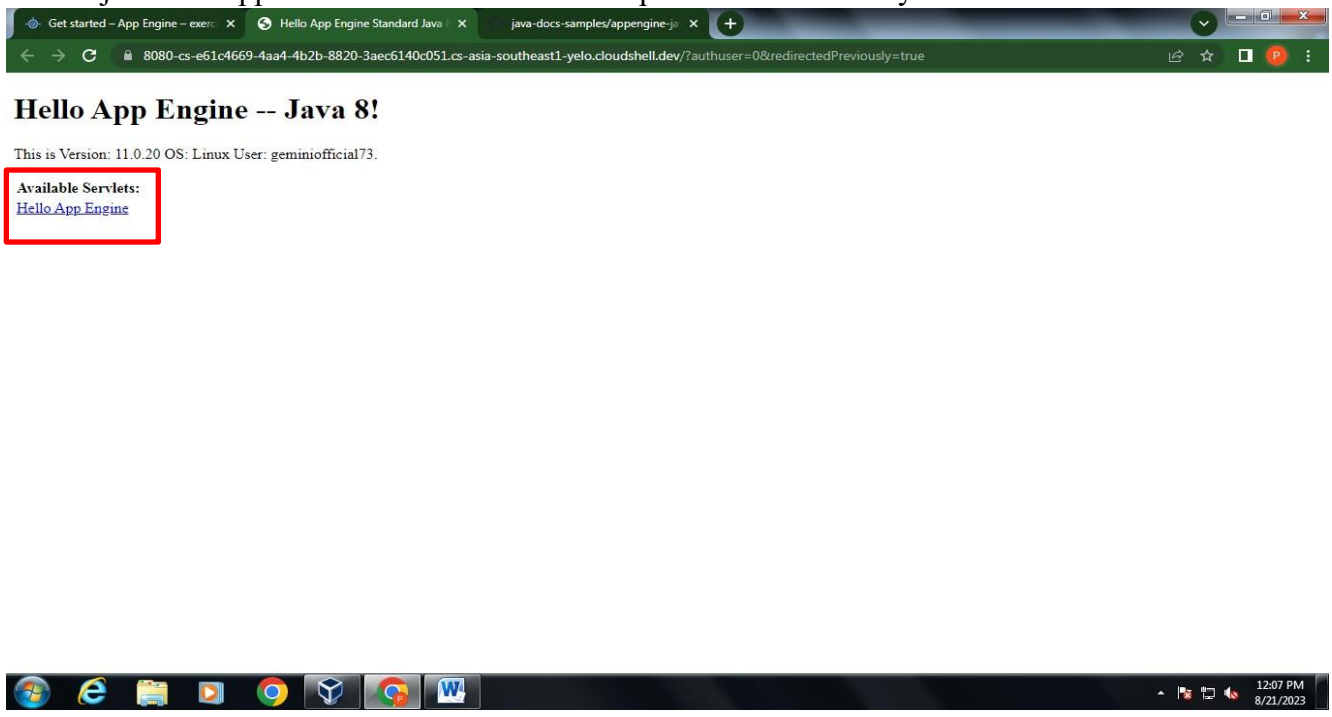
21. Now the device server from Maven is running

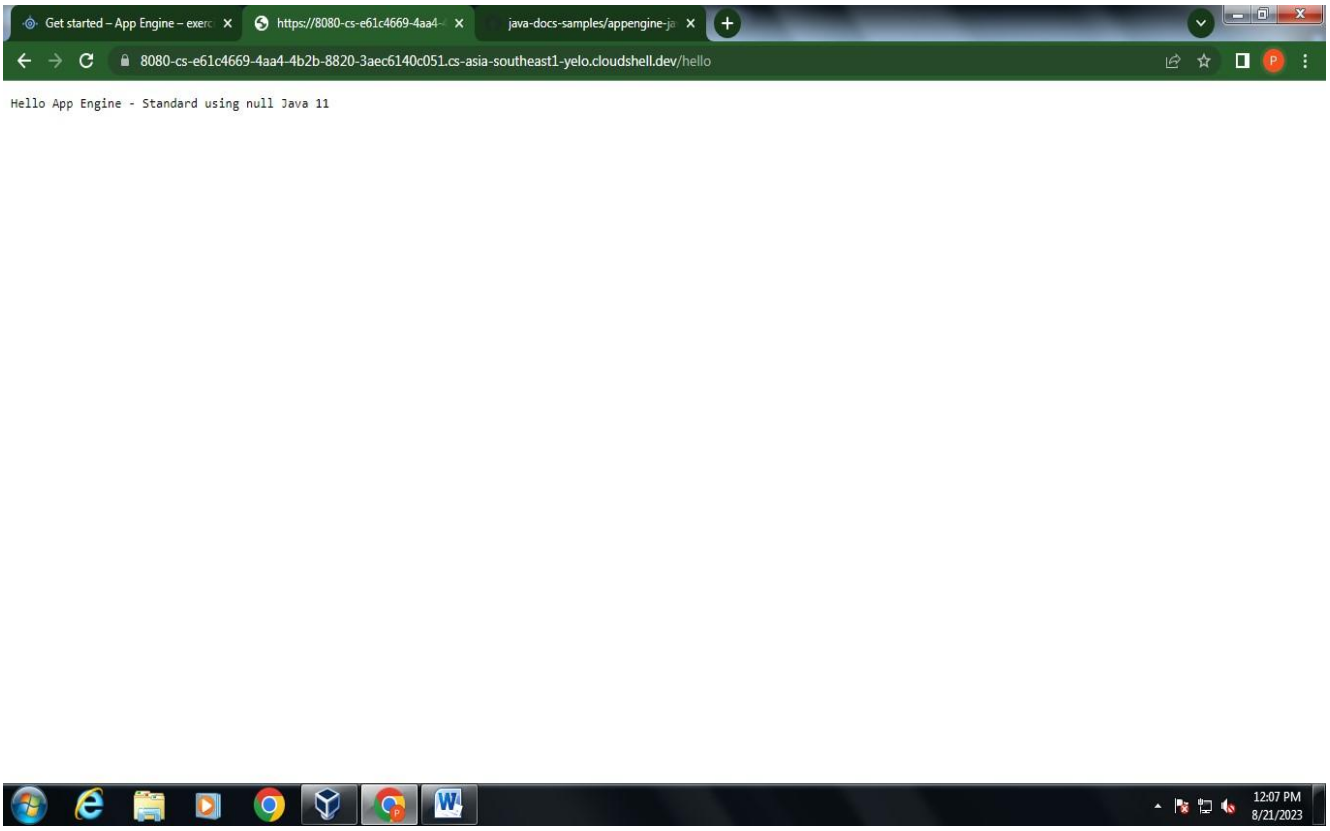


22.Click web preview and select " preview on port 8080"



23.The java web application is launched in cloud platform successfully **OUTPUT:**





RESULT:

Thus the project with the help of GAE launcher to launch web application Helloworld in java is executed successfully.

EX.NO-5: **SIMULATE A CLOUD SCENARIO USING CLOUDSIM AND**
DATE: **RUN A SCHEDULING ALGORITHM THAT IS NOT PRESENT IN CLOUDSIM.**

AIM:

To execute the program by simulate a cloud scenario using cloudsim and run a scheduling program that is not present in cloudsim.

PROCEDURE:

Step 1:

Java installation

Install java setup

C:\Program Files\Java\jdk-13.0.2

Set path variable for java

Goto ->Edit the System Environment variables ->select Environment variable -> select path in system variable -> edit ->new -> C:\Program Files\Java\jdk-13.0.2\bin -> ok **Step**

2:

Download CloudSim 3.0.3

Goto the link

https://github.com/Cloudslab/cloudsim/releases

https://github.com/Cloudslab/cloudsim/releases/download/cloudsim-3.0.3/cloudsim-3.0.3.zip

Step 3:

Eclipse IDE installation https://www.eclipse.org/downloads/

https://www.eclipse.org/downloads/download.php?file=/oomph/epp/202306/R/eclipse-inst-jre-win64.exe

Step 4:

Run Cloudsim in Eclipse

Build a java project(Scheduling algorithm) with CloudSiM 3.0.3 Folder.

ECLIPSE IDE INSTALLATION STEPS:

1. Download the Eclipse Installer.

Download Eclipse Installer from http://www.eclipse.org/downloads

Eclipse is hosted on many mirrors around the world. Please select the one closest to you and start to download the Installer. 2. Start the Eclipse Installer executable**.**

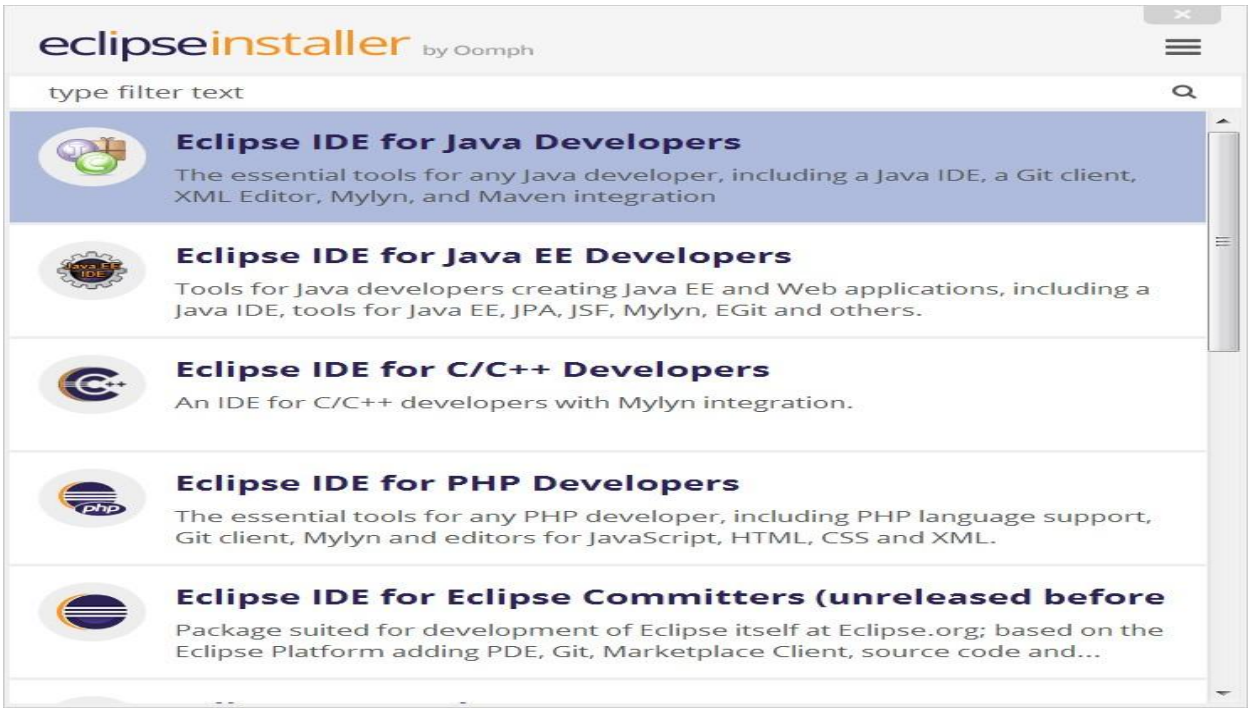
For Windows users, after the Eclipse Installer executable has finished downloading it should be available in your download directory. Start the Eclipse Installer executable. You may get a security warning to run this file. If the Eclipse Foundation is the Publisher, you are good to select Run.



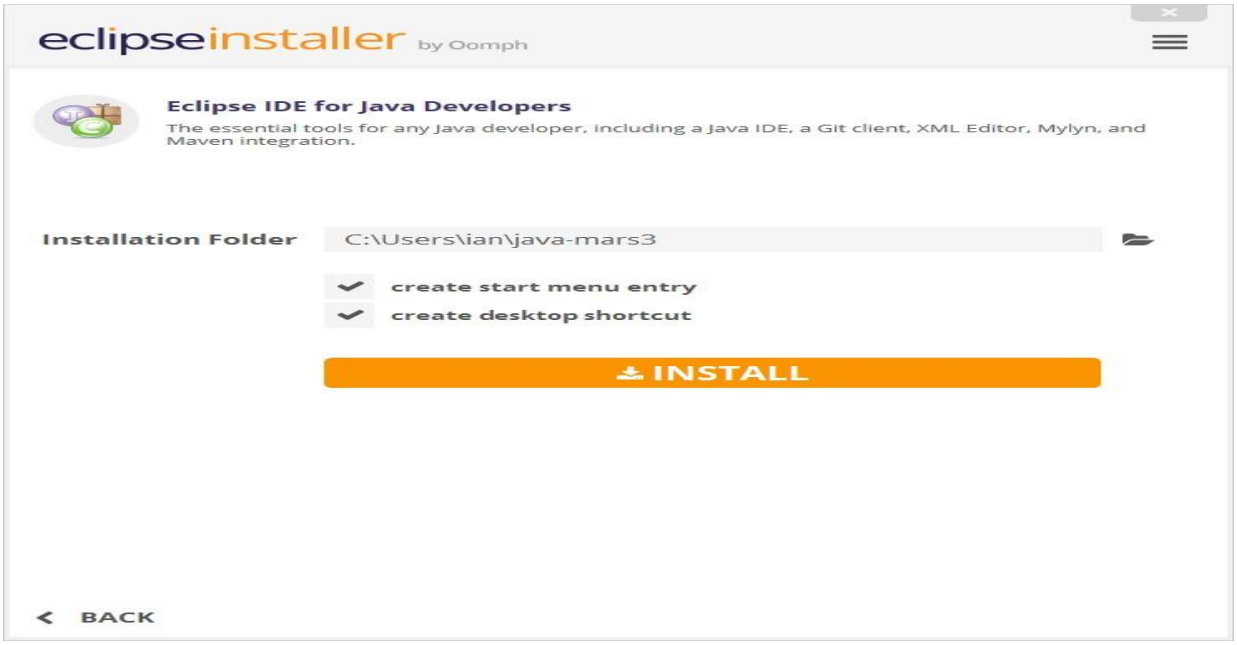
3. Select the package to install.

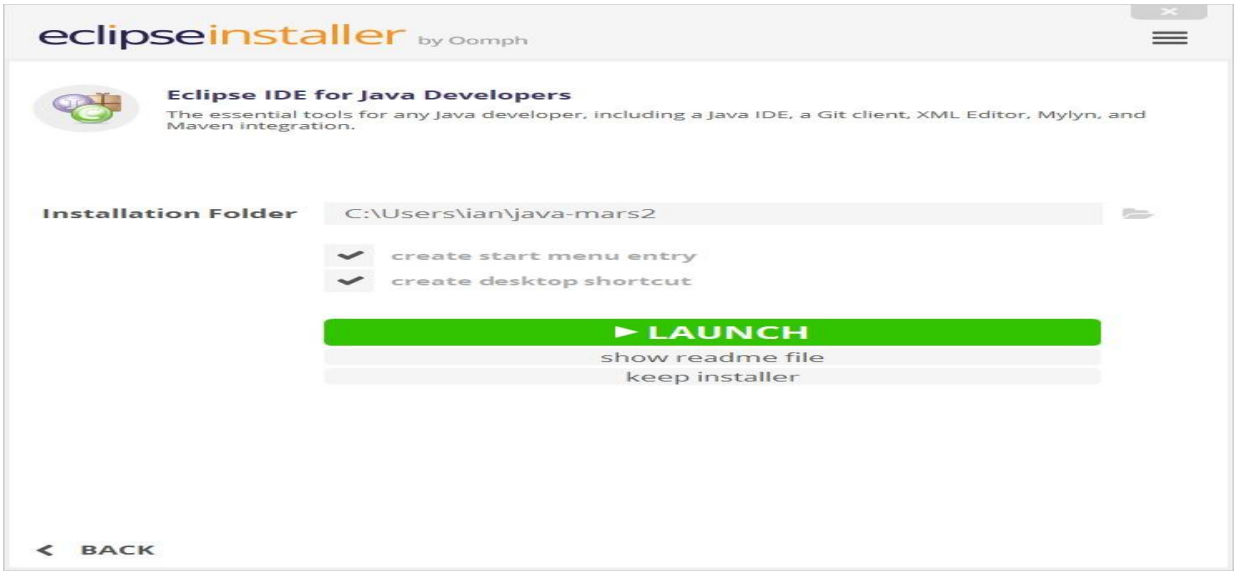
The new Eclipse Installer shows the packages available to Eclipse users. You can search for the package you want to install or scroll through the list.

Select and click on the package you want to install.



4. Select your installation folder
Specify the folder where you want Eclipse to be installed. The default folder will be in your User directory.
Select the 'Install' button to begin the installation.

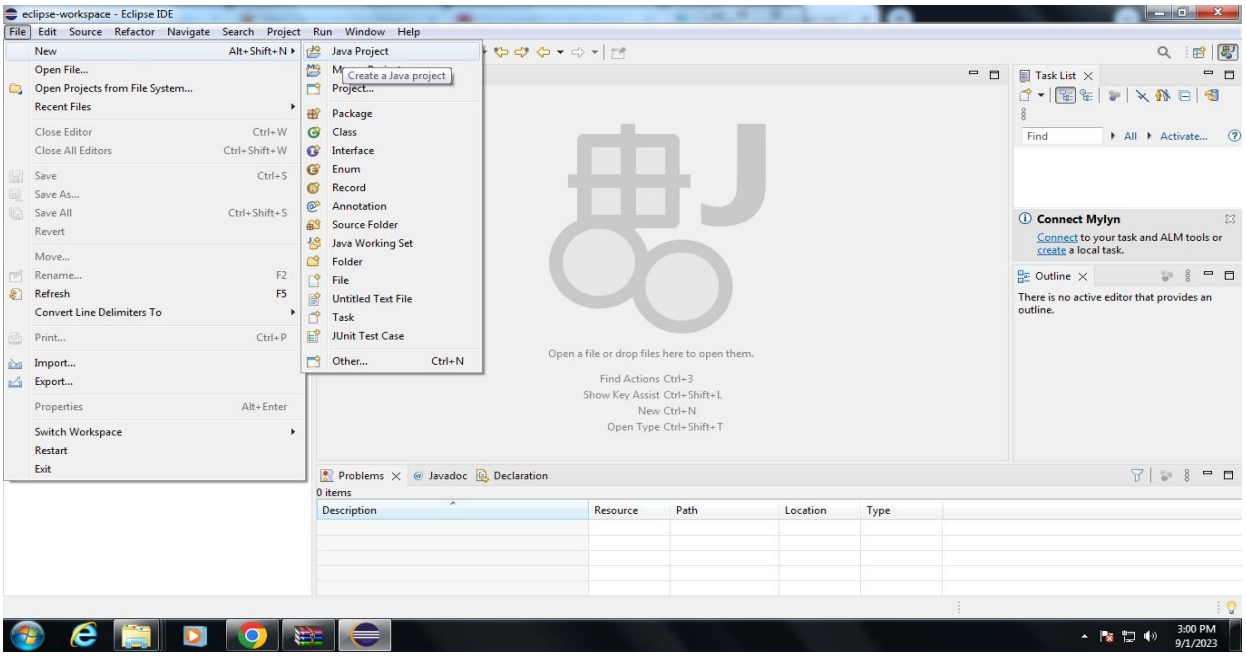




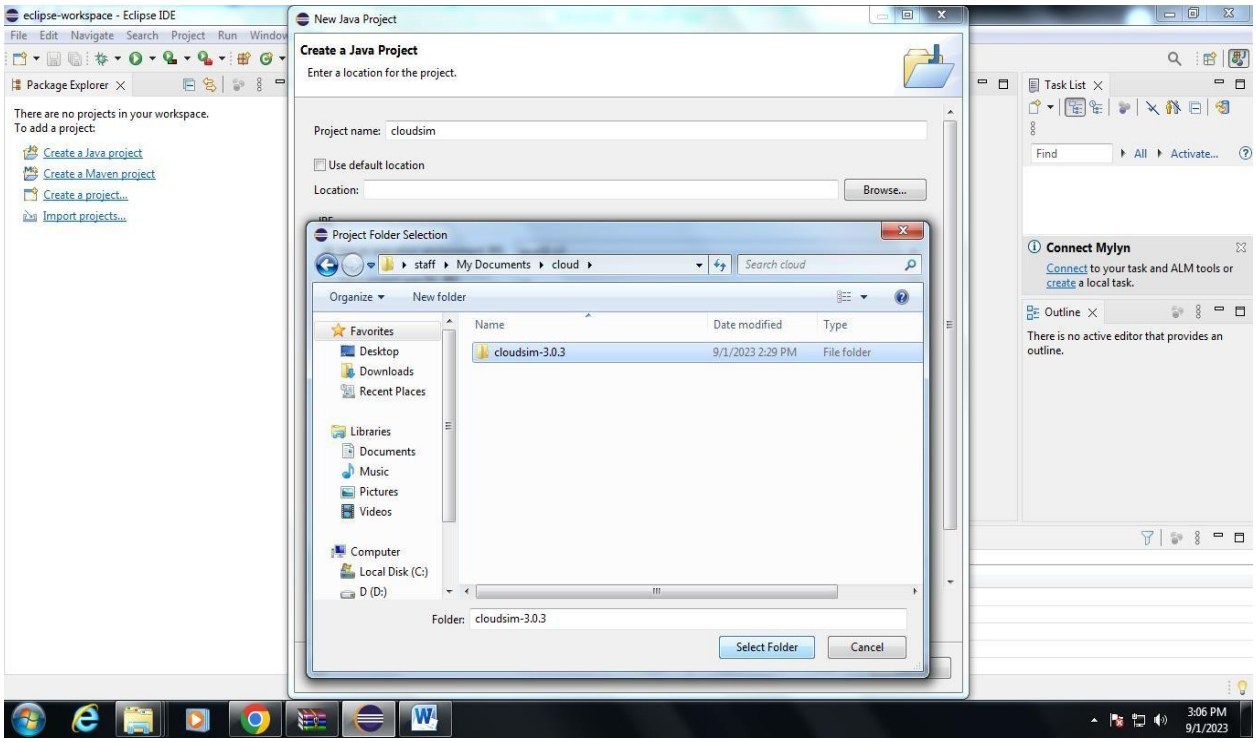
5. Launch Eclipse.
Once the installation is complete you can now launch Eclipse. The Eclipse Installer has done it's work.

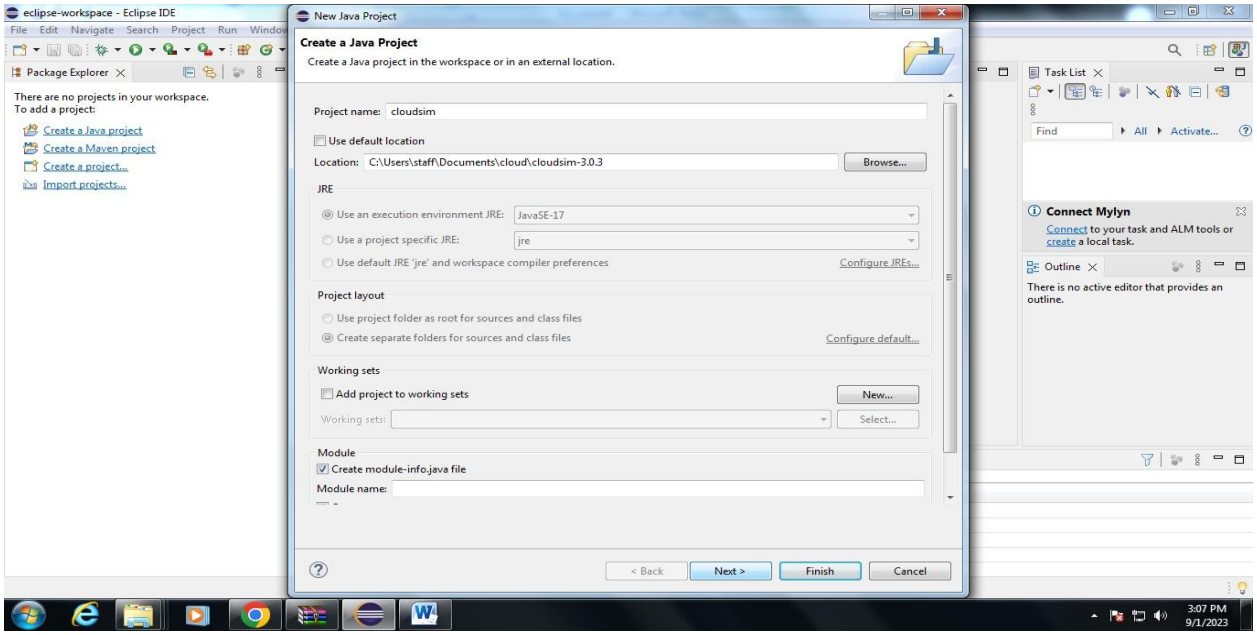
RUN CLOUDSIM IN ECLIPSE:

1. File -> new -> java project

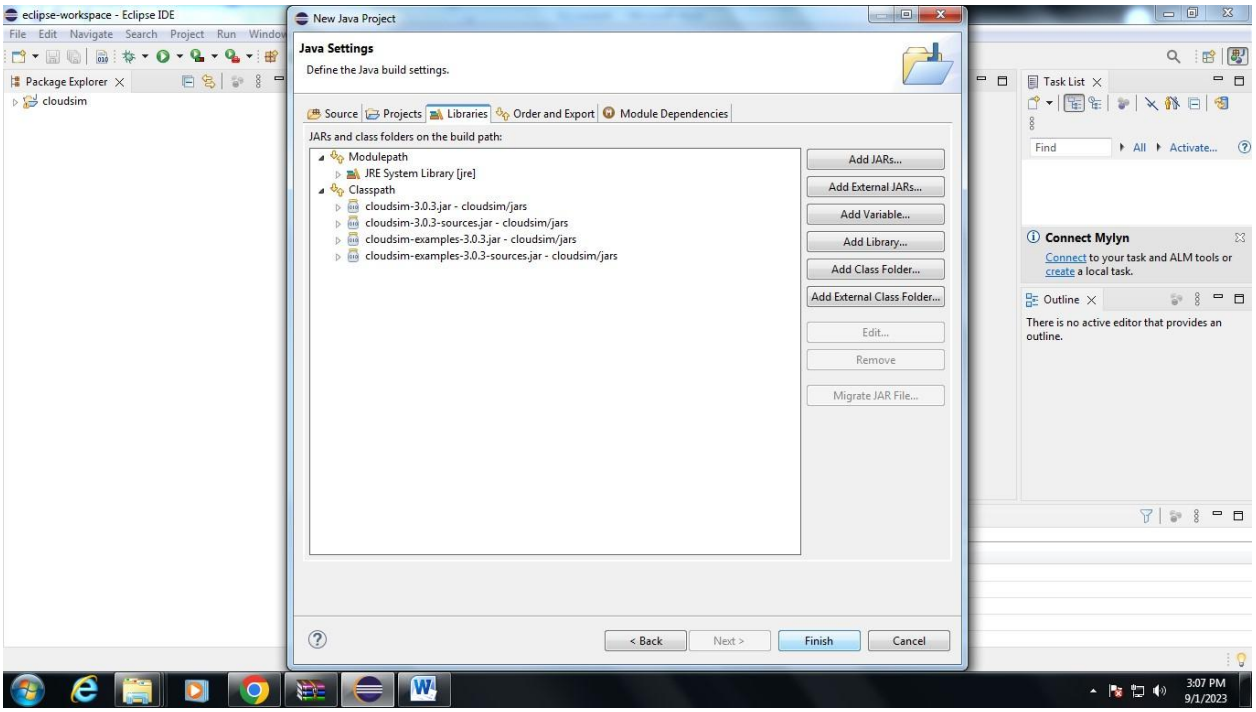


2. Select location
   Set this path
   C:\Users\ICON\Documents\installation\cloudsim-3.0.3


eclipse-workspace - Eclipse IDE
New Java Project
Create a Java Project
Enter a location for the project.
Project name: cloudsim
Use default location
Location:
Browse...
Project Folder Selection
staff ▸ My Documents ▸ cloud ▸
Search cloud
Organize
New folder
Favorites
Desktop
Downloads
Recent Places
Libraries
Documents
Music
Pictures
Videos
Computer
Local Disk (C:)
D (D:)
Name
Date modified
Type
cloudsim-3.0.3
9/1/2023 2:29 PM
File folder
Folder: cloudsim-3.0.3
Select Folder
Cancel
Package Explorer
There are no projects in your workspace.
To add a project:
Create a Java project
Create a Maven project
Create a project...
Import projects...
Task List
Find
All
Activate...
Connect Mylyn
Connect to your task and ALM tools or create a local task.
Outline
There is no active editor that provides an outline.
3:06 PM
9/1/2023

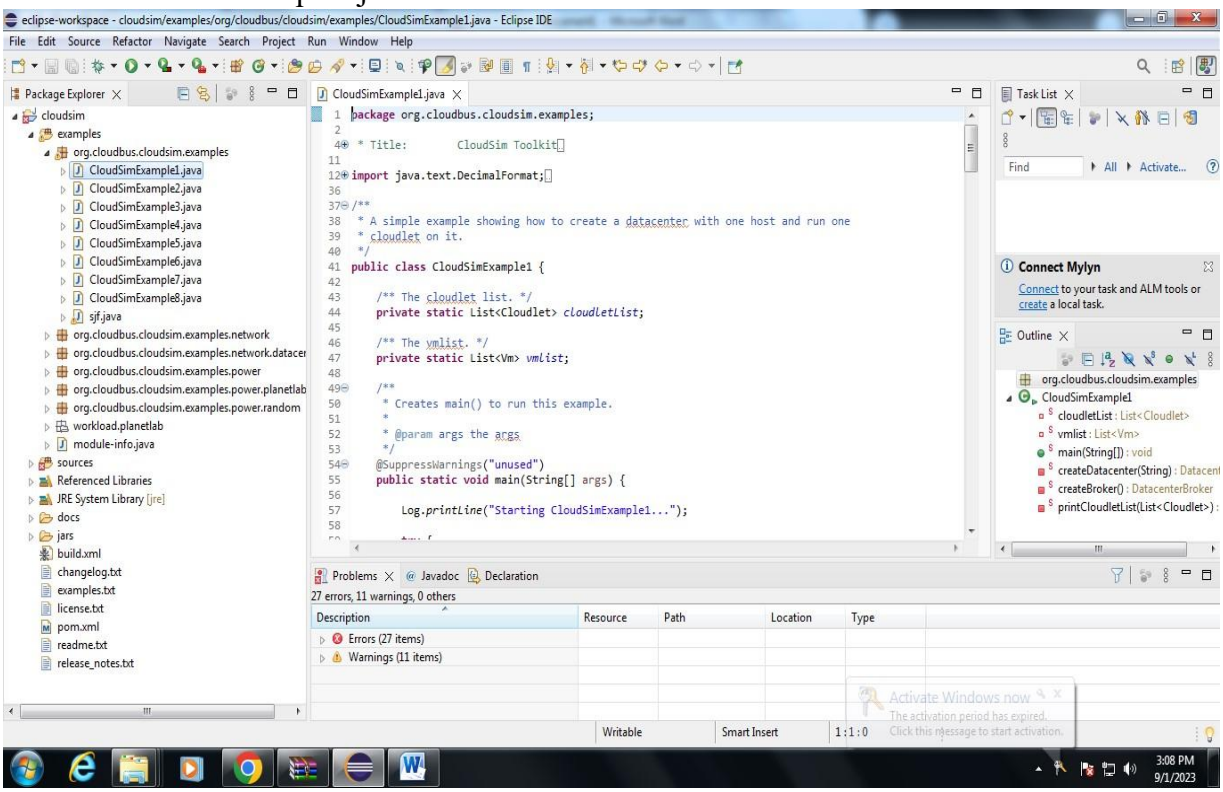

eclipse-workspace - Eclipse IDE
New Java Project
Create a Java Project
Create a Java project in the workspace or in an external location.
Project name: cloudsim
Use default location
Location: C:\Users\staff\Documents\cloud\cloudsim-3.0.3
Browse...
JRE
Use an execution environment JRE: JavaSE-17
Use a project specific JRE: jre
Use default JRE 'jre' and workspace compiler preferences
Configure JREs...
Project layout
Use project folder as root for sources and class files
Create separate folders for sources and class files
Configure default...
Working sets
Add project to working sets
New...
Working sets:
Select...
Module
Create module-info.java file
Module name:
< Back
Next >
Finish
Cancel
Package Explorer
There are no projects in your workspace.
To add a project:
Create a Java project
Create a Maven project
Create a project...
Import projects...
Task List
Find
All
Activate...
Connect Mylyn
Connect to your task and ALM tools or create a local task.
Outline
There is no active editor that provides an outline.
3:07 PM
9/1/2023

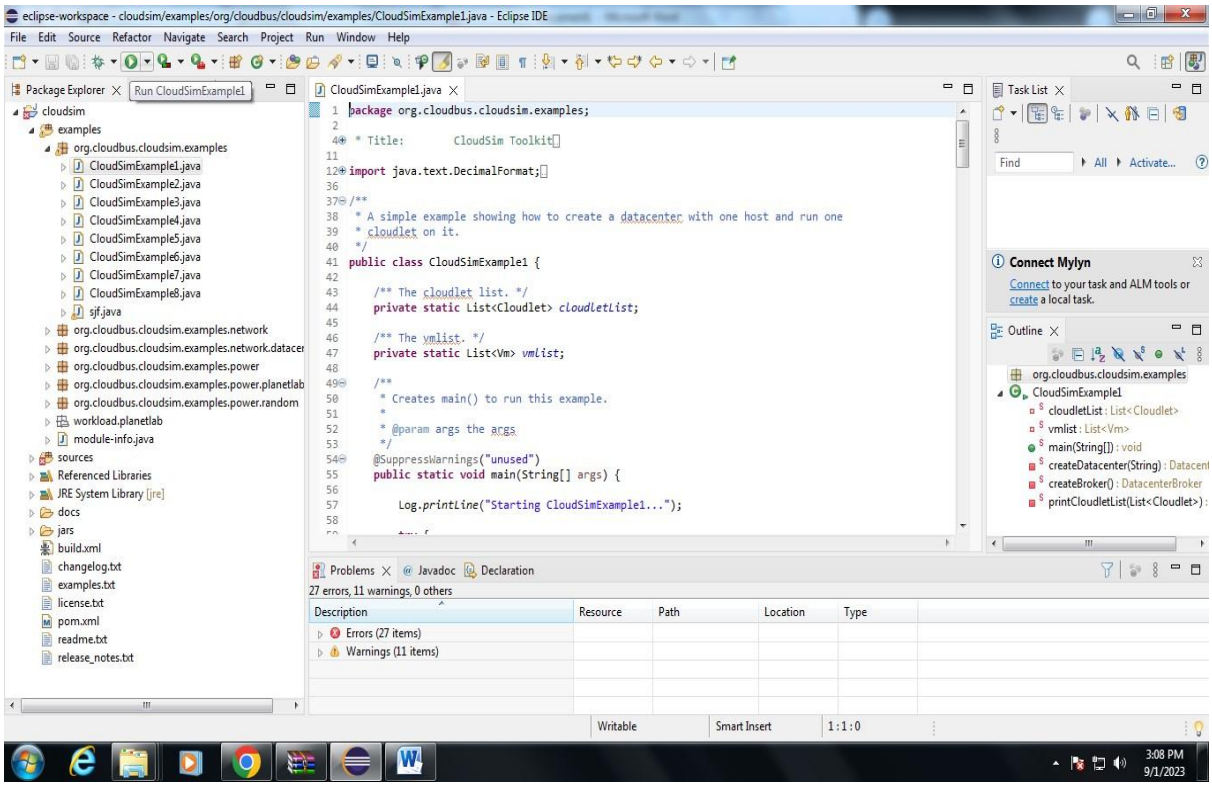
3. Click next



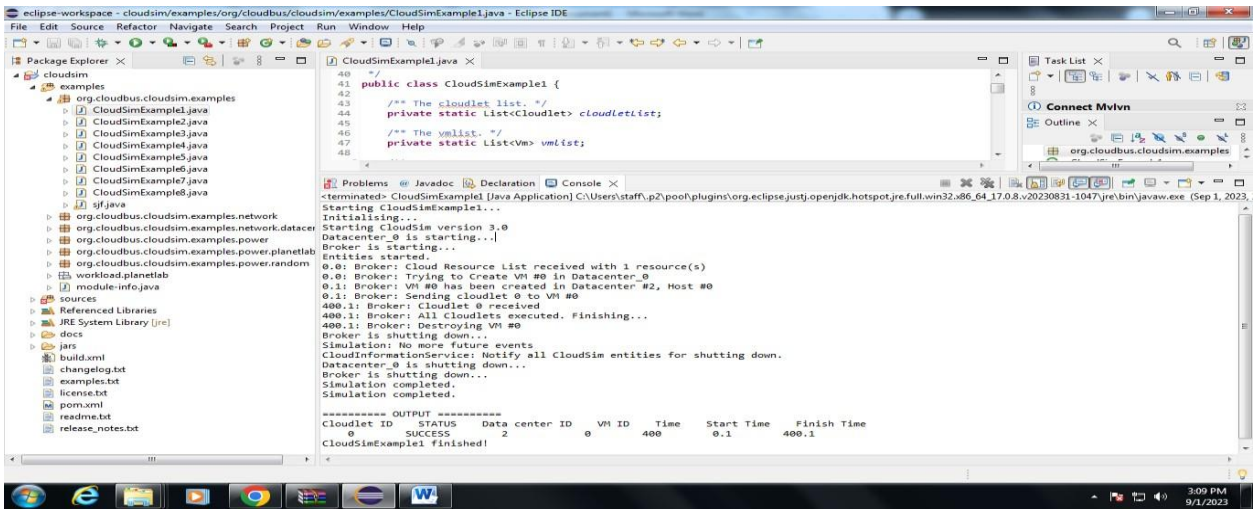
4. Cloudsim libraries are added over here Click finish

5. In example of cloudsim folder, run the first program which is present in cloudsim CloudSimExample1.java

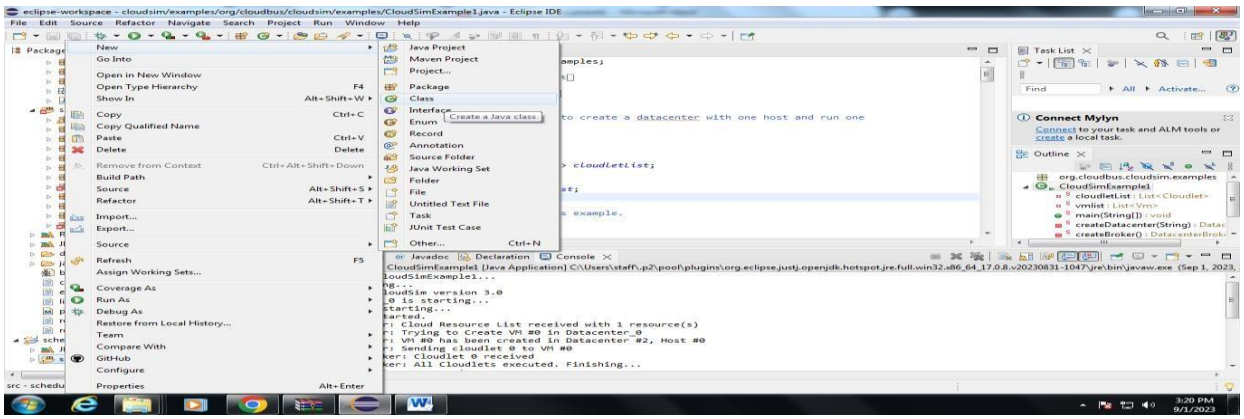




OUTPUT: (PROGRAM ALREADY IN CLOUDSIM)



6. CloudSimExample1.java successfully executed
7. Next we have to add and execute scheduling program in java which is not already present in cloudsim folder
8. Next we have to create a java class for scheduler program under cloud bus package as a java project



9. Default cloudbus package will be selected
10. Give the java program name " sjf "
11. Select the option " public static void main(String{} args) "
12. Click finish
13. Open that java class program sjf.java which is created
14. Type the scheduler program in that file
15. Run that program

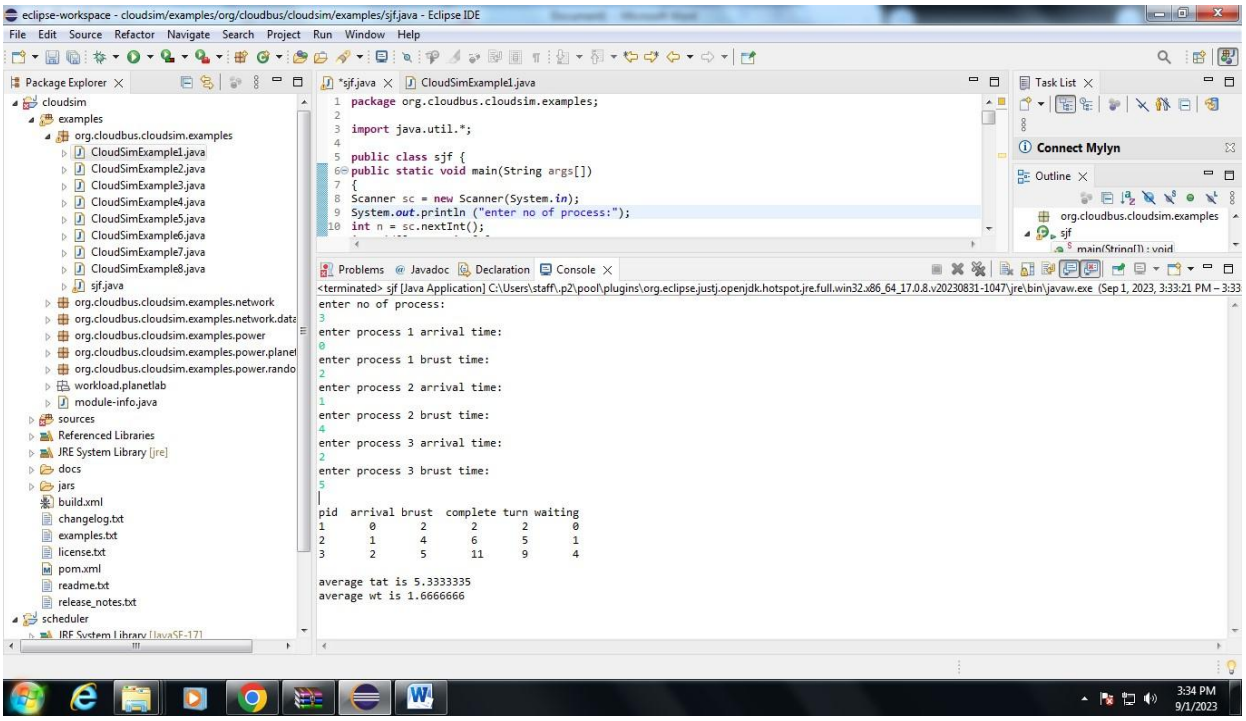
SJF.JAVA:

```
package org.cloudbus.cloudsim.examples;
import java.util.*; public
class sjf {
public static void main(String args[])
{
Scanner sc = new Scanner(System.in); System.out.println
("enter no of process:");
int n = sc.nextInt(); int pid[] = new int[n]; int at[]
= new int[n]; // at means arrival time int bt[] =
new int[n]; // bt means burst time int ct[] = new
int[n]; // ct means complete time int ta[] = new
int[n]; // ta means turn around time int wt[] =
new int[n];  //wt means waiting time
int f[] = new int[n];  // f means it is flag it checks process is completed or not int
st=0, tot=0;
float avgwt=0, avgta=0;

for(int i=0;i<n;i++)
```

```
{
System.out.println ("enter process " + (i+1) + " arrival time:"); at[i]
= sc.nextInt();
System.out.println ("enter process " + (i+1) + " brust time:");
bt[i] = sc.nextInt(); pid[i] = i+1; f[i] = 0; } boolean a = true;
while(true)
{
int c=n, min=999;
if (tot == n) // total no of process = completed process loop will be terminated break;
for (int i=0; i<n; i++)
{
/*
* If i'th process arrival time <= system time and its flag=0 and burst<min
* That process will be executed first
*/
if ((at[i] <= st) && (f[i] == 0) && (bt[i]<min))
{
min=bt[i];
c=i; }
}
/* If c==n means c value can not updated because no process arrival time< system time so
we increase the system time */ if (c==n)
st++; else {
ct[c]=st+bt[c];
st+=bt[c];
ta[c]=ct[c]-at[c];
wt[c]=ta[c]-bt[c];
f[c]=1; tot++;
}
}
System.out.println("\npid  arrival brust  complete turn waiting"); for(int
i=0;i<n;i++)
{ avgwt+=
wt[i]; avgta+=
ta[i];
System.out.println(pid[i]+"\t"+at[i]+"\t"+bt[i]+"\t"+ct[i]+"\t"+ta[i]+"\t"+wt[i]); }
System.out.println ("\naverage tat is "+ (float)(avgta/n));
System.out.println ("average wt is "+ (float)(avgwt/n)); sc.close();
}
}
```

OUTPUT 1: (WHICH IS NOT ADDED IN CLOUDSIM)



RESULT:

Thus the simulate a cloud scenario using cloudsim and run a scheduling algorithm that is not present in cloudsim was successfully completed.

EX.NO-6 **FIND A PROCEDURE TO TRANSFER THE FILES FROM**
DATE: **ONE VIRTUAL MACHINE TO ANOTHER VIRUTAL MACHINE**

AIM:

To Find a procedure to transfer the files from one virtual machine to another virtual machine.

PROCEDURE:

Steps:

1. Select Oracle virtual box
2. Go to file and import Appliance
3. Select Open-nebula sandbox
4. Open Browser, type localhost:9869
5. Login using username: oneadmin, password: opennebula 6. Then follow the steps to migrate VMs

a. Click on infrastructure

b. Select clusters and enter the cluster name

c. Then select host tab, and select all host

d. Then select Vnets tab, and select all Vnet

e. Then select datastores tab, and select all datastores

f. And then choose host under infrastructure tab

g. Click on + symbol to add new host, name the host then click on create. 7. On instances, select VMs to migrate then follow the steps

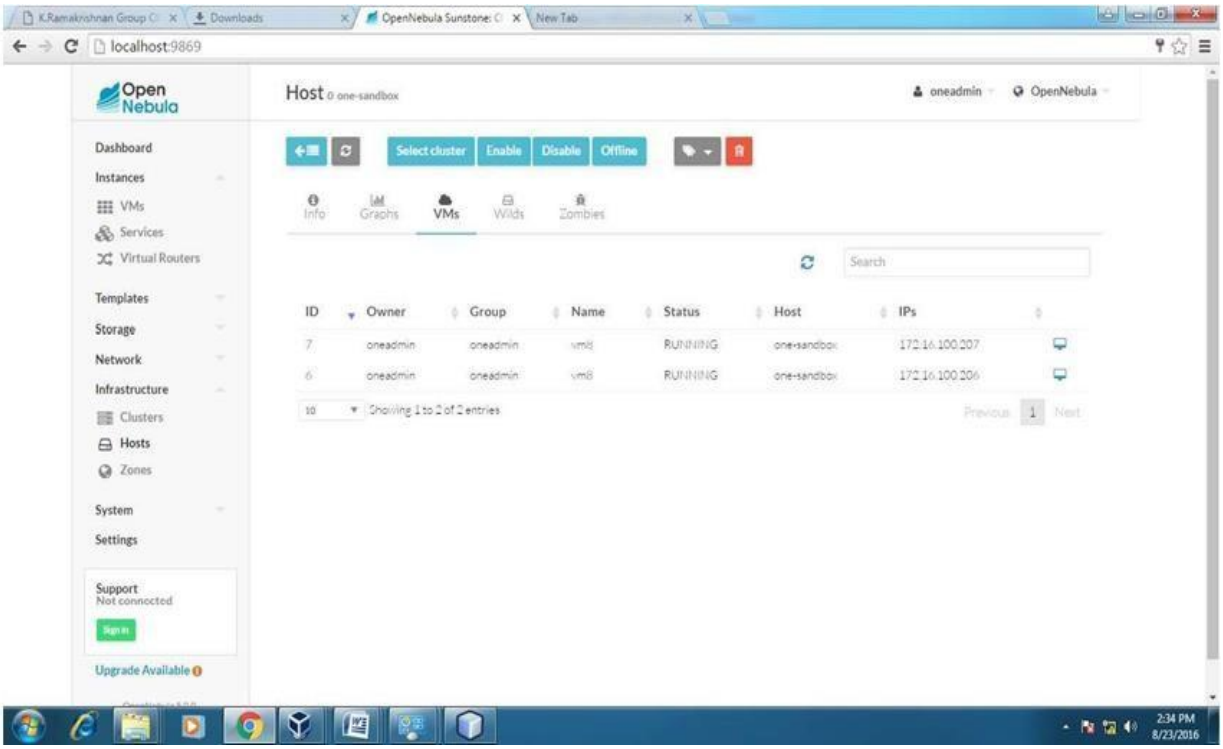
a. Click on 8 th icon, the drop-down list display

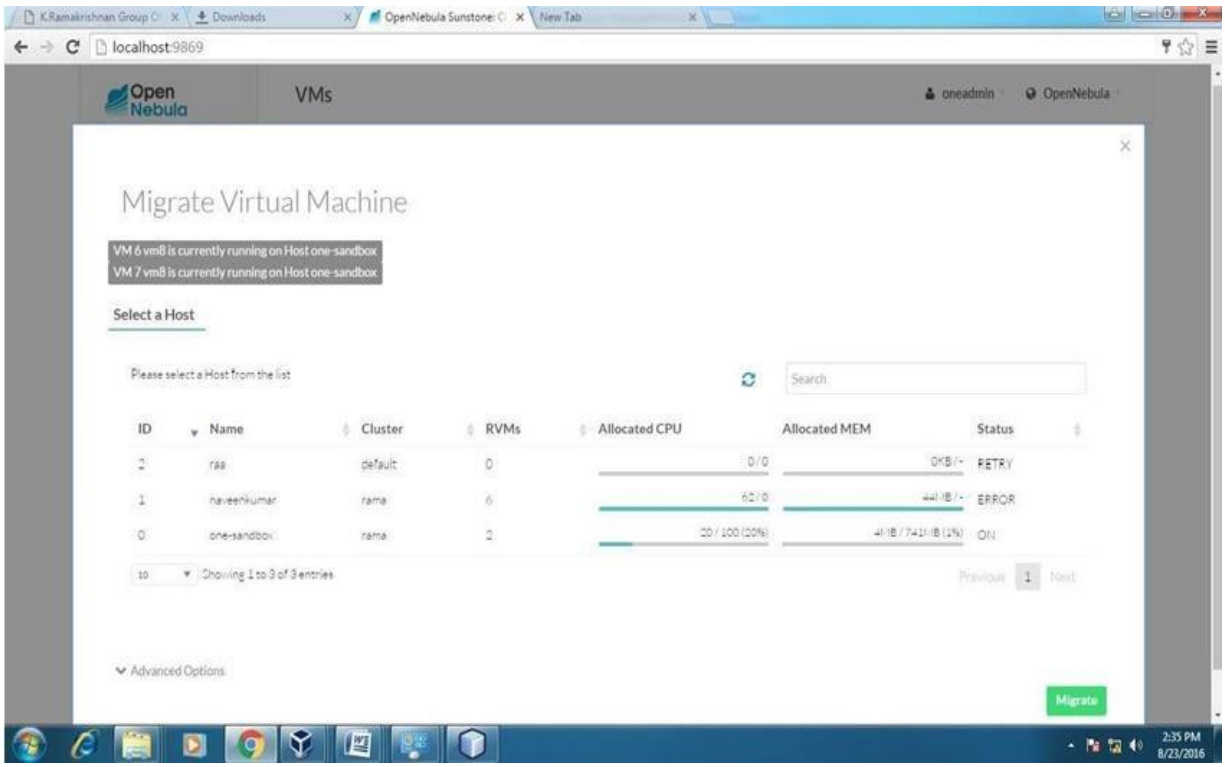
b. Select migrate on that, the popup window display

c. On that select the target host to migrate then click on migrate.

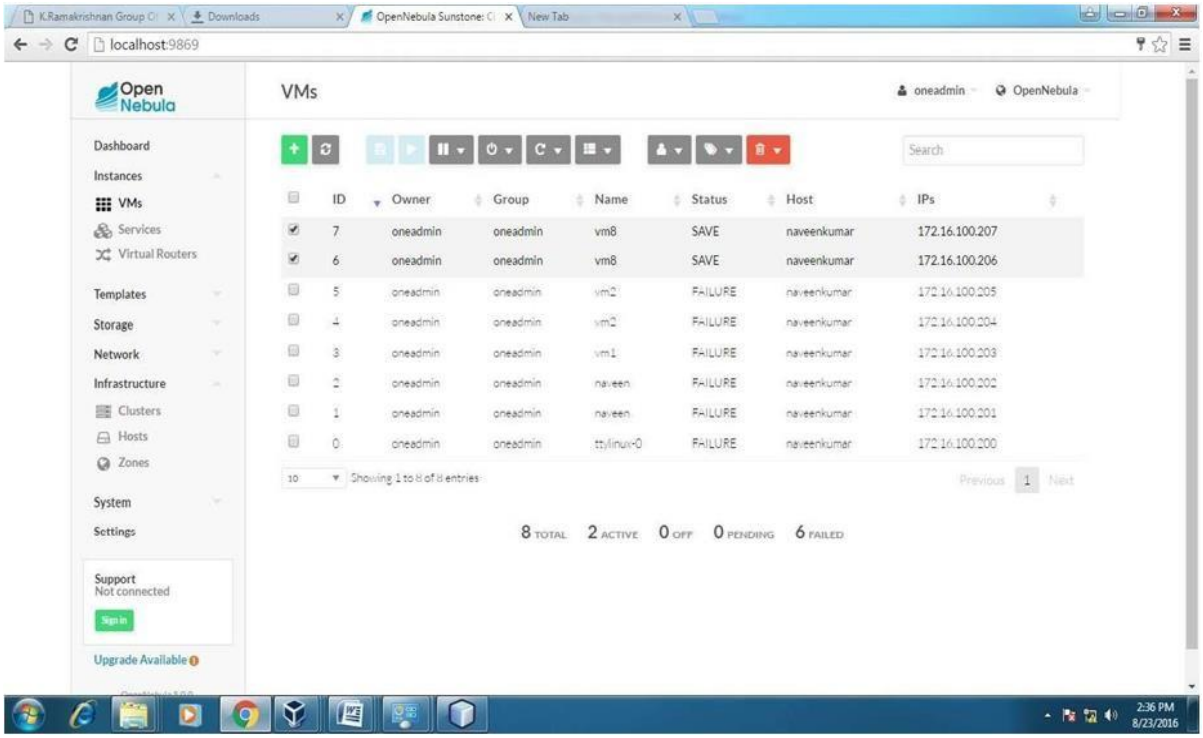
BEFORE MIGRATION :

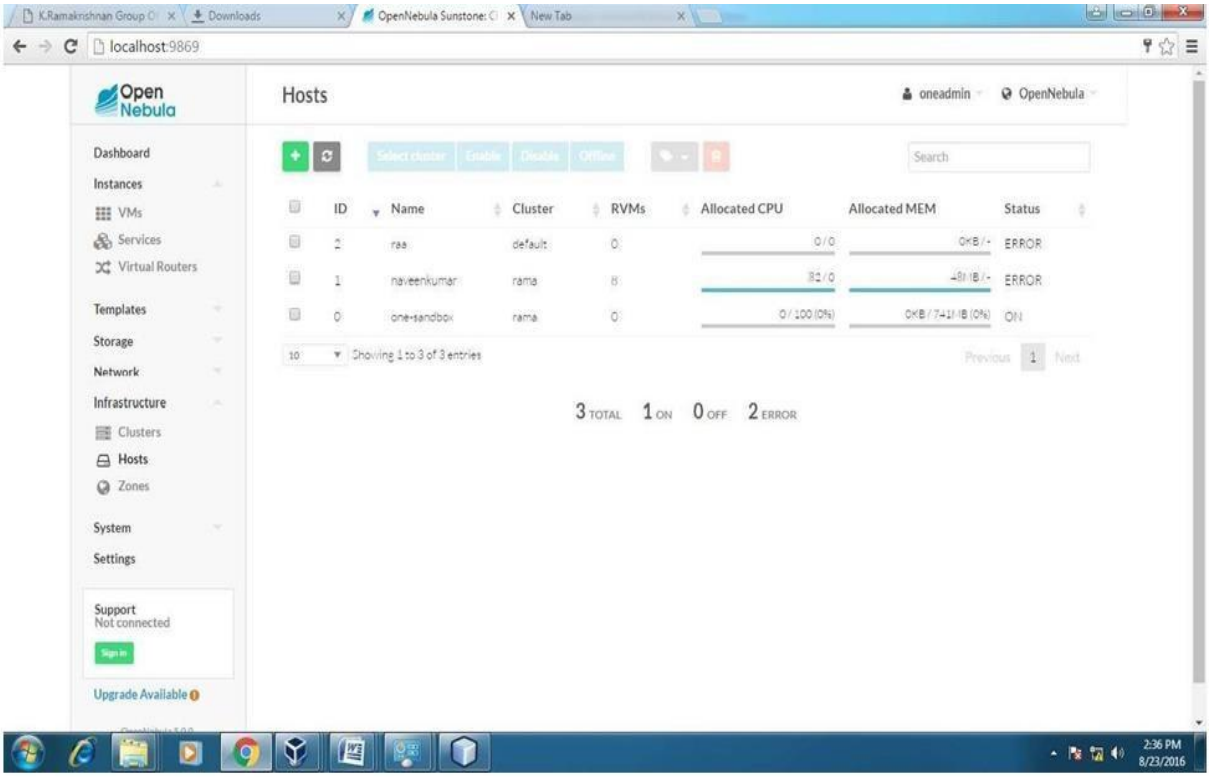
HOST: ONE-SANDBOX



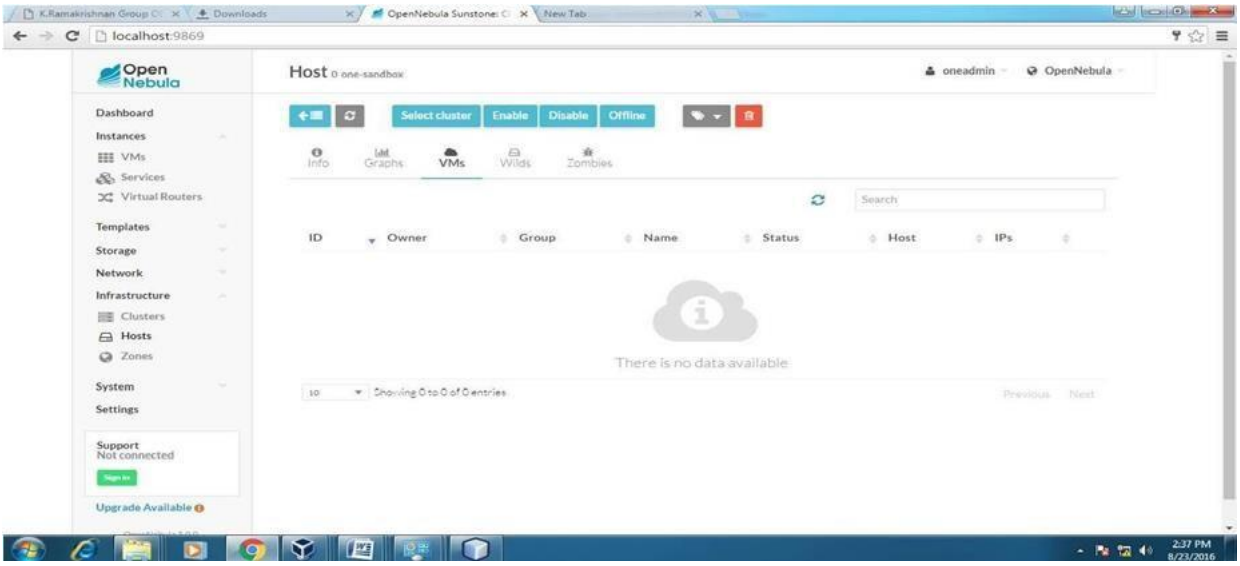


AFTER MIGRATION:





HOST:ONE-SANDBOX



RESULT:

Thus the file transfer between VM was successfully completed.

EX .NO -7 **INSTALL HADOOP SINGLE NODE CLUSTER AND RUN**
DATE: **SIMPLE APPLICATIONS LIKE WORD COUNT**

AIM:

To install hadoop single node cluster and run simple applications like word count

PROCEDURE:

1. Download Cloudera Quickstart VM and start it up in VirtualBox or VMWare Fusion.
2. Open the terminal of cloudera VM
3. Before running WordCount example, we need to create some input text file, then move it to HDFS. First, create an input test file in your local file system.
4. Next, we need to move this file into HDFS. The following commands are the most basic HDFS commands to manage files in HDFS. In order of appearance below, we create a folder, copy the input file from local filesystem to HDFS, and list the content on HDFS.
5. It should be noted that for a fresh Cloudera VM, there is a "/user" folder in HDFS but not in the local filesystem. This example illustrates that local file system and HDFS are separate, and the Linux's "ls" and HDFS's "ls" interact with those independently.
6. Next, we want to run some MapReduce example, such as WordCount. The WordCount example is commonly used to illustrate how MapReduce works. The example returns a list of all the words that appear in a text file and the count of how many times each word appears. The output should show each word found and its count, line by line.
7. We need to locate the example programs on the sandbox VM. On Cloudera Quickstart VM, they are packaged in this jar file "hadoop-mapreduce-examples.jar". Running that jar file without any argument will give you a list of available examples.
8. To run the WordCount example using the input file that we just moved to HDFS, use the following command:

```
[cloudera@quickstart temp]$ hadoop jar /usr/lib/hadoop-mapreduce/hadoop-mapreduce-examples.jar wordcount /user/cloudera/input/wordcount.txt /user/cloudera/output
```

9. The output folder is specified as "/user/cloudera/output" in the above command. Finally, check the output of WordCount example in the output folder.

COMMANDS:

```
[cloudera@quickstart ~]$ mkdir temp
[cloudera@quickstart ~]$ cd temp
[cloudera@quickstart temp]$ ls -ltr
total 0
[cloudera@quickstart temp]$ echo "this is the file which iss
used to count the word for the program which is simulated for
the hadoop exercise" > wordcount.txt
[cloudera@quickstart temp]$ hdfs dfs -mkdir /user/cloudera/input
[cloudera@quickstart temp]$ hdfs dfs -ls /user/cloudera/input
[cloudera@quickstart temp]$ hdfs dfs -put
/home/cloudera/temp/wordcount.txt/user/cloudera/input
put: `/home/cloudera/temp/wordcount.txt/user/cloudera/input': No
such file or directory
 [cloudera@quickstart temp]$ hdfs dfs -put /home/cloudera
/temp/wordcount.txt/user/cloudera/input
put: `/temp/wordcount.txt/user/cloudera/input': No such file or
directory
[cloudera@quickstart temp]$ hdfs dfs -put
/home/cloudera/temp/wordcount.txt/user /cloudera/input
put: `/cloudera/input': No such file or directory
[cloudera@quickstart temp]$ hdfs dfs -put
/home/cloudera/temp/wordcount.txt  /user/cloudera/input
[cloudera@quickstart temp]$ hdfs dfs -ls /user/cloudera/input
Found 1 items
-rw-r--r--   1 cloudera cloudera         109 2017-09-23 01:20
/user/cloudera/input/wordcount.txt
[cloudera@quickstart temp]$ ls -ltr /usr/lib/hadoop-mapreduce/
```

```
total 29424
-rw-r--r-- 1 root root   94672 Jun 29 04:59 xz-1.0.jar
-rw-r--r-- 1 root root   15010 Jun 29 04:59 xmlenc-0.52.jar
-rw-r--r-- 1 root root   23346 Jun 29 04:59 stax-api-1.0-2.jar
-rw-r--r-- 1 root root  995968 Jun 29 04:59 snappy-java-
1.0.4.1.jar
-rw-r--r-- 1 root root  105112 Jun 29 04:59 servlet-api-2.5.jar
-rw-r--r-- 1 root root  533455 Jun 29 04:59 protobuf-java-
2.5.0.jar
-rw-r--r-- 1 root root   29555 Jun 29 04:59 paranamer-2.3.jar
-rw-r--r-- 1 root root   64661 Jun 29 04:59 okio-1.4.0.jar
-rw-r--r-- 1 root root  319099 Jun 29 04:59 okhttp-2.4.0.jar
-rw-r--r-- 1 root root 1419869 Jun 29 04:59 mockito-all-
1.8.5.jar
-rw-r--r-- 1 root root  582965 Jun 29 04:59 microsoft-
windowsazure-storage-sdk-0.6.0.jar
-rw-r--r-- 1 root root   85448 Jun 29 04:59 metrics-core-
3.0.2.jar
-rw-r--r-- 1 root root  489884 Jun 29 04:59 log4j-1.2.17.jar
-rw-r--r-- 1 root root  245039 Jun 29 04:59 junit-4.11.jar
-rw-r--r-- 1 root root   33031 Jun 29 04:59 jsr305-3.0.0.jar
-rw-r--r-- 1 root root  100636 Jun 29 04:59 jsp-api-2.1.jar
-rw-r--r-- 1 root root  185746 Jun 29 04:59 jsch-0.1.42.jar
-rw-r--r-- 1 root root  177702 Jun 29 04:59 jetty-util-
6.1.26.cloudera.4.jar
-rw-r--r-- 1 root root  540685 Jun 29 04:59 jetty-
6.1.26.cloudera.4.jar
-rw-r--r-- 1 root root   67758 Jun 29 04:59 jettison-1.1.jar
-rw-r--r-- 1 root root  539735 Jun 29 04:59 jets3t-0.9.0.jar
-rw-r--r-- 1 root root  713089 Jun 29 04:59 jersey-server-
1.9.jar
```

```
[cloudera@quickstart temp]$ hadoop jar /usr/lib/hadoop-
mapreduce/hadoop-mapreduce-examples.jar
An example program must be given as the first argument.
Valid program names are:
  aggregatewordcount: An Aggregate based map/reduce program that
counts the words in the input files.
  aggregatewordhist: An Aggregate based map/reduce program that
computes the histogram of the words in the input files.
  bbp: A map/reduce program that uses Bailey-Borwein-Plouffe to
compute exact digits of Pi.
  dbcount: An example job that count the pageview counts from a
database.
  distbbp: A map/reduce program that uses a BBP-type formula to
compute exact bits of Pi.
  grep: A map/reduce program that counts the matches of a regex
in the input.
  join: A job that effects a join over sorted, equally
partitioned datasets
  multifilewc: A job that counts words from several files.
  pentomino: A map/reduce tile laying program to find solutions
to pentomino problems.
  pi: A map/reduce program that estimates Pi using a quasi-Monte
Carlo method.
  randomtextwriter: A map/reduce program that writes 10GB of
random textual data per node.
  randomwriter: A map/reduce program that writes 10GB of random
data per node.
  secondarysort: An example defining a secondary sort to the
reduce.
[cloudera@quickstart temp]$ hadoop jar /usr/lib/hadoop-
mapreduce/hadoop-mapreduce-examples.jar wordcount
/user/cloudera/input/wordcount.txt /user/cloudera/output
17/09/23 01:26:03 INFO client.RMProxy: Connecting to
ResourceManager at /0.0.0.0:8032
17/09/23 01:26:04 INFO input.FileInputFormat: Total input paths
to process : 1
17/09/23 01:26:04 INFO mapreduce.JobSubmitter: number of
splits:1
17/09/23 01:26:04 INFO mapreduce.JobSubmitter: Submitting tokens
for job: job_1506154437319_0001
17/09/23 01:26:05 INFO impl.YarnClientImpl: Submitted
application application_1506154437319_0001
17/09/23 01:26:05 INFO mapreduce.Job: The url to track the job:
http://quickstart.cloudera:8088/proxy/application_1506154437319_
0001/
17/09/23 01:26:05 INFO mapreduce.Job: Running job:
job_1506154437319_0001
```

```
Map-Reduce Framework
        Map input records=1
        Map output records=21
        Map output bytes=193
        Map output materialized bytes=180
        Input split bytes=130
        Combine input records=21
        Combine output records=15
        Reduce input groups=15
        Reduce shuffle bytes=180
        Reduce input records=15
        Reduce output records=15
        Spilled Records=30
        Shuffled Maps =1
        Failed Shuffles=0
        Merged Map outputs=1
        GC time elapsed (ms)=206
        CPU time spent (ms)=1330
        Physical memory (bytes) snapshot=359673856
        Virtual memory (bytes) snapshot=3015249920
        Total committed heap usage (bytes)=226627584
Shuffle Errors
        BAD_ID=0
        CONNECTION=0
        IO_ERROR=0
        WRONG_LENGTH=0
        WRONG_MAP=0
        WRONG_REDUCE=0
File Input Format Counters
        Bytes Read=109
          File Input Format Counters
                Bytes Read=109
          File Output Format Counters
                Bytes Written=114
    [cloudera@quickstart temp]$ hdfs dfs -ls /user/cloudera/output/
    Found 2 items
    -rw-r--r--   1 cloudera cloudera          0 2017-09-23 01:26
    /user/cloudera/output/ SUCCESS
    -rw-r--r--   1 cloudera cloudera        114 2017-09-23 01:26
    /user/cloudera/output/part-r-00000
    [cloudera@quickstart temp]$ hdfs dfs -ls
    /user/cloudera/output/part-r-00000
    -rw-r--r--   1 cloudera cloudera        114 2017-09-23 01:26
    /user/cloudera/output/part-r-00000
    [cloudera@quickstart temp]$ hdfs dfs -cat
    /user/cloudera/output/part-r-00000
    count   1
```

OUTPUT:

```
count  1
exercise 1
file 1
for 2
hadoop 1

is 2

iss 1

program 1

simulated 1 the 4

this 1 to 1
used 1
which     2
word 1
[cloudera@quickstart temp]$
```

RESULT:

Thus the installation of single node Hadoop cluster and executing the word count application is successfully completed.

EX.NO – 8 **CREATING AND EXECUTING YOUR FIRST CONTAINER**
DATE: **USING DOCKER**

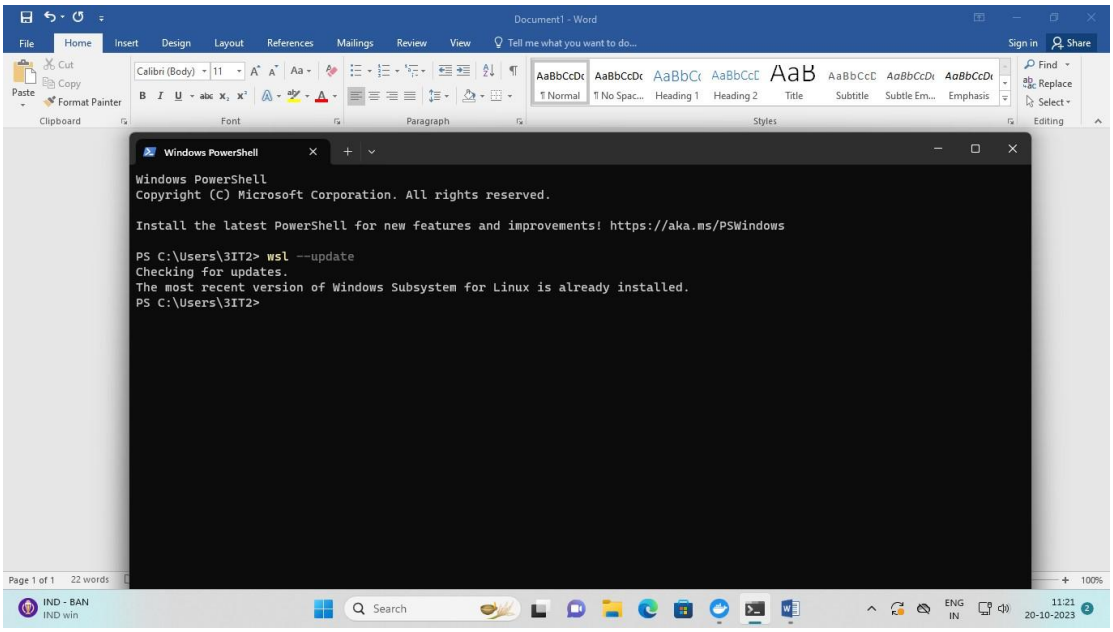
AIM:

To create and execute your first container using Docker.

PROCEDURE:

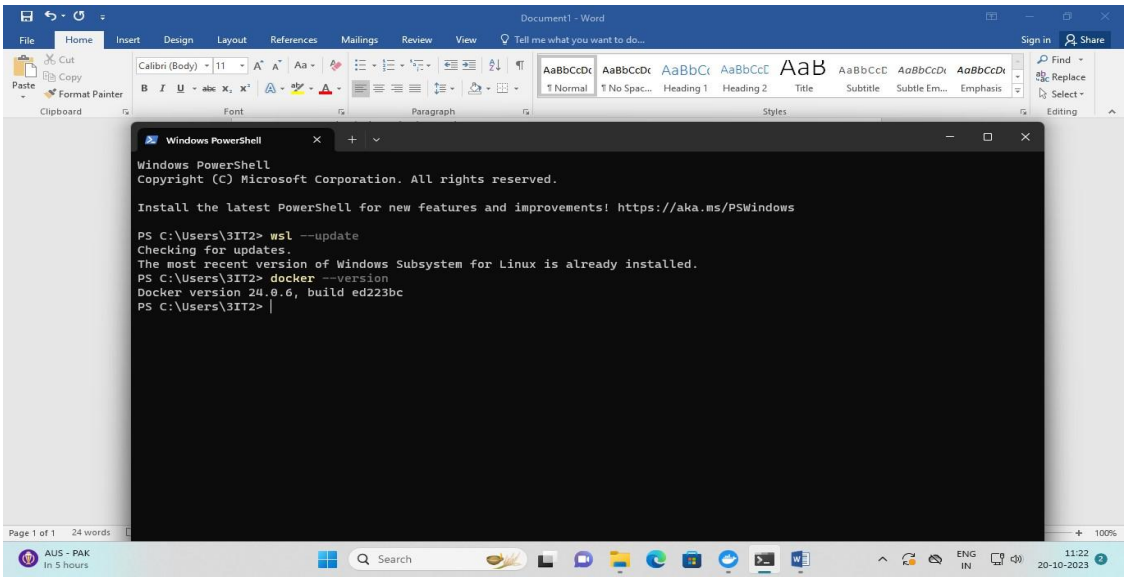
1. Install docker desktop for windows
2. Open the docker and run as administrator
3. Run the docker in background
4. Open power shell and type the following commands

wsl - - update



5. Check the docker version using this command

docker - - version



docker pull ubuntu
docker run –it –d ubuntu

```
PS C:\Users\3IT2> docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
Digest: sha256:2b7412e6465c3c7fc5bb21d3e6f1917c167358449fecac8176c6e496e5c1f05f
Status: Image is up to date for ubuntu:latest
docker.io/library/ubuntu:latest

What's Next?
  1. Sign in to your Docker account → docker login
  2. View a summary of image vulnerabilities and recommendations → docker scout quickview ubuntu
PS C:\Users\3IT2> docker run -it -d ubuntu
01cb94a2eec369ebf496dd8b446fbeff6513f7e9519f1ca266239b3dc090f2fd
```

6. Create an account in docker repository
7. Create your own image using the Ubuntu container using the below command

Docker commit “id” username/ubuntu

```
PS C:\Users\3IT2> docker ps -a
CONTAINER ID   IMAGE     COMMAND       CREATED         STATUS         PORTS     NAMES
01cb94a2eec3   ubuntu    "/bin/bash"   2 minutes ago   Up 2 minutes             awesome_kepler
PS C:\Users\3IT2> docker commit 01cb94a2eec3 salmanahmed500/ubuntu
sha256:e0517a7984ec3d1faffb269930f0878c1f3962c592a895fe5a94898d4167dd3d
```

8. Create your own container using the Ubuntu by the below command
   Docker run –it –d username/ubuntu

```
PS C:\Users\3IT2> docker  images
REPOSITORY              TAG       IMAGE ID       CREATED          SIZE
salmanahmed500/ubuntu   latest    e0517a7984ec   2 minutes ago    77.8MB
<none>                  <none>    95d7c58a2f43   28 minutes ago   77.8MB
ubuntu                  latest    e4c58958181a   2 weeks ago      77.8MB
hello-world             latest    9c7a54a9a43c   5 months ago     13.3kB
PS C:\Users\3IT2> docker run -it -d salmanahmed500/ubuntu
c17cce45e0cd89bd908f8b93eb10286643e4fba94fbd12ad2011ce03b9eb808c
PS C:\Users\3IT2> docker ps -a
CONTAINER ID   IMAGE                   COMMAND       CREATED         STATUS         PORTS     NAMES
c17cce45e0cd   salmanahmed500/ubuntu   "/bin/bash"   7 seconds ago   Up 6 seconds             confident_lewin
01cb94a2eec3   ubuntu                  "/bin/bash"   5 minutes ago   Up 5 minutes             awesome_kepler
```

9. Now the new container is created eg-“c17cce45e0cd”

RESULT:

Thus the above program to create and execute your first container using Docker is successfully executed.

EX.NO – 9 **RUN A CONTAINER FROM DOCKER HUB**

DATE:

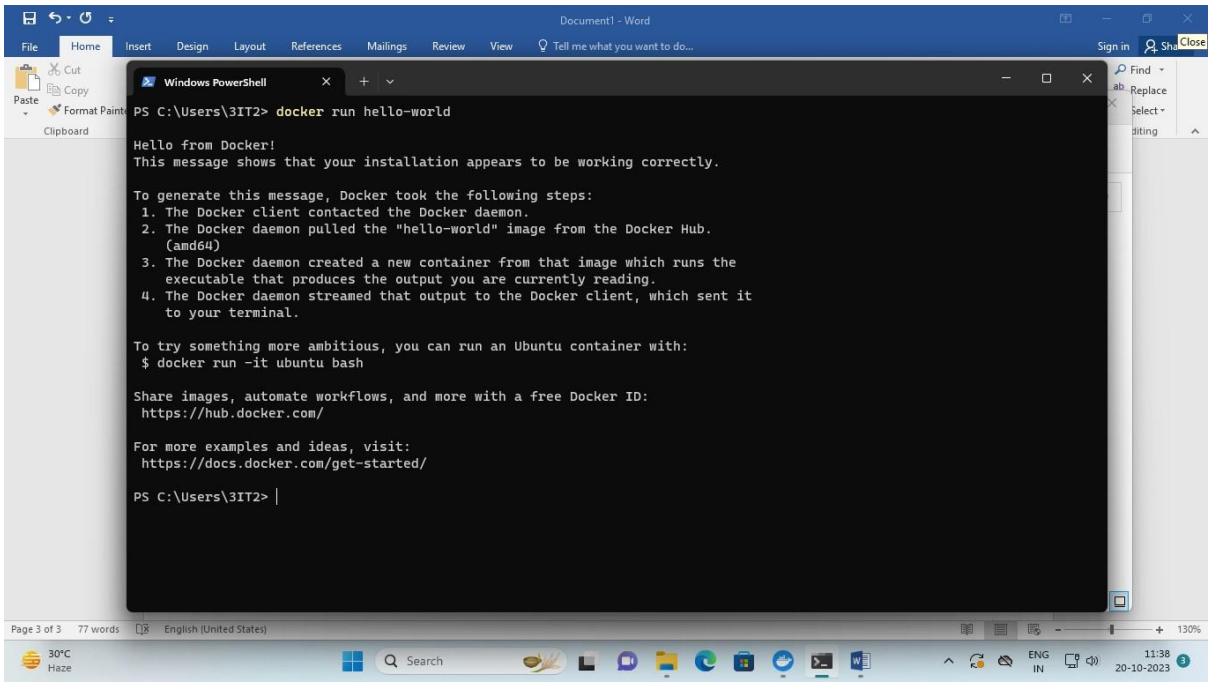
AIM:

To run a container from docker hub

PROCEDURE:

1. To run a container from docker hub, we need to run the image from docker hub
2. Run the image hello-world form the docker hub using the below command

docker run hello-world



3. Now the image is created
4. list the image using the below command **docker images**
5. The container is created with the image hello-world
6. list the container using the below command **docker ps -a**

```
PS C:\Users\3IT2> docker images
REPOSITORY              TAG       IMAGE ID       CREATED          SIZE
salmanahmed500/ubuntu   latest    e0517a7984ec   12 minutes ago   77.8MB
<none>                  <none>    95d7c58a2f43   38 minutes ago   77.8MB
ubuntu                  latest    e4c58958181a   2 weeks ago      77.8MB
hello-world             latest    9c7a54a9a43c   5 months ago     13.3kB
PS C:\Users\3IT2> docker ps -a
CONTAINER ID   IMAGE                     COMMAND       CREATED          STATUS                      PORTS     NAMES
0a87564a6f3a   hello-world               "/hello"      5 minutes ago    Exited (0) 5 minutes ago              trusting_maye
r
c17cce45e0cd   salmanahmed500/ubuntu     "/bin/bash"   10 minutes ago   Up 10 minutes                         confident_lew
in
01cb94a2eec3   ubuntu                    "/bin/bash"   15 minutes ago   Up 15 minutes                         awesome_keple
r
```

7. Now the container is created from the image hello-world Eg-id " 0a87564a6f3a"

RESULT:

Thus the above program to run a container from docker hub is successfully executed.